ENGINEERING REPORT / REVISION 06

ESP32-CAM FIREFIGHTER

From a schematic to a robot that follows a line and puts out a flame.

Afonso Teixeira · LETI, ISEP

PHASE I / REMOTE CONTROL

Wi-Fi drive system, live video transmission and computer-vision line following.

PHASE II / FIREFIGHTER

Raspberry Pi 5 inference, flame localization, pan-tilt aiming and water actuation.

TECHNICAL DOCUMENTATION / 2026

Contents

Executive summary 4

PART I · Electronics fundamentals 6

1. Reading the schematic: net labels ... 6
2. Driving the motors: PWM, the H-bridge and the L293D ... 9
3. Inside an ADC: successive approximation ... 14

PART II · The OV2640 sensor and image acquisition 17

4. The sensor: from photons to bytes ... 17
5. Two conversations: SCCB and the DVP bus ... 20
6. JPEG: why the image is compressed before it leaves ... 22
7. Capture: the I2S peripheral, DMA and the driver API ... 25
8. Getting the frame off the board: MJPEG or raw TCP ... 29

PART III · The ESP32 firmware 32

9. Two states: configuration and operation ... 32
10. The setup() flow ... 34
11. The loop() flow ... 34
12. NVS: how the credentials survive a power cut ... 36
13. Memory: fixed buffers instead of String ... 37
14. Pointers and string functions in C ... 39
15. Buffer handling: process everything, or only the last? ... 40
16. Improvement log ... 41

PART IV · Networking and remote control 44

17. How the ESP32 serves a session ... 44
18. What a socket really is ... 46
19. One message, four floors down ... 48
20. Reaching the robot from anywhere ... 49
21. The Python control server ... 51
22. Choosing and joining the strongest network ... 53
23. Latency and the FIFO queue ... 54
24. Why the video moved from TCP to UDP ... 54

PART V · Autonomy and image processing 56

25. What line following is, and what it is not ... 56
26. The vision pipeline ... 57
27. Morphology: erosion, dilation and opening ... 58
28. Proportional control, and how to compute Kp ... 60
29. Sharp corners and line recovery ... 62

PART VI · Phase I reference and handover 65

30. System reference ... 65
31. The safety mechanisms ... 66
32. Function reference ... 67
33. Test scenarios ... 68
34. Results, limitations and where Phase I ended ... 70

PART VII · Planning the second phase 72

35. The open questions ... 72
36. Why the model has to be tested first ... 73

PART VIII · Neural networks and fire detection 75

37. From pixels to a decision ... 75
38. How a network learns ... 81

39. From a feature map to a decision 85
40. The dataset 86
41. FireNet, training and export 87

PART IX · The Raspberry Pi and putting the fire out 91

42. PWM on the Raspberry Pi 5 91
43. The water system: how far the jet reaches 92
44. The program on the Raspberry Pi 96
45. Stopping the car: the GO/STOP wire 97
46. Aiming the nozzle 98
47. Spraying, and the reflections it creates 99
48. One settings file, and the fast test mode 100
49. Results and limitations of Phase II 101

Appendix A · Technical reference notes 102

Executive summary

This report documents the ESP32-CAM Firefighter: a small car that follows a line on the floor, notices a flame with its own camera and neural network, stops, aims a water nozzle at the flame and sprays it, and then carries on. Phase I built the vehicle: Wi-Fi driving from a PC with an Xbox controller, live video and computer-vision line following.

Phase II turned that vehicle into an experimental firefighting robot. A Raspberry Pi 5 with a second camera runs FireNet, a small detection network trained on images recorded with the robot itself. When it sees a flame it holds the car still through a wire to the ESP32, moves a pan/tilt mechanism until the nozzle points at the flame, and fires short pulses from a water pump.

HOW TO USE THIS REPORT

It is written to be read in order, because each part uses what the previous one established: the pin numbers of Part I are what Part II configures, the frames Part II captures are what Part V analyses, and the model Part VIII trains is what Part IX aims. Every chapter opens by saying what it rests on.

- For the mechanism behind something, read the chapter. Each one works from a concrete example, with the numbers computed.
- For a value or a command, the tables are the reference. The pin map is Table 5.4, the protocol is Listing 30.1, the test scenarios are Chapter 33.
- Historical material is labelled. Where an example comes from an earlier version — TCP control, a 160×160 network — a note at the top of the chapter says so and points to the current one.

THE SYSTEM AT A GLANCE

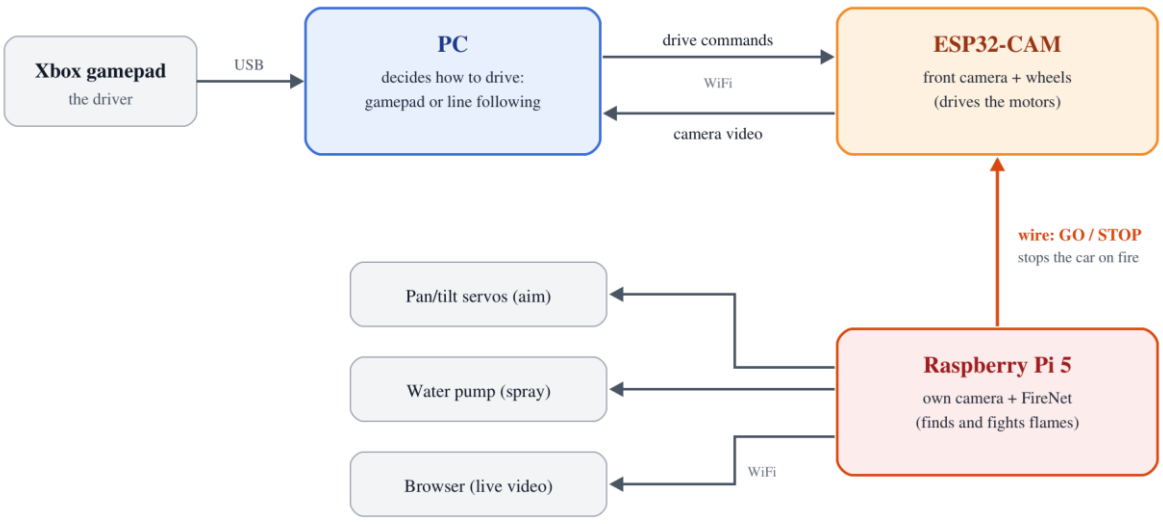


Figure 0.1 — The three computers and what each one decides.

Three computers share the work, and each one decides one thing:

- **ESP32-CAM** — drives the wheels and sends the front camera to the PC.
- **PC** — decides how to drive: from the controller in manual mode, or from the line in the image in automatic mode.
- **Raspberry Pi 5** — decides whether the car may move at all. While it sees a flame it holds the car still, aims and sprays.

When a flame appears, the sequence is always the same:

- The Pi sees the flame in two consecutive frames and pulls the GO/STOP wire to STOP.

- The chassis settles; the Pi confirms the flame over several frames.
- The servos move until the flame sits at the calibrated aim point.
- A short water pulse is fired; it can be retried a limited number of times.
- The camera returns home and, once the scene is clear, the wire goes back to GO.

THE NINE PARTS

PART	TITLE	WHAT IT ANSWERS
I	Electronics fundamentals	Which pin is wired to what, how a logic pin turns a wheel, how a voltage becomes a number.
II	The OV2640 sensor	What the sensor does to light, and how a frame reaches memory without the CPU touching a byte.
III	The ESP32 firmware	The two boot states, setup and loop, and the memory discipline that keeps it running.
IV	Networking and remote control	What a socket is, how a message crosses the internet, and why the video moved to UDP.
V	Autonomy and image processing	How the car follows a line, recovers when it loses it, and the corner it still struggles with.
VI	Phase I reference and handover	The hardware, the protocol, the function reference and the seven test scenarios.
VII	Planning the second phase	The open questions before Phase II, and the argument for the order of answering them.
VIII	Neural networks and fire detection	What a convolution computes, how a network learns, the robot's own dataset, FireNet, its training and its export.
IX	The Raspberry Pi and putting the fire out	The servo signals, the jet range, the program on the Pi, the stop wire, aiming, spraying, and the results.

Table 0.1 — The structure of the report.

PART I

Electronics fundamentals

Everything in this report starts at the board. Before a single line of firmware runs, three physical questions have to be answered: which ESP32 pin is wired to which camera pin, how a 3.3 V logic pin makes a motor turn, and how an analogue voltage becomes a number. This part answers those three, in that order — and the first one ends with the pin numbers written into the camera driver.

> **WHY THIS PART** Phase I started with one goal: drive a car over WiFi and see what it sees. The ESP32-CAM was chosen because it is a WiFi microcontroller and a camera on one small board. Owning the board was the easy part. To use the camera, the firmware first needs to know how the OV2640 is wired to the ESP32 (Chapter 1); to move, it needs to turn a 3.3 V pin into a spinning motor (Chapter 2). Chapter 3 is background: how a voltage becomes a number, which the next phase needs to measure the battery.

1. Reading the schematic: net labels

The ESP32-CAM is an ESP32 with an OV2640 camera on the same board. That says the camera is there; it does not say how to use it. To get a single image, the firmware has to tell the camera driver which ESP32 pin is wired to which pin of the OV2640, and the board cannot report that by itself. The only place that information exists is the AI-Thinker schematic. This chapter is how to read it — and it ends with the GPIO numbers written into the camera driver.

1.1 WHAT A NET LABEL IS

A net label is a name given to a wire. Instead of drawing a wire across the whole sheet, the engineer writes the same name at two points, and the software treats them as electrically connected.

> **RULE** Two pins carrying the same net label are electrically connected. There is no wire drawn between them — the name replaces the wire. Any component attached to that name is part of the same wire.

Reading one takes four steps:

- Find the label next to a pin — for example `CSI_PCLK` next to the camera's PCLK pin.
- Search for the same label elsewhere on the sheet — next to GPIO22.
- Those two pins are joined by an invisible wire.
- Any component carrying that same name is part of that same connection.

1.2 THE WORKED EXAMPLE: PCLK FROM THE SENSOR TO THE ESP32

The label `CSI_PCLK` leaves pin 8 (PCLK) of the camera, and the same label arrives at GPIO22 of the ESP32. Two labels, one electrical node.

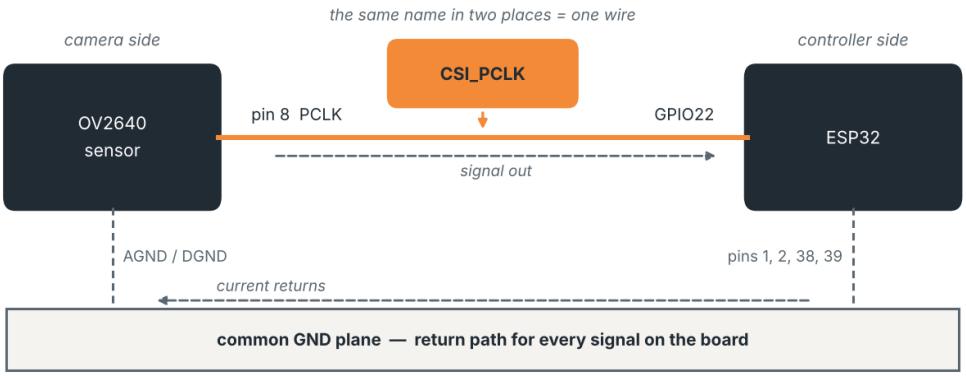


Figure 1.1 — The same name at both ends is one wire. Current returns through the common GND.

> **WATCH OUT** Where a symbol sits on the sheet does not imply a physical order along the wire. The label identifies the node, not a position.

1.3 THE RETURN PATH: HOW THE CURRENT GETS BACK

For current to flow anywhere, it needs a complete path — out and back. With no return path, no current flows at all. This is the one connection a schematic does not redraw for every signal, so it is worth knowing where to look for it.

- The PCLK signal leaves the OV2640 and travels down `CSI_PCLK` to GPIO22.
- The current returns to the OV2640 through the board's common GND.

> **WHERE THIS IS IN THE SCHEMATIC** Not next to CSI_PCLK. It is on the power pins: ESP32 pins 1, 2, 38 and 39 go to GND, and the OV2640's AGND and DGND pins go to GND. One common GND closes the circuit for every signal on the board.

1.4 THE FULL NET LIST

Applying those four steps to every labelled net on the sheet produces the complete map between the sensor and the controller. This is the output of reading the schematic, and the input to everything that follows.

OV2640 SIGNAL	ESP32 GPIO	FUNCTION
PWDN	32	Sensor logic power / power down
RESET	-1	No GPIO reset line in the AI-Thinker configuration
XCLK	0	External clock, generated by the ESP32
SIO_D / SCCB SDA	26	Sensor configuration data (bidirectional)
SIO_C / SCCB SCL	27	Clock of the configuration interface (ESP32 → sensor)
Y2 / D0	5	Bit 0 of the parallel bus
Y3 / D1	18	Bit 1 of the parallel bus
Y4 / D2	19	Bit 2 of the parallel bus
Y5 / D3	21	Bit 3 of the parallel bus (connector pin 5)
Y6 / D4	36	Bit 4 of the parallel bus (connector pin 7)
Y7 / D5	39	Bit 5 of the parallel bus
Y8 / D6	34	Bit 6 of the parallel bus
Y9 / D7	35	Bit 7 of the parallel bus
VSYNC	25	Frame synchronisation
HREF	23	Valid pixels of the line
PCLK	22	Clock that accompanies each output byte

Table 1.1 — Verified against the AI-Thinker ESP32-CAM V1.6 schematic. Y5 is connector pin 5 / GPIO21 and Y6 is pin 7 / GPIO36; camera RESET (pin 19) is unconnected.

Two groups are visible in that table, and they do different jobs. SIO_C and SIO_D are the SCCB pair that configures the sensor's registers. Y2–Y9, VSYNC, HREF and PCLK are the DVP bus that carries the image out of it. XCLK runs in the opposite direction to everything else, from the ESP32 into the sensor.

1.5 FROM THE NET LIST TO THE CODE

This is the payoff. Each row of Table 1.1 becomes one assignment in the camera driver's configuration structure, `camera_config_t`. The GPIO numbers are copied straight across — nothing is derived or guessed:

```
config.pin_pwdn     = 32;
config.pin_reset    = -1;
config.pin_xclk     = 0;
config.pin_sccb_sda = 26;
config.pin_sccb_scl = 27;

config.pin_d0 = 5;    config.pin_d1 = 18;
config.pin_d2 = 19;   config.pin_d3 = 21;
config.pin_d4 = 36;   config.pin_d5 = 39;
config.pin_d6 = 34;   config.pin_d7 = 35;

config.pin_vsync = 25;
config.pin_href  = 23;
config.pin_pclk  = 22;
```

Listing 1.1 — The net list of Table 1.1, written as firmware.

IN THE SCHEMATIC	IN THE CODE
`CSI_PCLK`	the net joining OV2640 pin 8 to GPIO22 → `config.pin_pclk = 22;`
`TWI_SDA`	the net joining SIO_D to GPIO26 → `config.pin_sccb_sda = 26;`
`RESET`	no net on the sheet → `config.pin_reset = -1;` (−1 means “not wired”)

Table 1.2 — Three rows traced end to end, from the drawing to the assignment.

> **WHY THIS MATTERS** The driver has no way to detect the wiring. A single wrong number here produces a camera that initialises without error and returns nothing usable. The schematic is not illustrative — it determines these values.

The excerpt above is only the electrical mapping. The complete configuration also sets the frame size, the JPEG quality, the frame-buffer allocation and the clock settings before the initialisation call; those are covered with the camera driver itself, in Part II.

2. Driving the motors: PWM, the H-bridge and the L293D

Chapter 1 turned a drawing into pin numbers. This chapter is what one of those pins can actually do, and where it falls short: a GPIO is only ever 0 V or 3.3 V, while a motor needs a variable voltage and two directions. PWM solves the first, an H-bridge the second, and the L293D packages two H-bridges into one chip.

2.1 PWM — PULSE WIDTH MODULATION

PWM is a digital signal that switches between 0 V and 3.3 V at high frequency. Instead of varying the voltage continuously, it varies how long the signal stays HIGH. That fraction is the duty cycle.

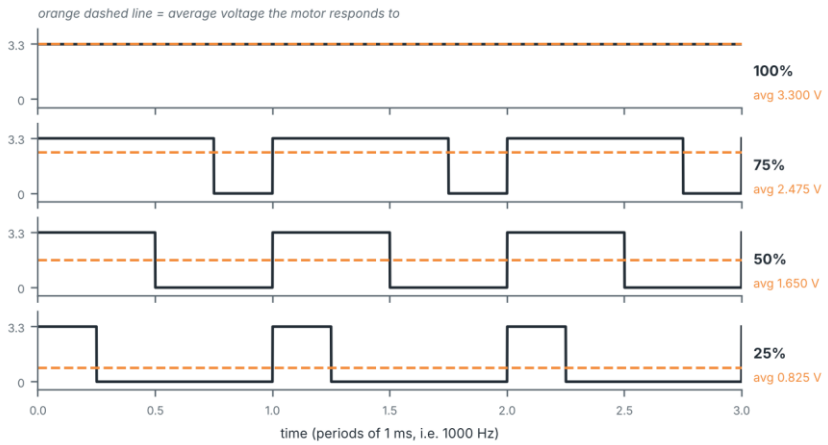


Figure 2.1 — The same 1000 Hz signal at four duty cycles. The dashed line is the average.

DUTY CYCLE	HIGH IN EACH PERIOD	IDEAL AVERAGE, 3.3 V AMPLITUDE
100%	The whole period	3.3 V
75%	Three quarters	2.475 V
50%	Half	1.65 V
25%	One quarter	0.825 V
0%	Never — the output stays LOW	0 V

Table 2.1 — Duty cycle against average voltage.

$$V_{avg} = \frac{D_{\%}}{100} \times V_s$$

for example, at 50%: 0.50 × 3.3 = 1.65 V and at 75%: 0.75 × 3.3 = 2.475 V

At the two extremes the transistor stops pulsing altogether. Between 1% and 99% it switches on and off 1000 times per second; at 100% it is permanently ON, giving maximum speed with no pulsing; at 0% it is permanently OFF and the motor is stopped.

2.2 WHY THE MOTOR RESPONDS TO THE AVERAGE

A DC motor contains a coil: a wound wire that creates a magnetic field when current flows through it. The voltage across a coil is:

$$V_L = L\frac{dI}{dt}$$

$$\frac{dI}{dt} = \frac{V}{L}$$

rearranged. V in volts, L in henry, I in amperes, t in seconds

dI/dt is the rate of change of the current — how many amperes it changes per second. At 1000 Hz each full cycle lasts 1/1000 s = 1 ms, so with a 50% duty cycle:

- The signal rises to 3.3 V and the current tries to rise — but in 0.5 ms it only climbs by a very small amount.
- The signal drops to 0 V and the current tries to fall — and in 0.5 ms it falls by that same small amount.
- The result is a small ripple around an average value.

> **HOW SMALL IS THE RIPPLE** Its real size depends on the motor's parameters, the duty cycle and the frequency. 1000 Hz on its own does not guarantee a nearly constant current.

Why the average specifically? In steady state, the energy stored in the inductance returns to the same value at the end of every cycle. That periodic balance is what pins the current to the value corresponding to the average voltage. The source supplies energy to the mechanical work and to the losses; it does not follow that all the energy taken in during HIGH leaves again during LOW.

2.3 THE H-BRIDGE

An H-bridge is four transistors arranged so that a DC motor can be driven in either direction. The motor turns one way when current flows through it in one direction, and the other way when the current is reversed.

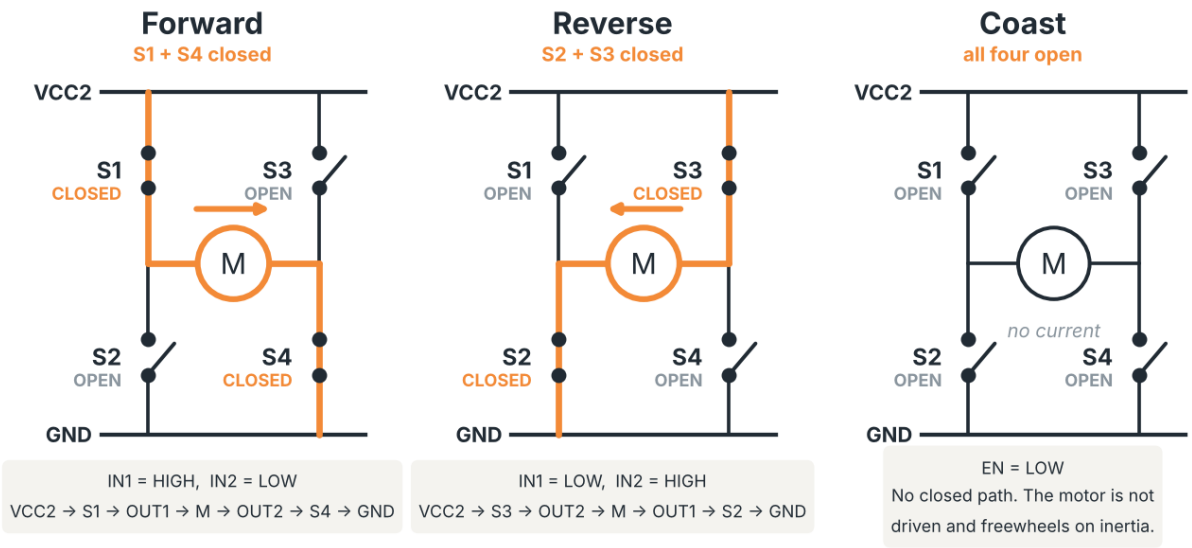


Figure 2.2 — The three switch combinations. The orange path is where current actually flows.

> **COASTING IS NOT BRAKING** With all four switches open nothing drives the motor, but it can keep spinning on its own inertia. That is a free stop, not active braking.

A transistor cannot open halfway — it is binary, ON or OFF. So speed is controlled by applying PWM to one of the switches: S4 is held permanently ON, and S1 is driven by the PWM signal, switching 1000 times per second. The motor then responds to the average voltage, as in section 2.2.

> **SHOOT-THROUGH — NEVER SWITCH ON S1+S2 OR S3+S4 TOGETHER** If S1 and S2 are ON at the same time the path is VCC → S1 → S2 → GND: a short circuit. The current goes straight from VCC to GND without passing through the motor, and it destroys the circuit immediately.

> **ON MAPPING DUTY CYCLE TO SPEED** At 100% the supply stays applied in this configuration. At 50% it is applied for half the period, but the speed is not necessarily half — it depends on the load and the motor. The relationship has to be measured.

2.4 THE L293D — TWO H-BRIDGES IN ONE CHIP

The L293D contains two complete H-bridges in a single integrated circuit, so it can drive two DC motors independently. That is eight transistors in total: S1–S4 for motor A, S5–S8 for motor B.

PIN	FUNCTION	DESCRIPTION
VCC1	Logic supply	5 V for the IN and EN pins
VCC2	Motor supply	Up to 36 V for the motors
EN1	Enable motor A	PWM here controls the speed of motor A
EN2	Enable motor B	PWM here controls the speed of motor B
IN1, IN2	Inputs motor A	Set the direction of motor A
IN3, IN4	Inputs motor B	Set the direction of motor B
OUT1, OUT2	Outputs motor A	Wire to motor A
OUT3, OUT4	Outputs motor B	Wire to motor B

Table 2.3 — L293D pinout.

Why only two inputs per motor, for four transistors? Because each input controls one output of a half-bridge. With EN active, the output follows the logic level of its input, within the driver's voltage drops:

- `IN1 = HIGH` → OUT1 goes near the motor supply; `IN1 = LOW` → OUT1 goes near GND.
- `IN2 = HIGH` → OUT2 goes near the motor supply; `IN2 = LOW` → OUT2 goes near GND.
- The voltage difference between OUT1 and OUT2 is what sets the direction. Equal inputs allow braking; a LOW on EN disables the outputs and lets the motor spin freely.

> **SOURCE** Texas Instruments, L293/L293D datasheet, sections 8.4 and 9.2 — https://www.ti.com/lit/ds/symlink/l293.pdf

EN1	IN1	IN2	RESULT
HIGH	HIGH	LOW	Motor forward
HIGH	LOW	HIGH	Motor reverse
HIGH	LOW	LOW	Motor stops
LOW	X	X	Motor stopped (enable off)
PWM	HIGH	LOW	Forward, with variable speed

Table 2.4 — Truth table for motor A.

2.5 THE ESP32 LEDC PERIPHERAL

The ESP32 has a hardware peripheral called LEDC (LED Control) dedicated to generating PWM, with 16 independent channels. Internally it holds a counter that runs continuously from 0 to 255 (8-bit resolution), and at 1000 Hz that full sweep happens 1000 times per second.

Setting the duty cycle means setting a comparison value:

```
ledcWrite(PWM_CHANNEL_1, 128);   // comparison value = 128

counter <  128  ->  pin = HIGH  (ON)
counter >= 128  ->  pin = LOW   (OFF)
```

Listing 2.1 — The counter holds the output HIGH for the first half of the values and LOW for the second.

$$\text{value} \approx \frac{D_{\%}}{100} \times 255$$

100% → 255 50% → 128 (50.2%) 25% → 64 (25.1%) 0% → 0

Why does this have to be hardware? Because in software the CPU would be trapped in a loop toggling the pin 1000 times a second, unable to do anything else:

```
// WRONG - the CPU is blocked:
while (true) {
    digitalWrite(EN1, HIGH);
    delayMicroseconds(500);
    digitalWrite(EN1, LOW);
    delayMicroseconds(500);
}
```

```
// RIGHT - the hardware does the work:
ledcWrite(PWM_CHANNEL_1, 128);   // the CPU sets it ONCE
// LEDC generates the signal on its own; the CPU is free
```

Listing 2.2 — The same PWM, done by software and by the peripheral.

2.6 THE CODE ON THE ESP32

```
const int EN1_pin = 14;   // Enable motor A (PWM)
const int EN2_pin = 15;   // Enable motor B (PWM)
const int IN1_pin = 12;   // Direction motor A
const int IN2_pin = 13;   // Direction motor A

const int PWM_CHANNEL_1 = 0;   // LEDC channel 0
const int PWM_CHANNEL_2 = 1;   // LEDC channel 1

void pwm_channel_setup() {
    ledcAttachChannel(EN1_pin, 1000, 8, PWM_CHANNEL_1);
    //                pin      freq bits channel
    //                          1000 Hz, 8-bit resolution (0-255)
    ledcAttachChannel(EN2_pin, 1000, 8, PWM_CHANNEL_2);
}
```

Listing 2.3 — Setting up the two PWM channels.

CALL	WHAT IT DOES
`ledcAttachChannel()`	Binds a pin to an LEDC channel and sets its frequency and resolution. After this the peripheral generates the signal on its own.
`ledcWrite()`	Changes the comparison value, which is the duty cycle. The CPU writes it once and moves on.
`digitalWrite()`	Drives a plain logic pin HIGH or LOW. Used here for the direction pins, which do not need PWM.

Table 2.5 — The three calls used in this chapter.

```
// Forward at 50%
digitalWrite(IN1_pin, HIGH);
digitalWrite(IN2_pin, LOW);
ledcWrite(PWM_CHANNEL_1, 128);   // 50% of 255

// Reverse at 100%
digitalWrite(IN1_pin, LOW);
digitalWrite(IN2_pin, HIGH);
ledcWrite(PWM_CHANNEL_1, 255);   // 100%

// Stop
digitalWrite(IN1_pin, LOW);
digitalWrite(IN2_pin, LOW);
ledcWrite(PWM_CHANNEL_1, 0);
```

Listing 2.4 — Direction on the IN pins, speed on the EN pin.

```
void motor_logic(float speed, float steering) {
    // speed:    0.0 to 1.0  (positive = forward, negative = reverse)
    // steering: -1.0 to 1.0 (left / right)

    int pwm = (int)(abs(speed) * 255);   // 0.0-1.0 -> 0-255

    if (speed > 0) {              // forward
        digitalWrite(IN1_pin, HIGH);
        digitalWrite(IN2_pin, LOW);
    } else if (speed < 0) {       // reverse
        digitalWrite(IN1_pin, LOW);
        digitalWrite(IN2_pin, HIGH);
    } else {                      // stop
        digitalWrite(IN1_pin, LOW);
        digitalWrite(IN2_pin, LOW);
        pwm = 0;
    }

    ledcWrite(PWM_CHANNEL_1, pwm);
}
```

Listing 2.5 — motor_logic(): the sign of speed picks the direction, its magnitude sets the duty cycle.

3. Inside an ADC: successive approximation

SCOPE OF THIS CHAPTER The ATmega328P is used here as a study example of analogue-to-digital conversion. The current car does not read any analogue value; the next phase will, to measure the battery voltage through a divider so the robot knows when to return to its station.

Chapters 1 and 2 went from a schematic to a turning wheel, all in the digital domain. This chapter closes the loop the other way: how a continuous voltage on a pin becomes a number a program can read. It is a bottom-up study of what sits inside the chip when `analogRead()` is called.

3.1 THE THREE BLOCKS INSIDE THE CHIP

BLOCK	ITS PHYSICAL JOB
Sample & Hold	Freezes the input voltage
DAC + comparator	Produces test voltages and compares them against the frozen one
SAR + register	The logic that decides, and stores the result

Table 3.1 — The three blocks, explained one at a time below.

3.2 BLOCK 1 — SAMPLE & HOLD

Sample & Hold takes a snapshot of the input before any comparison happens. The input may keep changing afterwards, but the conversion works on the captured value. On an Arduino with the usual settings a conversion takes roughly 100 µs; the exact time depends on the ADC clock.

- The switch closes for a few microseconds, and the capacitor charges to the current voltage on the pin.
- The switch opens, the capacitor is isolated, and it holds that voltage like a photograph.
- Throughout the conversion, the ADC works on that sampled value.

A simple Sample & Hold model is a switch and a capacitor. In the ATmega328P equivalent circuit the path also includes an input resistance, and the 14 pF capacitor is referenced to VCC/2. The simple drawing leaves out pin leakage and protection.

> **SOURCE** Microchip, ATmega328P datasheet, Analog Input Circuitry.

The idea is the same as taking a photograph before measuring, instead of measuring a moving target.

3.3 BLOCK 2 — DAC AND COMPARATOR

A DAC does the opposite of an ADC: it takes a digital code and produces a corresponding voltage or current. In the successive-approximation process it is what produces the test values to compare against the input.

> **ON THE R–2R LADDER** The R–2R network often drawn for this is a teaching example of a DAC, usually shown with three bits so it stays readable. It should not be read as the confirmed internal circuit of the ATmega328P. Source: Texas Instruments, “What is an R-2R DAC?”

The circuit does not need memorising. What matters is the behaviour: give it a 10-bit binary number and it produces an output voltage between 0 V and 5 V, proportional to that number.

CODE	DAC OUTPUT VOLTAGE
0	0.000 V
512	2.500 V
768	3.750 V
1023	4.995 V

Table 3.2 — Four points of the DAC’s transfer characteristic.

The comparator then compares two voltages and gives a logic result. The triangular symbol with + and − inputs says which of the two is larger. Its output is digital: either 1 or 0, with no middle ground.

> **WATCH OUT** That symbol does not mean an ordinary operational amplifier is suitable for every comparison application.

3.4 BLOCK 3 — THE SAR, STEP BY STEP

The SAR (Successive Approximation Register) is the logic that runs the operation: a 10-bit register and a state machine performing a binary search. Below is the conversion of a real 3.2 V on pin A3. Before it starts, Sample & Hold has captured 3.2 V on the capacitor.

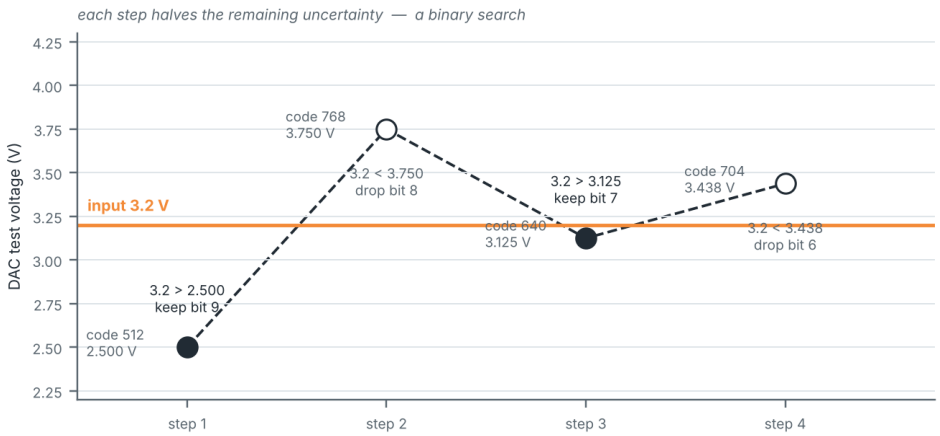


Figure 3.1 — The first four steps of the search for 3.2 V. Filled dot = bit kept, hollow = bit dropped.

STEP	BIT TESTED	REGISTER	DAC VOLTAGE	COMPARATOR AND DECISION
1	bit 9 (512)	1000000000	512/1024 × 5 = 2.500 V	3.2 > 2.500 → output 1 → keep bit 9
2	bit 8 (256)	1100000000 = 768	768/1024 × 5 = 3.750 V	3.2 < 3.750 → output 0 → drop bit 8, back to 512
3	bit 7 (128)	1010000000 = 640	640/1024 × 5 = 3.125 V	3.2 > 3.125 → output 1 → keep bit 7
4	bit 6 (64)	1011000000 = 704	704/1024 × 5 = 3.4375 V	3.2 < 3.4375 → output 0 → drop bit 6, back to 640

Table 3.3 — It carries on like this, bit by bit, from the most significant down to bit 0.

Each step halves the remaining interval of uncertainty, which is exactly a binary search.

3.5 THE RESULT

- After 10 steps the SAR register holds a number between 0 and 1023 representing the input voltage.
- That number is copied into the hardware registers `ADCH` and `ADCL`, which `analogRead()` reads and returns.
- Total conversion time: about 104 µs. Resolution: 5 V / 1024 = 4.9 mV per level.

$$N = \min\left(1023,\ \max\left(0,\ \left\lfloor 1024 \times \frac{V_{in}}{V_{ref}} \right\rfloor\right)\right)$$

ideal model of a ten-bit ADC, for an input inside the reference range

PART II

The OV2640 sensor and image acquisition

Part I ended with sixteen GPIO numbers written into camera_config_t. This part is everything those numbers were for: what the sensor actually does to light, the two separate conversations it holds with the ESP32, why the image leaves as JPEG rather than raw pixels, how 2.5 megabytes per second reach memory without the CPU touching a byte, and how a frame finally leaves the board.

WHY THIS PART With the pin numbers known, the camera still gives nothing until the ESP32 knows how to talk to it and how to collect the image. This part follows the picture from the light on the sensor to a JPEG in memory and out of the board, because that stream is what the PC later drives from.

4. The sensor: from photons to bytes

Before any of the wiring matters, it is worth knowing what is on the other end of it. The OV2640 is not a camera that hands over an image; it is a chain of blocks, and the ESP32 only ever sees the last one.

4.1 A PHOTODIODE MEASURES BRIGHTNESS, NOT COLOUR

The image sensor is an array of millions of photodiodes on a grid. Each one is a semiconductor device that turns photons into electric current: more light means more current. The underlying physics is the photoelectric effect — photons strike silicon atoms and free electrons, and the resulting charge is proportional to the light intensity.

- More light → more photons → more electrons → larger charge → high digital value.
- Less light → fewer photons → fewer electrons → smaller charge → low digital value.

THE CATCH A photodiode measures intensity only. It cannot tell one colour from another — it is like an eye that sees brightness but no colour at all.

4.2 THE BAYER FILTER AND DEMOSAICING

Colour is recovered by putting a physical filter — a piece of coloured glass — in front of each photodiode, so that each one only receives red, green or blue light and measures the intensity of that one colour. The arrangement of those filters is the Bayer pattern, invented by Bryce Bayer at Kodak in 1976.

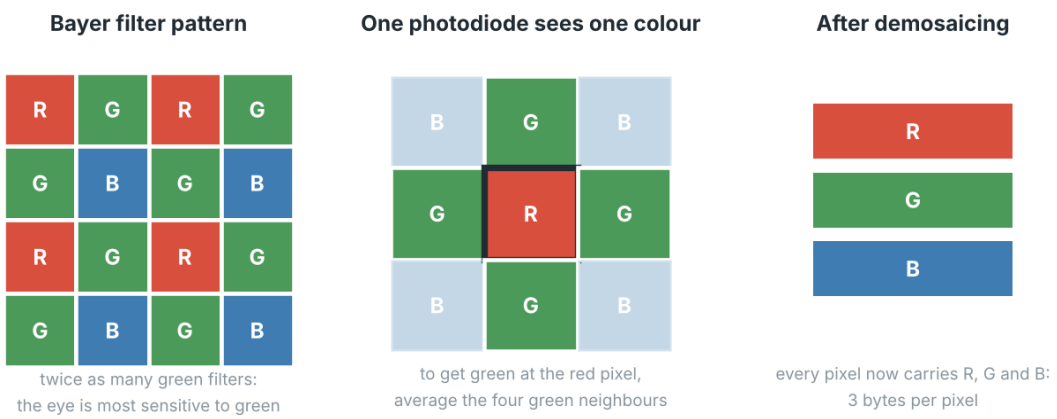


Figure 4.1 — The Bayer pattern, the interpolation at one pixel, and the result.

There are twice as many green filters as red or blue, because the human eye is most sensitive to green — that is where the photoreceptors peak.

Since each photodiode only ever sees one colour, the chip interpolates from the neighbours to produce a full RGB value at every position. That process is called demosaicing. A bilinear approximation of the green value at a red pixel is simply the average of the four green neighbours above, below, left and right:

$$G_{centre} = \frac{G_{up} + G_{down} + G_{left} + G_{right}}{4}$$

After demosaicing every pixel carries R, G and B. A 640 × 480 frame has 307 200 pixels, and at three bytes each that is:

$$640 \times 480 \times 3 = 921\,600 \text{ bytes}$$

roughly 900 kB for a single uncompressed frame

4.3 THE CHAIN INSIDE THE CHIP

The datasheet's functional block diagram shows the path light takes from the lens to the output pins. It reads left to right, in three phases.

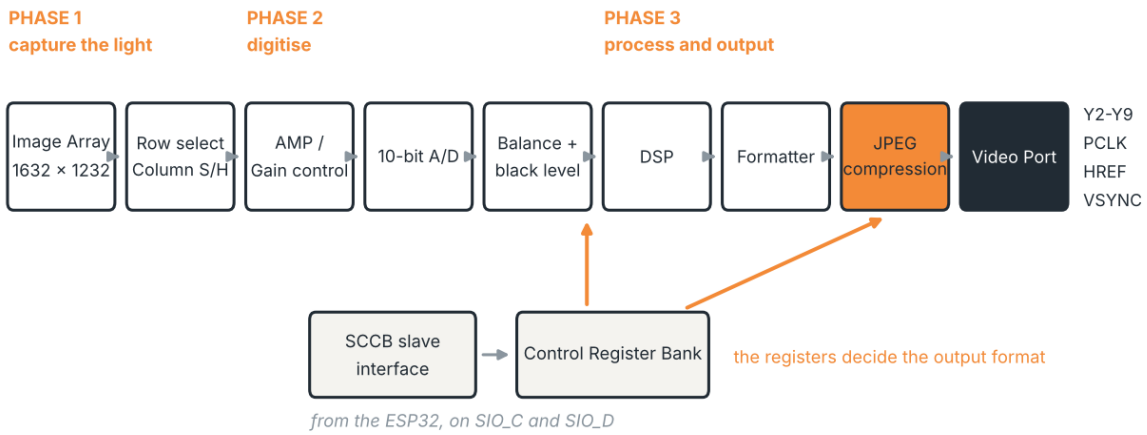


Figure 4.2 — The internal chain of the OV2640. The control side feeds it from below.

BLOCK	WHAT IT DOES
Image Array	The physical pixel matrix, 1632 × 1232, that receives light from the lens
Row select / Column S&H	Choose which row and column of pixels is read at a given microsecond
AMP / Gain control	Amplifies the analogue signal of each pixel; the gain is adjustable through the registers
10-bit A/D	Turns each pixel's voltage into a 10-bit digital number
Balance + black level	Balances the red, green and blue channels (white balance) and removes the background electrical noise so black is really black
DSP	Takes the raw digital numbers and computes sharpness, true colours and interpolation
Formatter	Arranges those pixels into the chosen output format
JPEG compression	If JPEG is enabled, the pixels pass through here and come out as compressed bytes
Video Port	The physical output. Sends the final data on Y2–Y9, with the timing signals HREF, PCLK and VSYNC from the Timing Generator

Table 4.1 — The blocks of Figure 4.2, in order.

4.4 WHO GIVES THE ORDERS

Two more blocks sit below the chain. The SCCB slave interface receives configuration operations from the ESP32, and it gives access to the Control Register Bank. The arrows in the datasheet show that bank driving the Output Formatter — which is exactly why the output format is chosen by writing registers over SCCB, and not by any wire.

That same diagram answers how the available formats are found. The feature list beside it names what the output block supports: Raw RGB, YUV422, RGB565 and compressed data. The reasoning for this project is short:

FORMAT	WHY IT WAS OR WAS NOT CHOSEN
RGB565	5 bits red, 6 green, 5 blue — 16 bits, so two bytes per pixel. The eight-wire bus carries those two bytes on separate cycles. Heavy to send over Wi-Fi.
Raw RGB / YUV422	Uncompressed or lightly compressed; same bandwidth problem.
Compressed data (JPEG)	Compressed and light. This is the one the project uses.

Table 4.2 — The format decision, made from the datasheet's feature list.

> **A NOTE ON NAMES** The compression engine's internal blocks — DCT, quantisation, entropy encoder, plus the Q-table of quantisation divisors and the H-table of Huffman codes — are stages of JPEG, covered in Chapter 6. Those names on their own do not uniquely identify a file format; what confirms JPEG is the documentation and the format configured in the driver.

> **HOW TO STUDY A DATASHEET** Start at the functional block diagram to understand the blocks and their connections. Then look up the features and the registers needed for the function you want. Also check the electrical limits — voltages, timings and supply requirements. There is no need to memorise the whole document, but those limits decide whether a connection works.

5. Two conversations: SCCB and the DVP bus

A frame is a large number of bytes — a VGA JPEG is perhaps 20 to 50 kB. The engineering question is how those bytes get from the sensor into the ESP32's memory. Answering it requires splitting the problem in two, because these are two completely different conversations, on different pins, with different protocols.

QUESTION 1 — CONFIGURATION	QUESTION 2 — IMAGE DATA
How does the ESP32 configure the sensor? "I want VGA, in JPEG, at 15 fps." Two wires. Slow. These are only orders.	How do the image bytes arrive in real time? VGA at 15 fps is 15 images per second, each up to 50 kB. Eight data wires plus three timing wires. Fast. Continuous data.

Table 5.1 — Once this split is clear, the whole architecture follows from it.

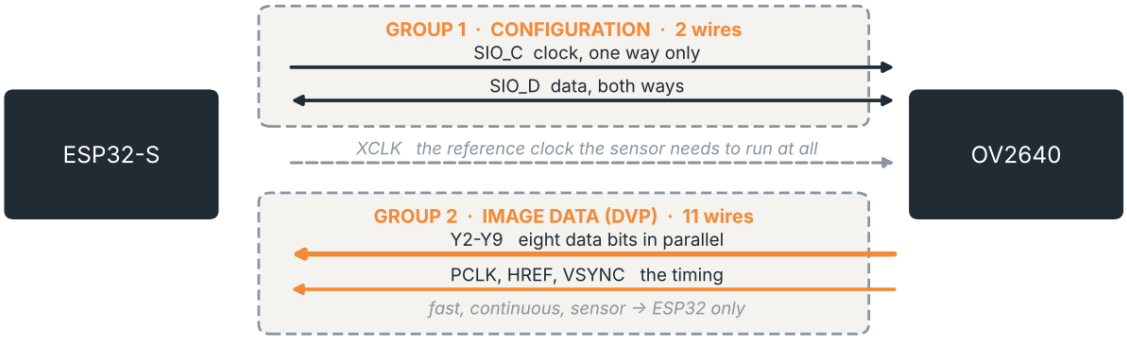


Figure 5.1 — The two groups of wires between the two chips, and the clock the sensor needs.

5.1 GROUP 1 — TWO WIRES FOR CONFIGURATION

The names decompose: SIO means Serial Input/Output, C is clock and D is data. One wire carries the clock, the regular pulses that set the rhythm; the other carries the bits of the message in series, one at a time.

- `SIO_C` goes one way only, ESP32 → sensor, always. The sensor never answers on the clock wire; it only uses it to count at the same rhythm. It is a conductor beating time, and the orchestra does not answer back.
- `SIO_D` is the bidirectional one, but with strict rules: at any moment only one side talks, and the protocol defines exactly who talks when.

On an I²C-like connection a pull-up resistor holds the line high when nobody forces it low, and a device transmits a zero by pulling the line down to GND. The protocol decides which device may drive the line in each phase.

> **SCCB IS NOT I²C** The two have a similar organisation of clock and data, but they are not identical protocols. The SCCB specification includes don't-care bits, and the library handles the details of the transaction, so it must not be assumed that every SCCB phase obeys I²C's ACK rules exactly. The driver adapts the transactions to the sensor; the resemblance does not allow the timing and response differences to be ignored.

Two wires are enough to write and read the OV2640's registers without dedicating a wire to each bit. The benefit is fewer pins and simpler wiring for configuration. The initial setup happens at boot, but later adjustments are possible too.

5.2 GROUP 2 — ELEVEN WIRES FOR IMAGE DATA (DVP)

Eight of them, Y2 to Y9, carry one byte in parallel. That does not mean a complete pixel is always eight bits — RGB565 uses two bytes per pixel, and JPEG is a compressed stream of variable size. The chosen format decides what the bytes mean. A four-bit bus would need more cycles for the same byte; a sixteen-bit bus would use more pins but could carry more per cycle.

> **THREE DIFFERENT NUMBERINGS** The names Y2 to Y9 come from the sensor's interface. In the eight-bit mode used here those eight signals carry the bits D0 to D7 received by the ESP32. The physical pin position on the connector, the sensor's Y name and the GPIO number are three separate numberings — do not mix them.

The other three wires are the punctuation. Without them the ESP32 would receive bytes without knowing where each line or each frame begins.

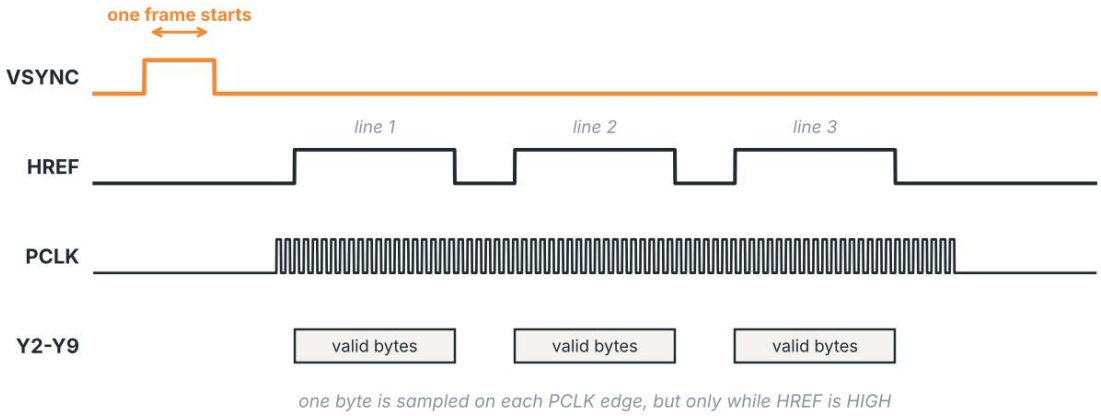


Figure 5.2 — VSYNC marks the frame, HREF marks the valid part of each line, PCLK sets the rhythm.

SIGNAL	WHAT IT SAYS
VSYNC	Pulses once per frame. Rises at the start of each image and falls at the end, so the ESP32 knows when to begin and when to stop capturing.
HREF	HIGH during the valid pixels of each line, LOW in the gap between lines.
PCLK	Tells the receiver when to sample the eight data signals. The driver uses the edge and polarity expected for the sensor's configuration — not both edges indiscriminately, because the data settling times have to be respected.

Table 5.3 — The three timing signals. The whole set, Y2–Y9 plus these three, is called the DVP: Digital Video Port.

> **DO NOT ASSUME THE FRAME SIZE** A VGA image is 640 × 480 pixels, but it does not necessarily contain 307 200 bytes: RGB565 takes two bytes per pixel and JPEG has a variable size. The peripheral and the driver combine the timing signals with the configured format to build each frame.

5.3 XCLK AND PCLK ARE NOT THE SAME CLOCK

- `XCLK` goes from the ESP32 into the sensor. The sensor cannot run without it, and the ESP32 generates it with the LEDC peripheral — the same hardware that drives the motors in Chapter 2.
- `PCLK` comes out of the sensor into the ESP32. It says "read now".

The sensor uses XCLK as a reference, and its own internal clock logic — including a PLL and configurable dividers — determines the capture timing. The two have different jobs, and it must not be assumed that they share a frequency. The sensor accepts 6 to 24 MHz on XCLK; this project uses 20 MHz.

5.4 PWDN — THE SENSOR'S OWN SWITCH

PWDN means Power Down. If it is HIGH the sensor is completely off and answers nothing — not SCCB, not DVP. To use the sensor, the ESP32 must drive GPIO32 LOW before any communication.

> **A VERY COMMON MISTAKE** Forgetting this step. The sensor looks dead, but it is only in power-down.

5.5 THE COMPLETE PIN TABLE

This is the table that every `#define` at the top of the firmware comes from. It is the same net list derived in Chapter 1, here with the sensor pin numbers and the job of each signal.

GPIO	SENSOR PIN	SIGNAL	FUNCTION
GPIO0	pin 12	XCLK	Clock ESP32 → sensor, 20 MHz from LEDC. Without it the sensor does not run.
GPIO26	pin 22	SIO_D	SCCB data — bidirectional configuration wire
GPIO27	pin 20	SIO_C	SCCB clock — one way, ESP32 → sensor
GPIO32	pin 17	PWDN	Power down — LOW means the sensor is active
GPIO22	pin 8	PCLK	Pixel clock sensor → ESP32. One valid byte on Y2–Y9 per pulse.
GPIO23	pin 16	HREF	HIGH during the valid pixels of each line, LOW between lines
GPIO25	pin 18	VSYNC	Pulses once per frame. Marks the start of each image.
GPIO5	pin 3	Y2	Least significant bit of the parallel bus
GPIO18	pin 4	Y3	Parallel bus
GPIO19	pin 6	Y4	Parallel bus
GPIO21	pin 5	Y5	Parallel bus
GPIO36	pin 7	Y6	Parallel bus
GPIO39	pin 9	Y7	Parallel bus
GPIO34	pin 11	Y8	Parallel bus
GPIO35	pin 13	Y9	Most significant bit of the parallel bus

Table 5.4 — Each row is one #define in the firmware. If any number is wrong, the sensor does not communicate; the schematic is the source of truth, never memory.

```
#define XCLK_GPIO_NUM   0    // GPIO0  -> XCLK  (sensor pin 12)
#define SIOD_GPIO_NUM  26    // GPIO26 -> SIO_D (sensor pin 22)
#define SIOC_GPIO_NUM  27    // GPIO27 -> SIO_C (sensor pin 20)
#define PWDN_GPIO_NUM  32    // GPIO32 -> PWDN  (sensor pin 17)
#define PCLK_GPIO_NUM  22    // GPIO22 -> PCLK  (sensor pin  8)
#define HREF_GPIO_NUM  23    // GPIO23 -> HREF  (sensor pin 16)
#define VSYNC_GPIO_NUM 25    // GPIO25 -> VSYNC (sensor pin 18)
#define Y2_GPIO_NUM     5    // GPIO5  -> Y2    (sensor pin  3)
// ... Y3 to Y9 follow the same pattern
```

Listing 5.1 — At this stage a #define is only a text label saying "this name is this number". Nothing is configured as INPUT or OUTPUT yet; the driver does that at init.

6. JPEG: why the image is compressed before it leaves

Chapter 5 established that eleven wires carry the image out. This chapter is about what travels on them, and why it is not raw pixels.

6.1 THE SIZE PROBLEM

FORMAT	SIZE OF ONE 640 × 480 FRAME
RGB, 8 bits per channel	640 × 480 × 3 = 921 600 bytes, about 900 kB
RGB565	640 × 480 × 2 = 614 400 bytes
JPEG	Tens of kilobytes. The exact value depends on the scene and the configured quality, and should be measured with fb->len.

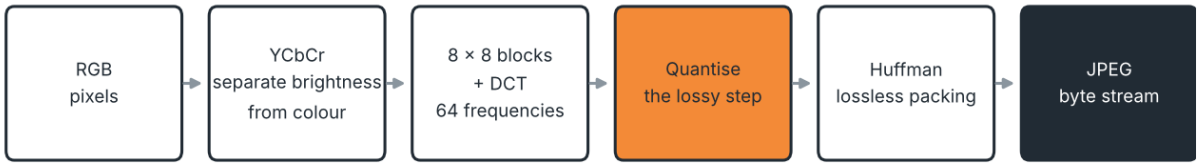
Table 6.1 — The same image, three formats.

- An RGB frame of that size exceeds the ESP32's internal SRAM. The board does have PSRAM, but the available memory and the bandwidth still set limits.
- At 30 frames per second, uncompressed RGB is about 27.6 MB/s before any overhead — a heavy load for this system.
- As a budget example, 30 frames per second of 20 kB is 600 kB/s before overhead. That does not guarantee the complete system can hold that rate.

WHAT COMPRESSION COSTS Uncompressed RGB keeps the processed RGB values; RAW Bayer keeps the sensor samples from before that processing; JPEG discards information in order to shrink. That loss can be visible, particularly on small flames, on edges and on fine textures — which matters directly for the detection work in Part VIII.

The OV2640 compresses in hardware, so the ESP32's CPU is not involved.

6.2 THE FOUR STEPS OF JPEG



only the quantisation step throws information away; everything else is reversible packing

Figure 6.1 — The JPEG pipeline. Only one step throws information away.

STEP 1 — RGB to YCbCr. The eye behaves differently for brightness and for colour: it has many rods for luminance and is very sensitive to it, but few colour cones, so it notices fine colour variation much less. YCbCr separates exactly those two components, so that brightness can be kept at high quality while colour is reduced.

$$Y = 0.299\,R + 0.587\,G + 0.114\,B$$

$$C_b = -0.169\,R - 0.331\,G + 0.500\,B + 128$$

$$C_r = 0.500\,R - 0.419\,G - 0.081\,B + 128$$

Y is luminance; Cb and Cr are the blue and red differences from that brightness

ON SUBSAMPLING Reducing the colour resolution — 4:2:2 or 4:2:0, depending on the mode — can preserve a good appearance with less data, but the difference can be visible. It is not always half, and it is not always imperceptible.

STEP 2 — 8 × 8 blocks and the DCT. Neighbouring pixels in an image are very similar: on a white wall, thousands of pixels hold almost the same value, and storing each one individually is waste. Instead of storing pixel by pixel, JPEG stores the general pattern of each block.

The DCT (Discrete Cosine Transform) converts the 64 pixels of an 8 × 8 block into 64 frequency coefficients. Low frequency means slow variation — general shapes, large outlines, uniform background. High frequency means rapid variation — fine detail, sharp edges, subtle textures.

$$F(u,v) = \frac{C(u)\,C(v)}{4} \sum_{x=0}^{7} \sum_{y=0}^{7} f(x,y) \cos\left[\frac{(2x+1)u\pi}{16}\right] \cos\left[\frac{(2y+1)v\pi}{16}\right]$$

$$C(0) = \frac{1}{\sqrt{2}}, \qquad C(n) = 1 \quad \text{for} \quad n > 0$$

f(x,y) is the sample after subtracting 128

There is no need to memorise it. The concept is that it decomposes the block into a sum of simple patterns — cosines at different frequencies — and each coefficient is the weight of one pattern.

A worked example: a perfectly uniform 8 × 8 block where every sample is 255. In 8-bit JPEG, 128 is subtracted before the DCT, so every sample becomes 127. With the normalisation above, the DC coefficient is:

$$F(0,0) = 8 \times 127 = 1016$$

the DC coefficient represents the constant component — it is not simply the original average of 255

The remaining 63 AC coefficients are zero in this perfectly uniform block. In a nearly uniform block some will be small, but not necessarily zero. Zeros are far easier to compress than 64 distinct values.

STEP 3 — Quantisation, where the discarding happens. After the DCT, each coefficient is divided by a value from the quantisation table and rounded:

$$F_q(u,v) = \text{round}\left[\frac{F(u,v)}{Q(u,v)}\right]$$

COEFFICIENT	TABLE VALUE	RESULT
Low frequency	2	127.5 ÷ 2 = 63.75 → 64, the detail is kept
High frequency	50	0.5 ÷ 50 = 0.01 → 0, the detail is discarded

Table 6.2 — The table values are what the quality setting controls.

> **TWO DIFFERENT QUALITY SCALES** On a generic 1–100 quality scale, raising the quality usually lowers the quantisation and grows the file; that convention depends on the encoder. In the ESP32 driver's jpeg_quality parameter for OV sensors the scale is 0–63 and the sense is inverted: a smaller number means higher quality. Values must never be carried between the two scales.

STEP 4 — Huffman coding. After quantisation there are many zeros, and Huffman compresses this data with no further loss. The principle is that more frequent symbols get shorter binary codes: with fixed 8-bit coding a zero always costs 8 bits, whereas a variable-length code might give the most frequent symbol one bit, the next three bits, the next four, and so on.

> **THE EXAMPLE IS ONLY THE IDEA** JPEG also encodes runs of zeros and the categories of the coefficients, so it does not necessarily assign one bit per zero. The gain from this stage depends on the distribution of the symbols, and there is no fixed guaranteed compression factor.

7. Capture: the I2S peripheral, DMA and the driver API

Chapter 5 described what the pins do: read Y2–Y9 at the rhythm of PCLK, but only while HREF is HIGH, and start a new frame when VSYNC rises. The natural question is whether that code has to be written. It does not — and it could not be.

7.1 WHY THE CPU COULD NEVER KEEP UP

One CPU cycle at 240 MHz lasts about 4 ns. Between two PCLK pulses there are roughly 50 ns, which is about 12 cycles. That sounds like it might be enough until the work is counted:

TO READ ONE BYTE BY HAND	COST
Check the state of HREF	~2 cycles
Read the eight scattered Y2–Y9 pins (they are not one register)	~4 cycles
Write the byte to memory	~3 cycles
Increment the pointer	~1 cycle
Jump back to the start of the loop	~2 cycles
Minimum total	~12 cycles — already at the limit

Table 7.1 — And this ignores compiler overhead, the pipeline and cache misses. The result would be lost bytes and a corrupted image.

> **SO THIS LINE DOES NOT EXIST IN THE FIRMWARE** if (HREF == HIGH) { read_pins(); } — the ESP32's camera peripheral already does it, in hardware.

7.2 WHAT THE HARDWARE DOES ON ITS OWN

The I2S block in camera mode, combined with DMA, does all of this without involving the CPU:

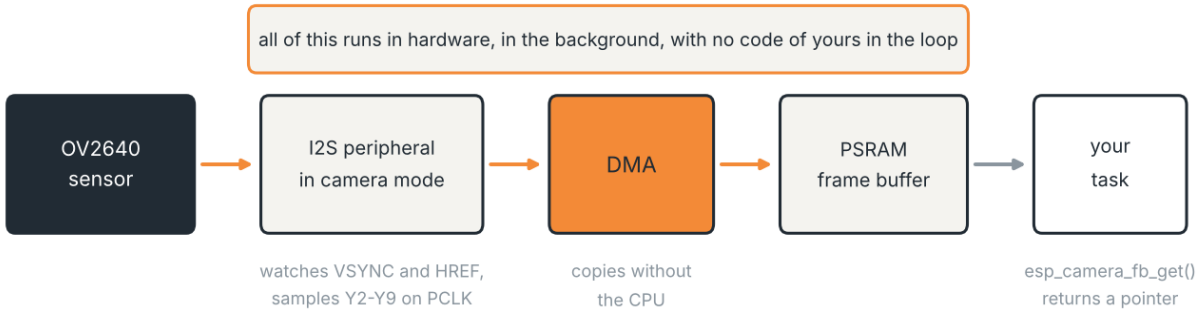


Figure 7.1 — The capture path. Nothing in it runs code of yours.

- It watches VSYNC: when it rises, the frame buffer is reset.
- It watches HREF: while it is HIGH, reading from the pins is enabled.
- At the rhythm of PCLK, it reads the eight bits of Y2–Y9, one byte per pulse.
- Through DMA, it copies those bytes straight into PSRAM without passing through the CPU.
- When the frame is complete, it wakes your code with an interrupt.

DMA means Direct Memory Access: a hardware block that copies data from one place to another without involving the CPU. You tell it "when bytes arrive on these pins, keep copying them to this memory address, and tell me when you are done". The CPU is free to do something else meanwhile.

WHY THE FRAME GOES TO PSRAM AND NOT NORMAL RAM An RGB565 VGA frame is 640 × 480 × 2 = 614 kB, and the ESP32 has only 520 kB of internal SRAM, so it does not fit. The ESP32-CAM board carries 4 MB of external PSRAM (chip U5 on the schematic) over a fast SPI link, and that is where the DMA writes. In JPEG the size is variable and much smaller, but the driver uses PSRAM anyway so as not to compete with the rest of the program for SRAM.

A CAVEAT ON THE DMA PATH DMA transfers data without requiring the CPU to copy each byte, but the driver still uses CPU time to manage buffers and events. The exact path into PSRAM depends on the chip and the driver version; the direct-DMA-to-PSRAM mode documented by the driver applies to the ESP32-S2 and S3. The sequence is: hardware capture → frame-available event → the task obtains the buffer → it sends the data → it returns the buffer. Capture, waiting and sending are not all the same operation.

7.3 THE API YOU ACTUALLY USE

The Espressif esp32-camera driver abstracts everything above. The complete API for capturing an image is two lines:

```
camera_fb_t *fb = esp_camera_fb_get();  // blocks until a frame is ready
                                        // fb->buf           = pointer to the bytes
                                        // fb->len           = size in bytes
                                        // fb->width, height = dimensions

esp_camera_fb_return(fb);               // release the buffer for the next frame
```

Listing 7.1 — The whole capture API.

CALL	WHAT IT DOES
`esp_camera_fb_get()`	Does not read anything at the moment you call it. The hardware has been capturing continuously since init. This call means “give me the next frame that is already ready”, and it blocks if none is complete yet.
`esp_camera_fb_return()`	Means “I no longer need this address, you can reuse it”. Without it, the PSRAM fills and the driver blocks.

Table 7.2 — Why it returns a pointer and not the data: a frame can be 50 kB or more, and copying it would be slow and duplicate the memory. You work on the data where it already is.

7.4 camera_config_t — GROUP 1, THE GPIOs

These are directly the `#define` values from Chapter 5. The driver uses them to configure the I2S peripheral, the LEDC channel for XCLK, and SCCB.

```
#include "esp_camera.h"

camera_config_t config;

config.pin_pwdn     = PWDN_GPIO_NUM;   // GPIO32 - turns the sensor on/off
config.pin_reset    = RESET_GPIO_NUM;  // -1     - not wired on this board
config.pin_xclk     = XCLK_GPIO_NUM;   // GPIO0  - the clock you give the sensor
config.pin_sccb_sda = SIOD_GPIO_NUM;   // GPIO26 - SCCB data
config.pin_sccb_scl = SIOC_GPIO_NUM;   // GPIO27 - SCCB clock

config.pin_d7 = Y9_GPIO_NUM;  // GPIO35 - MSB
config.pin_d6 = Y8_GPIO_NUM;  // GPIO34
config.pin_d5 = Y7_GPIO_NUM;  // GPIO39
config.pin_d4 = Y6_GPIO_NUM;  // GPIO36
config.pin_d3 = Y5_GPIO_NUM;  // GPIO21
config.pin_d2 = Y4_GPIO_NUM;  // GPIO19
config.pin_d1 = Y3_GPIO_NUM;  // GPIO18
config.pin_d0 = Y2_GPIO_NUM;  // GPIO5  - LSB

config.pin_vsync = VSYNC_GPIO_NUM;  // GPIO25
config.pin_href  = HREF_GPIO_NUM;   // GPIO23
config.pin_pclk  = PCLK_GPIO_NUM;   // GPIO22
```

Listing 7.2 — The pin half of the configuration.

7.5 camera_config_t — GROUP 2, THE OPERATING OPTIONS

```
config.xclk_freq_hz = 20000000;   // 20 MHz; the sensor accepts 6 to 24 MHz

// Must not collide with the motors on EN1 and EN2, which already use LEDC.
config.ledc_timer   = LEDC_TIMER_1;
config.ledc_channel = LEDC_CHANNEL_2;

config.pixel_format = PIXFORMAT_JPEG;  // much smaller than RGB, for streaming
config.frame_size   = FRAMESIZE_VGA;   // 640x480; also QVGA (320x240), SVGA (800x600)
config.jpeg_quality = 30;              // 0-63. Lower = better quality = bigger file.
                                       // 30 keeps every frame small enough for UDP.
config.fb_count     = 2;               // two buffers in PSRAM
config.fb_location  = CAMERA_FB_IN_PSRAM;     // required for VGA and above
config.grab_mode    = CAMERA_GRAB_WHEN_EMPTY; // only capture when a buffer is free
// alternative:      CAMERA_GRAB_LATEST       // discard old frames, always have the
newest
```

Listing 7.3 — The behaviour half of the configuration.

THE LEDC COLLISION WITH THE MOTORS PWM and LEDC are the same thing: PWM is the generic concept, LEDC (LED Controller) is Espressif's name for the ESP32 peripheral that generates it. It has 16 channels (8 high-speed and 8 low-speed) driven by 8 timers; the timer sets frequency and resolution, and channels that share a timer share the frequency. The motors use channels 0 and 1 through ledcAttachChannel(), and the core assigns their timer. Forcing the camera onto LEDC_TIMER_1 and channel 2 keeps the 20 MHz XCLK apart from the 1 kHz motor PWM; the timer carrying XCLK must never be reconfigured.

PSRAM MUST BE ENABLED IN THE IDE Tools → PSRAM → Enabled. (OPI PSRAM is the option for ESP32-S3 boards, not for the AI-Thinker ESP32-CAM.) If PSRAM is not active, the init fails even with correct code.

Why `fb_count = 2` and not 1: with a single buffer the DMA has to wait for you to finish sending the frame before it can capture the next one, and the frames the sensor produces meanwhile are discarded, so there is a pause between frames.

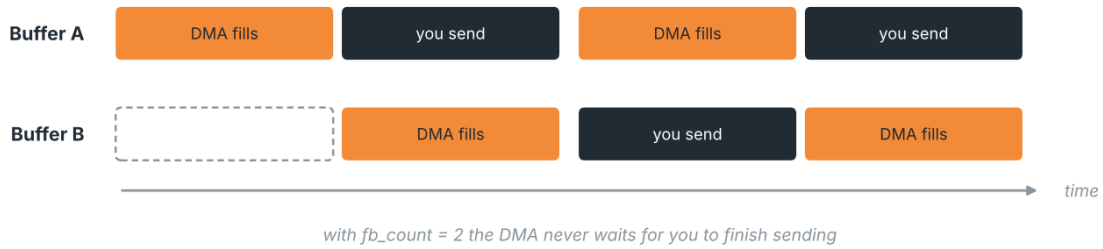


Figure 7.2 — With two buffers, capture and sending overlap, and the frame rate is higher and steadier.

7.6 WHAT THE INIT ACTUALLY DOES

```
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed: 0x%x\n", err);
    // 0x20004 = SCCB failure (wrong GPIO, or PWDN was not driven LOW)
    // 0x20002 = the sensor did not answer (XCLK is not running)
    return;
}
```

Listing 7.4 — The error code tells you which layer the problem is in.

STEP	WHAT esp_camera_init() DOES INTERNALLY
1	Drives PWDN LOW, which activates the sensor
2	Generates XCLK through LEDC, giving the sensor its rhythm
3	Configures the I2S peripheral in camera mode, preparing it to read the DVP pins
4	Talks to the sensor over SCCB and detects the model
5	Writes the registers in the sensor: resolution, format and the rest
6	Starts the DMA, which begins capturing frames in the background

Table 7.3 — If any step fails, the function returns an error code.

> **ORDER MATTERS** esp_camera_init() runs once, in setup(), before any esp_camera_fb_get(). Only after it does the hardware begin capturing. The get/return pair then repeats for every frame.

7.7 WHY THE CAMERA NEEDS ITS OWN TASK

Without an RTOS, the code is a straight line: setup, then loop, loop, loop, one instruction at a time. If the camera needed attention in the middle of a TCP command, it would have to wait, because there is nobody else.

FreeRTOS divides the CPU's time into slices and distributes them among tasks, each with its own loop and its own memory. The ESP32 also has two cores, so part of that parallelism is genuine: two physical processors executing at the same time.

- Core 0, camera task: `esp_camera_fb_get()` may block for as long as it likes.
- Core 1, main loop: the motors keep running while the camera task waits.
- `xTaskCreatePinnedToCore()` is the function that creates a task and pins it to a core.

> **TWO CORES IS NOT ISOLATION** The cores still share memory, peripherals and system resources, so splitting tasks across cores neither removes all interference nor guarantees that nothing blocks.

8. Getting the frame off the board: MJPEG or raw TCP

At the end of Chapter 7 a JPEG frame sits in PSRAM and a pointer to it is in hand. This chapter is the last step: two ways of sending it to the PC, and the reasoning for choosing one.

8.1 OPTION A — HTTP MJPEG STREAM

MJPEG (Motion JPEG) is a video format in which every frame is an independent JPEG. There is no inter-frame compression as in H.264 — each frame is complete. For the ESP32 that is an advantage: it is simple to implement, it needs little memory because only one frame is handled at a time, and browsers support it natively.

MJPEG uses HTTP with the content type multipart/x-mixed-replace, where each frame is separated by a boundary:

```
HTTP/1.1 200 OK
Content-Type: multipart/x-mixed-replace; boundary=frame

--frame
Content-Type: image/jpeg
Content-Length: 15234
[15234 bytes of JPEG]
--frame
Content-Type: image/jpeg
Content-Length: 14891
[14891 bytes of JPEG]
--frame
... continues indefinitely
```

Listing 8.1 — The browser interprets this format automatically and shows live video.

```
void handle_stream(WiFiClient client) {
    client.print("HTTP/1.1 200 OK\r\n");
    client.print("Content-Type: multipart/x-mixed-replace;boundary=frame\r\n");
    client.print("\r\n");

    while (client.connected()) {
        camera_fb_t *fb = esp_camera_fb_get();
        if (!fb) continue;

        client.printf("--frame\r\n");
        client.printf("Content-Type: image/jpeg\r\n");
        client.printf("Content-Length: %u\r\n\r\n", fb->len);
        client.write(fb->buf, fb->len);
        client.print("\r\n");

        esp_camera_fb_return(fb);
    }
}
```

Listing 8.2 — The stream handler: capture, send boundary, send JPEG, release the buffer.

A READY STARTING POINT The Arduino IDE ships a complete example at File → Examples → ESP32 → Camera → CameraWebServer. It already has camera init with all the ESP32-CAM pins, the HTTP server with a /stream endpoint, and web controls for quality, brightness and contrast. Only the camera pins and the Wi-Fi credentials need changing.

Remote access works exactly like the TCP control system: the same public IP and port forwarding. Add a router rule for port 80 alongside the existing one for the control port — a router can hold several rules.

8.2 OPTION B — RAW TCP, INTEGRATED WITH THE CONTROL LINK

TCP is a stream protocol with no concept of messages: the bytes arrive continuously with no separators. The PC needs to know how many bytes belong to the current JPEG, so the size is sent first, as four bytes, before the data.

```
uint32_t size = fb->len;
client_video.write((uint8_t*)&size, 4);  // send the size (4 bytes)
client_video.write(fb->buf, size);       // send the JPEG (N bytes)
```

Listing 8.3 — On the ESP32: length first, then payload.

```
import struct

size_bytes = conn_video.recv(4)
size = struct.unpack('<I', size_bytes)[0]   # '<I' = little-endian uint32

jpeg_data = b''
while len(jpeg_data) < size:
    chunk = conn_video.recv(size - len(jpeg_data))
    jpeg_data += chunk
```

Listing 8.4 — On the PC: read exactly four bytes, then exactly that many bytes.

8.3 THE PIN PROBLEM THAT DECIDES IT

The ESP32-CAM has very few free GPIOs. The camera occupies GPIO 32, 35, 34, 5, 18, 19, 21, 36, 39, 23, 22, 25, 26 and 27; GPIO4 drives the flash LED. That leaves only GPIO 12, 13, 14 and 15 — the same ones the motors already use.

> **GPIO12 AND GPIO13 HAVE BOOT FUNCTIONS** GPIO12 (MTDI) selects the operating voltage of the SPI flash. If GPIO12 is HIGH during boot, the ESP32 tries to use 1.8 V instead of 3.3 V and may fail to start. This can conflict with the L293D wired to these pins.

8.4 THE COMPARISON AND THE DECISION

ASPECT	OPTION A — HTTP MJPEG	OPTION B — RAW TCP
Viewing	A browser, on any device	An OpenCV window, only on the PC running Python
Extra code	~50 lines on the ESP32; an example already exists	~80 lines on the ESP32 plus ~40 in Python
Python integration	Possible, but separate	Fully integrated
Remote access	Yes, port forwarding on port 80	Yes, port forwarding on port 1884
Clash with commands	None; HTTP is separate	Possible, by saturating the link
Implementation time	2–3 hours	5–8 hours
ESP32-CAM pins	No conflict	Possible GPIO 12/13 problems

Table 8.1 — The two options side by side.

> **RECOMMENDATION: OPTION A TO START** A ready example in the Arduino IDE gives a solid base; it can be viewed in a browser from anywhere with no extra code; it does not interfere with the existing TCP control system; there is no risk of pin conflicts; and once it works, integrating with Python is optional and simple.

If the stream is wanted inside Python rather than a browser, OpenCV can read it directly. JPEG files carry markers for start (0xFF 0xD8) and end (0xFF 0xD9), and the client searches the continuous byte stream for them to cut out each frame:

```
import cv2, urllib.request
import numpy as np

url = "http://<public-ip>/stream"
stream = urllib.request.urlopen(url)
bytes_data = b''

while True:
    bytes_data += stream.read(1024)

    a = bytes_data.find(b'\xff\xd8')   # SOI, start of image
    b = bytes_data.find(b'\xff\xd9')   # EOI, end of image

    if a != -1 and b != -1:
        jpeg = bytes_data[a:b+2]        # the complete JPEG
        bytes_data = bytes_data[b+2:]   # drop it from the buffer

        frame = cv2.imdecode(np.frombuffer(jpeg, np.uint8), cv2.IMREAD_COLOR)
        cv2.imshow("ESP32-CAM", frame)

        if cv2.waitKey(1) == ord('q'):
            break

cv2.destroyAllWindows()
```

Listing 8.5 — Reading the MJPEG stream in Python, by cutting on the JPEG markers.

PART III

The ESP32 firmware

Parts I and II built the hardware picture: pins, sensor, capture. This part is the program that runs on top of it. It opens with the two states the robot can boot into and how it chooses between them, follows setup() and loop() line by line, and then goes down one level into the three things that make the difference between firmware that works for five minutes and firmware that works for a season: persistent storage, memory that does not fragment, and parsing that allocates nothing.

> **WHY THIS PART** Camera and motors worked; now they needed a program that joins WiFi on a robot with no screen or keyboard, runs for hours without crashing, and stops the car whenever something goes wrong.

9. Two states: configuration and operation

The robot has no screen and no keyboard, so it cannot be told its Wi-Fi password in the usual way. The whole firmware is shaped around solving that: on every boot it decides which of two states it is in, and everything else follows from that decision.

> **HISTORICAL CONTEXT** The examples in this part come from Phase I and use TCP sockets. They are kept because they explain the mechanism clearly. In the current version, Wi-Fi provisioning still uses TCP, but the ESP32's movement commands and video use UDP.

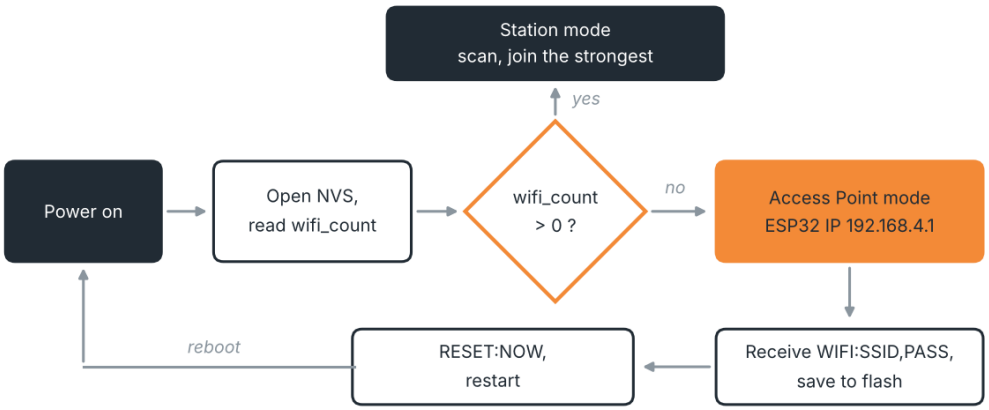


Figure 9.1 — The boot decision and the provisioning loop.

STATE	MODE	WHAT THE ESP32 DOES
A — Configuration	Access Point	It found nothing in flash. It creates its own network and waits for someone to connect and send WIFI:network_name,password. It stores that in flash and then restarts.
B — Operation	Station + client	It found Wi-Fi credentials in flash. It joins the home router and then opens the link to the control PC to receive the motor commands.

Table 9.1 — The two states. wifi_count, stored in flash, is what decides between them.

9.1 FIRST BOOT — NO SAVED NETWORK (wifi_count = 0)

STEP	PHASE	WHAT HAPPENS	KEY CALL
1	Power-on check	In setup(), the NVS is opened and wifi_count is read.	`nvs_get_u32()`
2	Access Point	The ESP32 creates its own Wi-Fi network. Inside that network its own IP is 192.168.4.1 (the default). It then waits for a client.	`ESPserver.begin()`
3	Credentials	The PC joins that network, connects to 192.168.4.1 on port 1883, and sends the message in Listing 9.1.	`WIFI:SSID,PASS`
4	Save	The prefix is identified and the network name is split from the password. Write_to_flash() stores each network under its own index — wifi_0, wifi_1 and so on — building a database of known networks. The ESP32 confirms over the link and keeps listening for more.	`Write_to_flash()`
5	Reboot	The system waits for one specific string. Only on receiving it does it print its goodbye on Serial and restart.	`RESET:NOW`

Table 9.2 — Access Point provisioning, step by step.

```
WIFI:Your_Wifi,Your_Password\n
```

Listing 9.1 — The credentials message, sent to port 1883.

9.2 NORMAL BOOT — A SAVED NETWORK EXISTS (wifi_count > 0)

- `WiFi.scanNetworks()` lists the networks currently in the air.
- Each one is compared against the networks saved in flash.
- The ESP32 joins the strongest match. Chapter 16 covers what happens when that one fails.

9.3 REACHING THE PC ACROSS THE INTERNET

Once associated with the local router, the bridge to the control computer is established. The sequence is worth following precisely, because it is the same path the video takes later.

STAGE	WHAT HAPPENS
The request	The ESP32, acting as a TCP client, uses the live Wi-Fi link to send a packet across the internet to a defined destination: the public IP and port 1883. Note that the Wi-Fi does not switch off — it is the physical road the connection travels on.
Across the WAN	The information travels through the internet provider's servers until it reaches the router at the house.
Port forwarding	At the router the packet is examined. Through the port-forwarding configuration, the router sees that traffic for port 1883 must be pushed internally to the local IP of the PC.
The socket	The PC accepts the connection. At that moment a socket — a dedicated communication channel — is created in the RAM of both devices.

Table 9.3 — From the robot to the PC. Part IV covers each of these stages in depth.

10. The setup() flow

Chapter 9 gave the decision; this is the code that implements it. After the camera has been initialised, setup() opens the NVS namespace and checks for saved networks. From there the program forks, and the fork is more consequential than it looks.

10.1 PATH A — NO SAVED NETWORKS

- `enable_access_point()` then `loop_config_mode()` — and it stays there forever.
- The `nvs_close()` that follows, and the rest of setup(), never run, because `loop_config_mode()` does not return.
- The only way out of this state is a chip reset, triggered by the `RESET:NOW` command.

10.2 PATH B — SAVED NETWORKS EXIST

- `WiFi.scanNetworks()` lists the networks in the air.
- Two nested loops: for each network in the air, compare against each network in flash. When an SSID matches, try to connect with `WiFi.begin()` and wait up to 10 s.
- If it connects, the flag is set and both loops are left. Then `nvs_close()`.
- If none connected, it falls into the same AP plus `loop_config_mode()` as Path A.
- If it did connect, `xTaskCreatePinnedToCore()` creates the camera task on core 0, and setup() ends.

10.3 THE CONSEQUENCE FOR loop()

> **A GUARANTEE WORTH RELYING ON** loop() only ever runs in Path B, and only on success. If the firmware went into AP mode it is stuck in loop_config_mode() and loop() is never called. So whenever loop() is running, it is guaranteed that the Wi-Fi is connected and the camera task already exists — which is why loop() never has to check for either.

11. The loop() flow

loop() runs on core 1, without stopping, and each pass deals with one thing only: keeping the car under safe control. It is organised in layers, from the most basic to the most specific, and a layer that fails sends the pass back to the start without reaching the ones below — there is no point reading commands if there is not even a link.

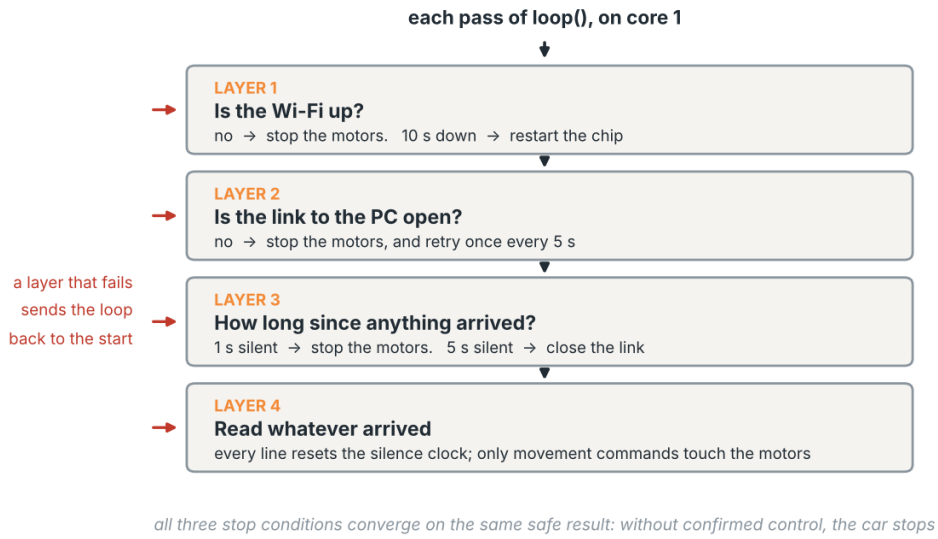


Figure 11.1 — The four layers. Failure at any layer short-circuits the rest of the pass.

11.1 LAYER 1 — IS THE WI-FI UP?

- If it is not: stop the motors and start counting from the moment it went down.
- If 10 s pass without it returning, restart the chip. This is the most serious failure, and there is nothing software can do at that level.
- If the Wi-Fi is up, clear the timestamp so that a future drop is counted again from zero.

11.2 LAYER 2 — IS THE LINK TO THE PC OPEN?

- Only reached with Wi-Fi connected.
- If the link to the PC is down: stop the motors and try to reopen it — but only once every 5 s, because the attempt blocks the program while it waits for an answer.
- It stays in this layer until there is a link.

11.3 LAYER 3 — HOW LONG SINCE ANYTHING ARRIVED?

Reached with Wi-Fi and the link both up. Since the PC sends a sign of life every 0.2 s, silence is the indicator of trouble.

SILENCE	ACTION
1 second	Stop the motors, so the car does not carry on with the last command.
5 seconds	Assume the link died without notice and close it. On the next pass, Layer 2 finds it closed and reopens it.

Table 11.1 — The two silence thresholds.

11.4 LAYER 4 — READ WHAT ARRIVED

- Read every pending line.
- Each line received counts as a sign of life and resets the silence clock.
- Only movement-command lines touch the motors; the signs of life keep the link alive but do nothing else.

> **THE SAFETY ARGUMENT, IN ONE SENTENCE** The three motor-stop conditions — Wi-Fi down, link down, 1 s of silence — all converge on the same safe result: without confirmed control, the car stops. And the two restart mechanisms — reopen the link at 5 s, restart the chip at 10 s — cover the two levels of failure: the link to the program, and the network underneath it.

12. NVS: how the credentials survive a power cut

Chapter 9 assumed that `wifi_count` and the saved networks are still there after the power goes off. This chapter is the mechanism behind that assumption.

A microcontroller has two kinds of memory. **RAM** is fast and holds the program's variables while it runs, but it is emptied every time the power goes off. **Flash** is slower and wears out after many writes, but it keeps its contents without power — it is where the program itself is stored. The Wi-Fi credentials from configuration mode (Chapter 9) are sent only once, so they must survive a power cut: they have to live in flash.

	RAM	Flash
Keeps data without power	No	Yes
Speed	Very fast	Slower, especially to write
On the ESP32-CAM	520 kB inside the chip, plus 4 MB of PSRAM	4 MB
Used for	Variables, buffers, camera frames	The program, settings, Wi-Fi credentials
Limit	Size	Each sector can only be erased a limited number of times

Table 12.0 — The two memories and what each is for.

Writing flash directly is awkward: it can only be erased in whole 4 kB sectors, and each sector survives a limited number of erasures. **NVS (Non-Volatile Storage)** is Espressif's library that hides this. It reserves a part of the flash and lets the program store small named values — a key and a value, such as `wifi_count = 2` — while it spreads the writes across the sectors and survives a power cut in the middle of one.

12.1 PARTITIONS AND NAMESPACES

Flash memory is organised into sectors and pages, but the program interacts with it through namespaces. `nvs_flash_init_partition()` mounts the physical partition and validates the integrity of its data. Namespaces logically isolate different sets of information — Wi-Fi credentials against camera settings, for instance — so that keys with the same name in different namespaces do not collide.

12.2 THE HANDLE

The central question is how the program knows where to look using only a number.

- `nvs_open()` returns an opaque handle for use in the operations that follow. The application stores that identifier; it does not interpret it as a physical flash address.
- Without opening a valid handle, the application has no context in which to operate on that namespace through these functions.
- The handle ties the calls to the open context. The library manages the lookup and the storage of the data internally.

12.3 READING AND WRITING

CALL	WHAT IT DOES, AND WHAT TO CHECK
`nvs_get_xxx()`	Reads the requested key through the handle. The application checks the return code, because the key may not exist or may hold a different type.
`nvs_set_xxx()`	Requests that a key be written or changed. The library manages the organisation and the wear of the flash; the application must check the result.
`nvs_commit()`	Called after the changes, and its result checked, to obtain the persistence the API promises.
`nvs_close()`	Releases the resources associated with the handle.

Table 12.1 — The four calls of the NVS lifecycle.

> **TWO THINGS NOT TO ASSUME** Do not assume that each set is only a change in RAM, and do not assume that several keys form one atomic transaction. The library's internal synchronisation also does not replace the coordination needed between tasks when one application operation spans several calls.

In a robot with a camera this discipline matters directly. Releasing the handle keeps the RAM — needed to process video frames in real time — from being held by session tokens for configuration that is no longer in use. The handle is not just a number; it is what lets the ESP32 balance data persistence against the execution speed the motors and the vision pipeline demand.

13. Memory: fixed buffers instead of String

The car is meant to run for hours, and every 0.2 s a message arrives from the PC that has to be stored somewhere while it is read. If storing each message leaves a little mess behind, nothing happens in the first minute — but after thousands of messages the ESP32 runs out of usable memory and resets in the middle of a drive. RAM is used in two ways: the **stack**, where fixed-size variables live and are released automatically, and the **heap**, where the program asks for blocks of any size while it runs and must give them back. This chapter is about keeping the heap healthy.

Memory management on a microcontroller is critical, and the most common way to get it wrong is the C++ `String` class. A String is an object that uses dynamic allocation: it reserves space on the heap at run time, unlike a static array, which occupies a fixed and predictable space on the stack.

13.1 THE PROBLEM — STRING FRAGMENTS THE HEAP

The fundamental problem with using Strings heavily inside a continuous loop is RAM fragmentation. Picture the RAM as a car park with numbered slots. When a String is created to receive a command, the system reserves an exact slot for it. If the command that arrives is larger than the space first set aside, the String has to grow — and the system cannot simply push the neighbouring data aside. It has to find a new, larger slot, move the command there, and free the old one.

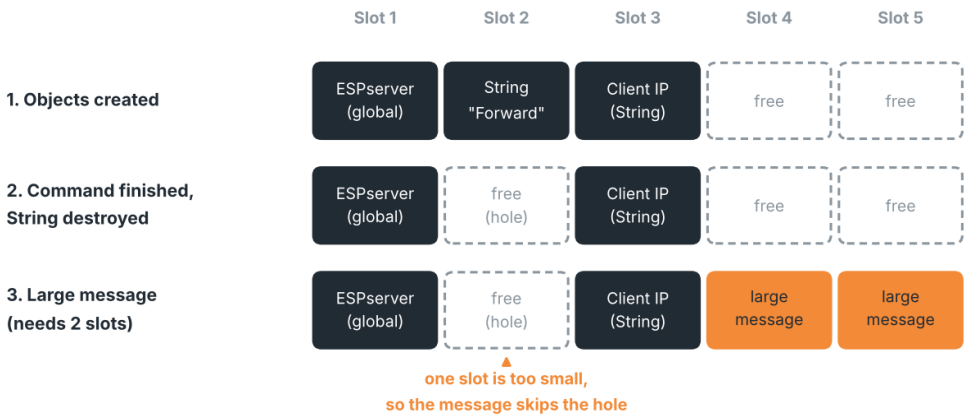


Figure 13.1 — A freed String leaves a hole that a larger message cannot use.

Following the figure: a global variable such as the `ESPserver` object parks in slot 1 and stays there for as long as the program runs. A command arrives and its String parks in slot 2. The client's IP is then stored in slot 3. When the command finishes being processed and its String is destroyed, slot 2 empties — and that is where the trouble is: slot 1 is taken, slot 2 is free, slot 3 is taken again.

The disaster comes with a large message — an image from the ESP32-CAM, say — that needs two consecutive slots. Slot 2 is technically free, but it is not enough on its own, because slot 3 is immediately in front of it blocking the way. The cars cannot move aside, so the system has to ignore that empty hole at the front and park the large message at the back, in slots 4 and 5.

> **WHY THIS CRASHES RATHER THAN JUST WASTING SPACE** After thousands of loop iterations the memory is full of small holes. The total of those holes may suggest plenty of free memory — 100 kB, say — but if the memory is fragmented there may be no single contiguous block large enough for a new operation. When the ESP32 tries to allocate and finds no contiguous block, the system panics and reboots.

13.2 THE SOLUTION — A FIXED BUFFER

The fix is to replace the dynamic allocation of the String class with static buffers. Declaring a fixed buffer such as `char buffer[32]` reserves a permanent area of RAM that never changes address while the program runs.

	STRING CLASS	FIXED BUFFER char buffer[N]
Allocation	Dynamic: reserves heap space at run time.	Fixed: reserved once, at program start.
Address	Can change when the text grows.	Never changes while the program runs.
After many loops	Heap full of small holes.	Heap stays contiguous.
Risk	No contiguous block found: the ESP32 crashes and reboots.	None from fragmentation.

Table 13.1 — String class against fixed buffer.

Unlike a String, which asks for new blocks of memory whenever the text grows, a static buffer always reuses the same space. When a new message arrives through `readBytesUntil()`, the microcontroller writes the new data straight over the old. With no reserve-and-release cycle, the holes never form. That keeps the heap contiguous and available for heavy work such as capturing and transmitting images — which is exactly what Chapter 7 needed.

```
// WRONG: String class (capital S)
String comando;                          // dynamic allocation
comando = client.readStringUntil('\n');  // grows as needed
```

```
// RIGHT: fixed buffer
char buffer[128];                          // declared once
memset(buffer, 0, sizeof(buffer));         // clear before each read
client.readBytesUntil('\n', buffer, 127); // write over the old data
```

Listing 13.1 — Reading a command with and without String.

> **RULE OF THUMB** A String used inside a continuous loop is a red flag. The code works for the first few minutes and crashes later, for no visible reason.

13.3 THE SECOND PROBLEM — MESSAGES ARRIVE AS A STREAM

TCP does not send information in closed, isolated packets as if they were letters. It behaves as a continuous stream, like a hose. If the reading is interrupted, or the network is delayed, it is like closing the tap halfway: the rest of the information stays in the pipe, in the Wi-Fi chip's buffer, waiting to be collected.

- The fix is a delimiter. The usual one for marking the end of a message is the newline character, `\n`.
- The right technical solution is `readBytesUntil()`. Rather than guessing whether the command is 7 or 10 bytes, it gives the ESP32 a clear instruction: read everything you find until you detect a `\n`; if the reserved space fills before that, stop.
- This guarantees the microcontroller waits for the complete message before trying to execute anything.

14. Pointers and string functions in C

Chapter 13 ended with a complete message sitting in one fixed buffer. This chapter is how that single buffer is turned into the separate pieces of information the program needs — without allocating a single byte more.

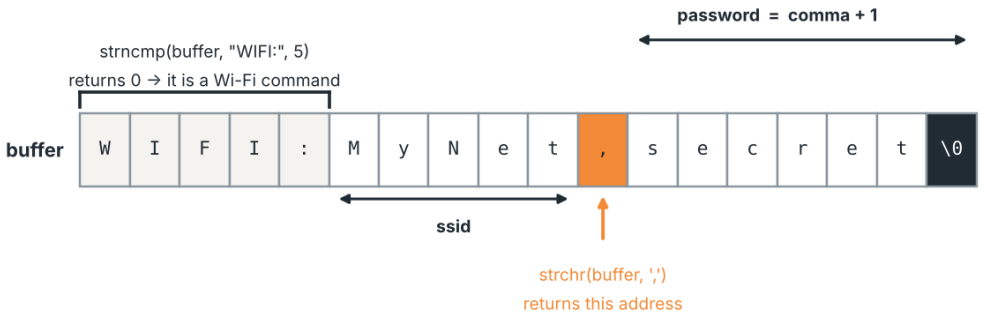


Figure 14.1 — One buffer, two usable strings, no allocation anywhere.

14.1 strncmp — THE SENTRY

`strncmp` (string number compare) is the precision tool for validating the intent of a command without instantiating dynamic String objects.

- It receives two memory addresses and compares their characters one by one, strictly limited to the `n` bytes you define.
- It is ideal for detecting command prefixes such as `WIFI:` or `RESET:` directly in the static buffer. A String would require reserving new heap blocks just to do the comparison; strncmp operates on the data already there, keeping the memory contiguous.

- It returns 0 if the first `n` characters are identical. That lets the microcontroller identify the command category and choose the processing path without creating the memory holes of Chapter 13.

14.2 strchr — THE LOCATOR

`strchr` (string character) acts as a navigator inside the car park of Chapter 13. Its job is to locate the exact position of a specific delimiter, such as the comma.

- It walks the buffer byte by byte until it finds the first occurrence of the character.
- Rather than copying the text, it returns only the pointer — the memory address where that character is physically parked.
- That address is the exact coordinate for making the logical cut in a command such as `WIFI:SSID,PASS`.
- Because it works only with addresses, it reuses the same memory already allocated by `readBytesUntil()`, so the system does not crash for want of sequential blocks — and the ESP32-CAM can handle both text commands and heavy image streams without instability.

14.3 THE NULL TERMINATOR AND WHAT A STRING REALLY IS

In C it is essential to understand that a string is not a heavy object. It is a starting address followed by bytes that end in a null terminator, `\0`.

IDEA	WHAT IT MEANS IN PRACTICE
A pointer is a start	Writing `char *password = comma + 1;` creates nothing on the heap. It only marks the spot in RAM where the password begins.
The role of \0	System functions such as `Serial.print()` or `Write_to_flash()` begin reading at that address and carry on consecutively until they meet the `\0` that was inserted after reading the buffer.
In-place manipulation	This turns one static buffer into several useful pieces of information, eliminating the reserve-and-release cycle that fragments the RAM.

Table 14.1 — Why the parsing costs no memory at all.

15. Buffer handling: process everything, or only the last?

The `while` loop that drains the buffer looks the same in AP mode and in the main loop, but it serves opposite purposes. Getting this backwards produces a robot that either loses configuration or obeys stale commands.

MODE	THE RULE, AND WHY
AP mode (configuration)	Read every message. If five Wi-Fi networks are sent one after another and then RESET:NOW, all five must be stored and only then the restart. So the processing call sits inside the while: each line read is executed immediately, before the next one is read.
Main loop (movement)	Time is critical. If the PC sends “Forward”, “Left” and “Stop” within 10 ms while the robot is busy with a camera frame, there is no point turning left when the last order was to stop. So the buffer is drained and only what remains at the end is executed.

Table 15.1 — The same loop shape, two opposite intentions.

16. Improvement log

Two defects found in the firmware during development, with the reasoning and the fix for each. Both concern the same stretch of setup() described in Chapter 10.

16.1 IMPROVEMENT 1 — CHECK THE FLAG BEFORE THE WI-FI TIMEOUT

In the original setup(), after calling `connect_to_current_strongest_wifi()`, the code always entered a 30-second wait loop for the Wi-Fi to connect — even when the function had found no known network in the scan:

```
// PROBLEM: WiFi.begin() was never called,
// but the code waits 30 seconds anyway
connect_to_current_strongest_wifi();   // may have done nothing

unsigned long tempo_inicial = millis();
while (WiFi.status() != WL_CONNECTED       // will never be true
       && (millis() - tempo_inicial < 30000)) {
    delay(500);
    Serial.print(".");                     // 30 s of useless dots
}
```

Listing 16.1 — The original code.

> **ROOT CAUSE** connect_to_current_strongest_wifi() was declared void — it returned no information about whether it had found a known network. The code that followed had no way of knowing whether there was any reason to wait.

The function was changed to return a `bool` saying whether it found a known network in the scan, and setup() checks that flag before entering the wait loop:

BEFORE — void, no return	AFTER — bool, with a return
`void connect_to_current_strongest_wifi() {`	`bool connect_to_current_strongest_wifi() {`
`    bool encontrou = false;`	`    bool encontrou = false;`
`    // comparison loops ...`	`    // comparison loops ...`
`    nvs_close(my_handle_read);`	`    nvs_close(my_handle_read);`
`    // returns nothing`	`    return encontrou;   // NEW`
`}`	`}`

Table 16.1 — The signature change that makes the rest possible.

```
if (!connect_to_current_strongest_wifi()) {
    // No known network found in the scan.
    // There is no point waiting 30 s.
    Serial.println("No known network. Rescue mode.");
    enable_access_point();
    loop_config_mode();
} else {
    // Found one AND called WiFi.begin() - now it is worth waiting
    unsigned long tempo_inicial = millis();
    while (WiFi.status() != WL_CONNECTED
           && (millis() - tempo_inicial < 30000)) {
        delay(500);
        Serial.print(".");
    }
}
```

Listing 16.2 — The corrected setup(), version 1.1.

- Removes 30 seconds of pointless waiting when the ESP32 boots somewhere with no known networks.
- The robot enters rescue AP mode immediately, so recovery is much faster.
- The code becomes semantically correct: it only waits for the Wi-Fi when there is a reason to.

16.2 IMPROVEMENT 2 — TRY THE OTHER NETWORKS BEFORE GIVING UP

Even after Improvement 1 the system had a resilience problem. If `WiFi.begin()` was called but the connection failed after 30 seconds — a router temporarily down, a weak signal — the ESP32 went straight into rescue AP mode without trying any of the other networks saved in flash.

> **THE CONCRETE CASE** The robot has three networks saved: Home, Office, Hotspot. The scan finds Home as the strongest, but the Home router is restarting. The original code gave up and entered AP mode, ignoring Office and Hotspot, which were both available.

The function was refactored to try the networks in priority order — strongest signal first — with setup() iterating over all the networks found and giving each its own timeout:

```
int numero_redes_ar = WiFi.scanNetworks();
bool conectou = false;

for (int i = 0; i < numero_redes_ar && !conectou; i++) {
    // Try each known network, in order of signal strength
    if (rede_conhecida(WiFi.SSID(i))) {
        WiFi.begin(ssid, password);
        unsigned long t0 = millis();
        while (WiFi.status() != WL_CONNECTED
               && millis() - t0 < 10000) {   // 10 s per network
            delay(500);
        }
    }
}
```

Listing 16.3 — The new approach: a loop of attempts, version 1.2.

- The robot is far more resilient: it only enters AP mode after exhausting every known network.
- The time per network drops from 30 s to 10 s, so the worst case is N × 10 s for N saved networks.
- Unavailable networks are discarded automatically with `WiFi.disconnect()` before the next attempt.

16.3 THE FINAL setup(), VERSION 1.2

```
void setup() {
    Serial.begin(115200);
    pinos_setup();
    pwm_channel_setup();
    setup_flash_memory();

    nvs_handle_t my_handle_read;
    uint32_t count = 0;
    esp_err_t err = nvs_open("wifi_storage_id", NVS_READONLY, &my_handle_read);

    if (err != ESP_OK || nvs_get_u32(my_handle_read, "wifi_count", &count) != ESP_OK)
{
        // No networks in flash - straight to AP
        enable_access_point();
        loop_config_mode();
    } else {
        nvs_close(my_handle_read);

        // Scan, ordered by signal, strongest first
        int numero_redes_ar = WiFi.scanNetworks();
        bool conectou = false;

        for (int i = 0; i < numero_redes_ar && !conectou; i++) {
            char ssid_buf[33], pass_buf[65];
            if (!get_credentials_for_ssid(WiFi.SSID(i).c_str(),
                                          ssid_buf, pass_buf)) continue;

            Serial.printf("Trying: %s\n", ssid_buf);
            WiFi.begin(ssid_buf, pass_buf);

            unsigned long t0 = millis();
            while (WiFi.status() != WL_CONNECTED
                   && millis() - t0 < 10000) {
                delay(500);
                Serial.print(".");
            }

            if (WiFi.status() == WL_CONNECTED) {
                Serial.printf("\nConnected to: %s\n", ssid_buf);
                conectou = true;
            } else {
                Serial.printf("\nFailed: %s. Trying next...\n", ssid_buf);
                WiFi.disconnect();
                delay(200);
            }
        }

        if (!conectou) {
            Serial.println("All networks failed. Rescue mode.");
            enable_access_point();
            loop_config_mode();
        }
    }
    // If it reaches here: it is connected to Wi-Fi
}
```

Listing 16.4 — setup() with both improvements integrated.

ONE NEW HELPER get_credentials_for_ssid() looks up a network's credentials in flash by name. It replaces the logic that used to live inside connect_to_current_strongest_wifi().

PART IV

Networking and remote control

Part III left the firmware holding a link to a PC and obeying whatever arrived on it. This part is that link, taken apart. It starts at the ESP32 end, goes down to what a socket actually is in memory, follows one message through the four layers of the stack and out across the internet, builds the Python server at the other end, and ends with the two decisions that made the system usable: dropping stale commands, and moving the video off TCP.

> **WHY THIS PART** The car is driven from a PC, so every command and every frame crosses a network. This part explains that path, and the two decisions that made driving feel immediate: dropping old commands and moving the video from TCP to UDP.

17. How the ESP32 serves a session

To receive anything, the ESP32 first has to be reachable. In configuration mode it creates its own Wi-Fi network; in normal mode it joins the house network. In both cases it has to open a door — a socket on a port — and accept a connection. This chapter is what happens inside the ESP32 when it does.

> **HISTORICAL CONTEXT** The examples in this part come from Phase I and use TCP sockets. They are kept because they explain the mechanism clearly. In the current version, Wi-Fi provisioning still uses TCP, but the ESP32's movement commands and video use UDP — Chapter 24 is the reason why.

17.1 TURNING THE RADIO INTO AN ACCESS POINT

CALL	WHAT IT ACTUALLY DOES
`WiFi.mode(WIFI_AP)`	A direct interaction with the chip's radio controller. It changes the physical state of the radio from Station (looking for signals) to Access Point (generating one). It is like switching a radio from FM reception to walkie-talkie: the hardware prepares to emit its own carrier.
`WiFi.softAP(ssid, pass)`	A hybrid, acting on both hardware and the LwIP stack. In hardware it orders the radio to fire beacon packets every 100 ms, carrying the network name, which is what makes the network visible. In software it starts the DHCP server in RAM, which hands out IP addresses to whoever connects.
`ESPserver.begin()`	Opens the listening socket on the port. Without it, the network exists but nothing answers on it.

Table 17.1 — The three calls that create the provisioning network of Chapter 9.

> **WHAT HAPPENS WITHOUT ESPserver.begin()** The network is visible — a phone finds the name in its Wi-Fi list. The phone can even connect and be given an IP by DHCP. But any attempt to send a motor command over TCP is refused. It is like arriving at a building (joining the Wi-Fi) and finding the office door (port 1883) locked, because the doorman was never hired.

17.2 THE PCB — WHERE A CONNECTION ACTUALLY LIVES

TCP communication on the ESP32 is centred on a critical data structure called the PCB (Protocol Control Block), managed by the LwIP network stack. It is the identity card of a live connection in RAM, holding the source and destination IP addresses, the ports and — crucially — the current state of the TCP state machine.

- When `ESPserver.begin()` runs, LwIP allocates a listening PCB.
- As soon as a client establishes a connection, a new PCB is generated to track that specific session.

17.3 THE HANDSHAKE, AND HOW THE SESSION REACHES YOUR CODE

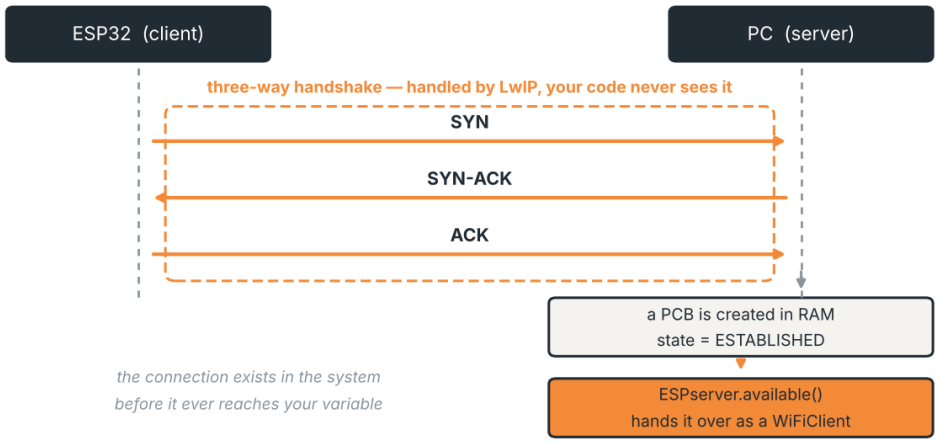


Figure 17.1 — The connection is fully established before any line of your firmware is involved.

The client sends a SYN packet. The instant that signal reaches the ESP32's radio, the hardware raises an interrupt, briefly pausing the processor so that the LwIP stack can answer automatically with SYN-ACK. The client finishes with an ACK, and from that moment the connection is established — but it still has not reached any variable in your code. It exists in system memory, as a PCB.

CALL	WHAT IT DOES
`ESPserver.available()`	Queries the LwIP stack for a connection in the waiting queue that has already completed the handshake. If there is one, it removes it from the queue and wraps it in a WiFiClient object.
`client.connected()`	Queries the internal state of the PCB to confirm the session is still ESTABLISHED. This is what distinguishes a client that is merely silent from one whose connection has dropped.
`client.available()`	Checks the receive buffer for the presence of bytes.

Table 17.2 — The cascade of checks: is the bridge there, are there bytes, then move the bytes.

That cascade is what keeps the microcontroller from wasting processing cycles or memory on phantom connections — the stability that real-time video processing and motor control require.

17.4 WHY THE RETURN VALUE MUST BE STORED

A `WiFiClient` is an object holding everything needed to talk to one specific peer: its IP, its port and, most importantly, access to the data buffer for that conversation. Without the object you would know somebody connected, but have no handset to hear them with.

- The access is not a file — it is a pointer in RAM. When the connection is made, LwIP reserves a piece of heap to be the bucket the data falls into.
- The `WiFiClient` stores the address of that memory. When `client.read()` is called, the ESP32 knows exactly where in RAM to fetch the bytes from.
- If the object disappears, the address is lost and the data is orphaned in memory until the system cleans it up.

EVEN WITH ONLY ONE CLIENT, THE ASSIGNMENT IS MANDATORY In C++, available() returns a temporary instance. If that return value is not assigned to a persistent variable such as newClient, the compiler destroys the object as soon as the line finishes executing — and the program loses the socket descriptor, so it can neither reach the data buffer nor check the connection state.

The system tells multiple clients apart by the unique pair of IP plus source port. Even if two requests come from the same computer, the client's operating system assigns a different port to each attempt, so the ESP32 never confuses one session's messages with another's.

18. What a socket really is

A connection always has two ends. Chapter 17 was the ESP32's end; this is the PC's, where the Python server waits for the car to connect.

The same mechanism, now from the PC side — and the most common misconception first.

A SOCKET IS NOT A PHYSICAL PORT It is not a hole in the computer and it is not hardware. It is a data structure in the operating system's RAM. The "port" is just a 16-bit number inside that structure, which the operating system uses to know which program to deliver arriving packets to.

A precise analogy: a building (the IP) has several flats (the ports). When a letter (a packet) arrives, the doorman (the operating system) looks at the flat number and delivers it to the right resident (the program). The socket is the agreement between resident and doorman — a card on the doorman's desk saying "flat 1883 belongs to Python".

18.1 THE FOUR CALLS, AND THE TABLE THEY CHANGE

The operating system keeps a table in memory called the socket table. Following one connection through it is the clearest way to see what each call does.

the socket table inside the operating system

`socket()` *memory allocated, nothing filled in*

FD	LOCAL IP	PORT	REMOTE IP	PORT	STATE
3	0.0.0.0	0	0.0.0.0	0	CLOSED

`bind()` *the port is now reserved*

FD	LOCAL IP	PORT	REMOTE IP	PORT	STATE
3	0.0.0.0	1883	0.0.0.0	0	CLOSED

`listen()` *passive, with a waiting queue*

FD	LOCAL IP	PORT	REMOTE IP	PORT	STATE
3	0.0.0.0	1883	0.0.0.0	0	LISTEN

`accept()` *a second socket is created for this one connection*

FD	LOCAL IP	PORT	REMOTE IP	PORT	STATE
3	0.0.0.0	1883	0.0.0.0	0	LISTEN
4	192.168.1.100	1883	192.168.5.10	54321	ESTABLISHED

Figure 18.1 — The same socket, after each of the four calls. Orange is the socket created by accept().

CALL	WHAT THE OPERATING SYSTEM DOES
`socket()`	Allocates RAM for a C structure, assigns a file descriptor (an integer, say 3) and fills in the basics: protocol TCP, state CLOSED. Everything else is empty.
`bind()`	Fills in the local IP and local port, checks that port 1883 is free, and registers it in the socket table. If it is already taken, this is where the error happens — which is why running the server twice fails here.
`listen()`	Changes the state from CLOSED to LISTEN and creates the waiting queue.
`accept()`	Puts the Python thread to sleep, consuming no CPU, until a connection arrives.

Table 18.1 — Each call fills in a little more of the same structure — until accept().

18.2 THE CRITICAL MOMENT — accept() CREATES A SECOND SOCKET

When the SYN packet arrives, the network card passes it to the driver and then to the OS, which reads the header, asks "who is listening on 1883?", finds fd=3, and queues the connection. It then completes the handshake itself, without telling Python anything. Only afterwards does it create a brand-new socket, fd=4, for this one connection, and wake the sleeping `accept()`.

QUESTION	ANSWER
Why is the local IP 192.168.1.100 and not 0.0.0.0?	Because the packet arrived through one specific interface. The OS copies the concrete IP from the packet header's destination field.
Why two sockets rather than one?	Because fd=3 must carry on listening in order to accept further connections. If the same socket were used for communication, no second client could ever be accepted.
What does accept() return?	Two things: `conn`, a reference to the new socket, and `addr`, a tuple of the client's IP and port.

Table 18.2 — fd=3 is the company switchboard, which only ever rings; fd=4 is the live call on a dedicated line.

18.3 WHY 0.0.0.0, AND NOT A SPECIFIC IP

A PC can have several network interfaces at once — Wi-Fi on 192.168.1.100, Ethernet on 192.168.2.50, a VPN on 10.0.0.5, loopback on 127.0.0.1. Binding to one of them accepts only connections arriving through that one; anyone coming in over the Ethernet cable would be rejected.

```
server.bind(('192.168.1.100', 1883))   # Wi-Fi only
server.bind(('0.0.0.0',       1883))   # every interface
```

Listing 18.1 — 0.0.0.0 means "I accept connections on any network interface".

18.4 BACKLOG AND PORTS

The argument to `listen()` is the backlog: the size of the queue of pending connections.

SITUATION	WHAT HAPPENS
`listen(1)`	One connection may wait. If two clients send SYN almost simultaneously, the second waits; a third would be rejected because the queue is full.
`listen(5)`	Up to five may wait.
This project	listen(1), because only one ESP32 is expected. If two try at once, which is rare, the second waits rather than being refused.

Table 18.3 — The backlog only governs connections that have not yet been accepted.

- There are 65 536 ports (0 to 65 535) per IP, because the port field in the TCP and UDP headers is 16 bits.

- Port 1883 is the standard MQTT port. Nothing about it is special here — 5000 or 8080 or anything above 1024 would do. The advantage is that many routers already carry predefined MQTT settings.

19. One message, four floors down

Chapter 18 ended with bytes handed to the operating system. This chapter follows them to the antenna. The network is organised in layers, one on top of the other: the program is at the top, the network card at the bottom, and each layer takes the data from the one above, adds its own information, and passes it down.

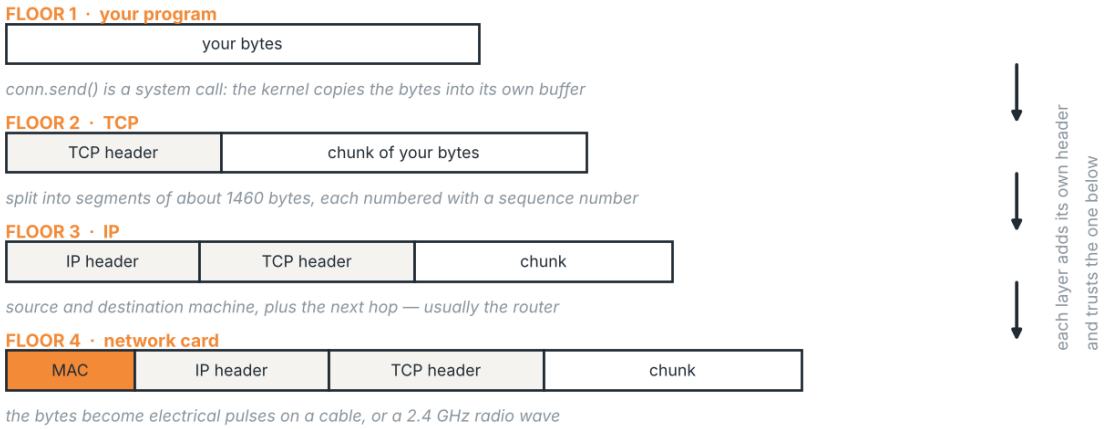


Figure 19.1 — The same bytes, wrapped one layer at a time.

19.1 FLOOR 1 — THE PROGRAM HANDS THE BYTES OVER

You write `conn.send(data)`. The `conn` is the number identifying the socket, and `send` is a system call — a special call that passes control from your program to the operating system kernel. Your program has no access to the network hardware; the kernel does. The system call is the door between those two worlds.

- You pass the kernel two things: the socket number, and the bytes.
- The kernel looks at the number, goes to that socket's internal structure — where the destination IP and port are stored — and copies your bytes into a buffer of its own.
- `send` returns and your program carries on. The bytes sit in an outgoing buffer waiting to be processed.

> **THE KEY POINT** You do not send anything over the network. You hand the bytes to the kernel and it sends them. The socket is the interface of that handover.

19.2 FLOOR 2 — TCP SPLITS AND NUMBERS

- It splits them up. The network does not carry one giant block at once; there is a maximum useful size per packet, around 1460 bytes. 5000 bytes becomes several pieces, each one called a segment.
- It puts a header on each segment, carrying the source port, the destination port, and the sequence number — the position of those bytes in the overall stream. That number is what lets the far end reassemble the segments in the right order and detect whether one went missing.

19.3 FLOOR 3 — IP ADDRESSES AND CHOOSES THE NEXT HOP

The IP layer wraps each TCP segment in a header of its own carrying the source and destination IP. If TCP says which port, IP says which machine. It also decides the first hop: it looks at the destination and, if it is not on the local network, sends it to the router. The packet does not know the whole route — only the next hop. Every router along the way repeats this.

19.4 FLOOR 4 — THE NETWORK CARD MAKES IT A SIGNAL

The packet reaches the network card driver. The card adds the final header, Ethernet or Wi-Fi, carrying the physical MAC address of the next hop, and makes the final conversion: bytes become a physical signal. On a cable they become electrical pulses. On Wi-Fi they become radio waves — the card modulates the bits onto a 2.4 GHz wave and the antenna radiates it into the air, the same mechanism as any antenna: current oscillating at high frequency produces a wave that propagates.

> **AND THE SAME IN REVERSE** At the other end: the signal becomes bytes, IP confirms the address, TCP reassembles in sequence order, and the data is delivered to the destination program. Each layer only handles its own job and trusts the one below — which is why your program only ever sees “send these bytes”.

20. Reaching the robot from anywhere

> **CURRENT VERSION** This chapter explains how the Phase I car was reached across the internet over TCP. The current car uses UDP and finds the PC with a broadcast on the local network, so all three computers must be on the same network. The chapter is kept because the mechanism is still how home networks work.

Chapter 19 got the signal onto the air. Between the two machines, though, sits the single hardest problem in this whole system: a home router is built to stop exactly the kind of connection this project needs.

20.1 THE ISP AND THE PUBLIC IP

The ISP is the company providing the internet connection, operating a large physical network: optical fibre, which carries light through glass cables; coaxial cable, as used for cable television and slower than fibre; and radio or satellite for remote areas where fibre does not reach. The ISP assigns a public IP to the router — unique on the internet, like a telephone number. When data is sent, the ISP routes it to other ISPs until it reaches its destination.

20.2 NAT, AND THE PROBLEM IT CREATES

A house has one public IP but many devices inside it. They share it through NAT (Network Address Translation): the router keeps an internal table translating between them.

INTERNAL (PRIVATE)	EXTERNAL (PUBLIC)
`192.168.1.100:54321`	`85.243.10.20:12000`
`192.168.1.50:8080`	`85.243.10.20:12001`
`192.168.1.10:443`	`85.243.10.20:12002`

Table 20.1 — When a packet leaves the house the router swaps the internal IP for the public one plus a temporary port. When the answer comes back it consults this table and delivers it to the right device.

That works perfectly for connections going outwards. Inwards it fails. If the ESP32, somewhere else, wants to connect to the PC inside the house, it knows the public IP — but when the packet reaches the router, the router has no idea who to give it to. It could be for the PC, the phone, the console. By

default it rejects the connection, and that is a security measure: without it, anyone on the internet could try to reach the devices inside.

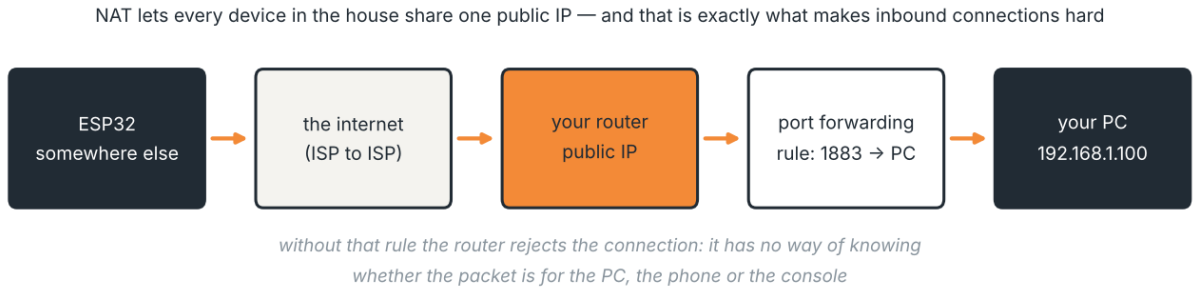


Figure 20.1 — The path from the robot to the PC, and the one rule that makes it possible.

20.3 PORT FORWARDING

- Find the PC's local IP: `ipconfig` on Windows, `ip addr` on Linux.
- Open the router's page, usually at `http://192.168.1.1`, and find Port Forwarding.
- Add the rule: external port 1883, internal IP the PC's, internal port 1883, protocol TCP.
- Test it from outside with a service such as canyouseeme.org on port 1883.

> **ADD A SECOND RULE FOR THE VIDEO** If port forwarding is already configured for the control port, add another rule for the video port rather than replacing it. A router holds several rules at once.

20.4 DDNS — WHEN THE PUBLIC IP CHANGES

- Create a subdomain at a DDNS provider such as DuckDNS and copy the token.
- Paste the token into the router's DDNS section.
- The ESP32 then uses the name instead of a numeric IP, and keeps working when the ISP changes the address.

20.5 IPv4 AND IPv6

	IPv4	IPv6
Size	32 bits → 4.3 billion addresses	128 bits → 3.4×10^{38} addresses
Format	192.168.1.100	2001:0db8:85a3::8a2e:0370:7334
Created	1981	1998
In Python	`AF_INET`	`AF_INET6`

Table 20.2 — The two addressing schemes.

- IPv6 exists because IPv4 ran out. In 1981, 4.3 billion looked like plenty; today there are 8 billion people with a phone, a PC, a tablet, a watch and a console each, plus the IoT devices — fridges, cars, cameras, light bulbs. IPv4 was exhausted in 2011.
- They cannot simply be swapped, because they are not compatible: IPv4 does not talk to IPv6 directly, and switching everything at once would mean turning off the entire internet. The transition is gradual, with both coexisting (dual-stack).
- For this project: an address of the form 192.168.x.x is IPv4, and one with `::` in it is IPv6. Most home and IoT networks are IPv4.

20.6 WHAT HAS TO BE WORKING FOR EACH LINE OF CODE

PYTHON CODE	WHAT HAPPENS	WHAT MUST BE UP
`socket.socket()`	Creates the structure in RAM	Nothing — it is only memory
`server.bind()`	Reserves the port in the OS	The operating system
`server.listen()`	Goes passive, registers on the network	An active network card
`server.accept()`	Waits for a SYN packet from the network	Router up, port open
`conn.send()`	Sends bytes through the network card	Cable or Wi-Fi up, internet live
ESP32 receives	ISP → internet → ISP → router → ESP32	Port forwarding configured, ESP32 on Wi-Fi

Table 20.3 — A debugging checklist as much as an explanation: each row fails differently.

21. The Python control server

The other end of the link. This chapter covers the gamepad, the arithmetic that turns its raw values into commands, and the complete server.

21.1 THE GAMEPAD AND ITS DRIVER

An Xbox controller is hardware — a chip, buttons, a USB cable. The operating system cannot talk to it directly; it needs a translator. That is the device driver: XInput on Windows, evdev on Linux. It reads the electrical signals and converts them into a data structure, which the operating system then exposes to Python through `pygame.joystick` as plain values: axis 0 = 0.75, button A = pressed.

21.2 NORMALISING THE RAW VALUES

The raw values are not the ones the robot wants. Three corrections are needed.

PROBLEM	FIX
Triggers rest at −1, but 0 is wanted	`(raw + 1) / 2` maps −1 → 0, 0 → 0.5, 1 → 1
The Y axis is inverted: up is −1	`y = -y` so that up becomes positive
A resting stick returns about 0.02 from electrical noise	A dead zone: anything below 0.1 in absolute value is forced to exactly 0

Table 21.1 — The three normalisations, and why each exists.

```
def dead_zone(value, limit=0.1):
    if abs(value) < limit:
        return 0.0
    else:
        return value
```

Listing 21.1 — The dead zone, written out rather than as a one-liner.

21.3 THE COMPLETE SERVER

```
import socket
import pygame
import time

HOST = '0.0.0.0'      # every interface
PORT = 1883

pygame.init()
pygame.joystick.init()
joystick = pygame.joystick.Joystick(0)
joystick.init()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen(1)
print(f'Listening on {HOST}:{PORT}')

conn, addr = server.accept()
print(f'ESP32: {addr}')

try:
    while True:
        pygame.event.pump()

        rt = joystick.get_axis(5)      # right trigger
        lt = joystick.get_axis(4)      # left trigger
        x  = joystick.get_axis(0)      # left stick, horizontal

        accel = (rt + 1) / 2           # -1..1  ->  0..1
        brake = (lt + 1) / 2

        if abs(x) < 0.1:               # dead zone
            x = 0.0

        if accel > 0:
            speed = int(accel * 100)   # forward
        elif brake > 0:
            speed = -int(brake * 100)  # reverse
        else:
            speed = 0

        steer = round(x, 2)
        cmd = f'MOV:{speed},{steer}\n'
        conn.send(cmd.encode())

        time.sleep(0.05)

except KeyboardInterrupt:
    pass
finally:
    conn.close()
    server.close()
    pygame.quit()
```

Listing 21.2 — The complete control server. The 0.05 s sleep is the 20 Hz sign of life that Chapter 11's Layer 3 watches for.

TESTING IT First run the axis-test script to see every axis live and confirm which index is which. Then test locally, with the PC and the ESP32 on the same network: start the server, power the ESP32 pointed at the PC's local IP, move the stick, and watch the commands appear in the Serial Monitor. Only then try it through port forwarding.

22. Choosing and joining the strongest network

Every time the car is switched on it has to choose which of its saved Wi-Fi networks to join — at home, at school, or a phone hotspot. The sensible choice is the strongest one in range.

Chapter 16 improved this logic; this chapter is how the pieces of it work.

22.1 WHAT scanNetworks() ACTUALLY RETURNS

- `WiFi.scanNetworks()` does not return a list of objects. It returns an integer: the number of networks detected in the air at that moment.
- The details — name, signal strength, channel — are kept in the WiFi library's internal memory.
- They are reached by index, from 0 to n−1: `WiFi.SSID(i)` returns the name of network i, and `WiFi.RSSI(i)` its signal strength in dBm.
- By default the results are already ordered by signal, so index 0 is the strongest — which is what makes the outer loop a priority order for free.

22.2 THE JOURNEY FROM FLASH TO RAM

The credentials do not live permanently in RAM, which would waste precious space. They stay dormant in flash. The ESP32 keeps a small index table in RAM acting as a map, saying where each key such as `wifi_0` physically sits in flash. When `nvs_get_blob()` is called, the system uses that index to locate the data in flash and copy it into a temporary variable in RAM — and only at that instant does the data arrive on the desk where the processor can manipulate and compare it.

22.3 FOUR DETAILS THAT MATTER

QUESTION	ANSWER
`uint32_t count = 0;`	This is defensive programming. If nvs_get_u32 fails — corrupted or empty NVS — the variable would keep whatever rubbish was previously in that RAM. Forcing zero guarantees that on error the program knows there are no networks, instead of looping over non-existent addresses.
`sprintf()`	A label factory. `sprintf(key, "wifi_%lu", j)` builds a formatted string into a character buffer: when j is 0 it writes wifi_0 into key. That is how each saved network can be called up dynamically, without declaring hundreds of variables by hand.
`break;`	It breaks only the innermost loop — the flash loop. To leave the outer scan loop as well, a control variable such as bool encontrou is needed. Without it, the robot would carry on searching after it had already found the best connection.
`.c_str()`	The critical memory-management point. WiFi.SSID(i) returns a String object using heap allocation, while strcmp requires a plain C-style character array. .c_str() exposes the address where the String's text is stored, as a const char*. It is the bridge that lets the low-level functions read the text.

Table 22.1 — The details behind connect_to_current_strongest_wifi().

22.4 WI-FI IS ASYNCHRONOUS

This is the key to the whole timeout discussion in Chapter 16. When `WiFi.begin()` is called, the ESP32 does not sit and wait for the connection. It only fires the starting gun — beginning the hardware negotiation — and immediately executes the next line of code.

- So `connect_to_current_strongest_wifi()` finishes in milliseconds, whether the connection succeeded or not.

- The `while` loop in setup() is where the watching happens: because the Wi-Fi runs in the background, the code enters a deliberate loop asking the hardware repeatedly whether it is connected yet, while `millis()` tracks the elapsed time.
- That loop is a wall preventing the code from reaching the main loop() until the connection is confirmed or the timeout expires.

23. Latency and the FIFO queue

In a remote-control system, latency is a safety matter, not a comfort one. By default the ESP32's LwIP network stack operates as a FIFO queue — first in, first out — so the first command to arrive at the hardware is the first delivered to the processor.

That structure is ideal for data integrity and harmful for robotics. If a delay occurs in the communication, the robot starts executing a sequence of old commands that no longer match what the operator currently intends.

> **THE FIX: FLUSHING THE BUFFER** Instead of processing each queued command in sequence, the code runs quickly through every message accumulated in the receive buffer on each pass of loop(), overwriting the earlier instructions. Obsolete orders are discarded and only the most recent command is executed.

This turns a communication that would otherwise be history-based into a real-time response. If the operator sends a stop order after a sequence of movements, that order is prioritised and executed immediately, ignoring the movement commands still pending in the network queue. Chapter 15 described when this applies and when the opposite rule is correct.

24. Why the video moved from TCP to UDP

The video stuttered. That observation is what drove the change, and it comes directly from how the two protocols handle a lost packet.

24.1 THE TWO MODELS

PROTOCOL	THE ANALOGY
TCP — a phone call	First you dial. The other person answers. You confirm you can hear each other. Only then do you speak. During the call, if you miss a word you say "repeat that", and they repeat it. At the end you hang up. There is an established connection — an open channel with confirmation that everything arrives, in the right order. If a piece is lost, TCP stops, resends, and everything behind it waits.
UDP — a postcard	You write it, drop it in the postbox, and that is that. You did not call anyone and did not confirm they were home. The postcard either arrives or is lost — no confirmation, no resend. There is no connection, only the sending. Fire and forget: you send and move straight on to preparing the next one.

Table 24.1 — The difference is whether a connection is established between the two sides.

24.2 WHY THAT MATTERS FOR LIVE VIDEO

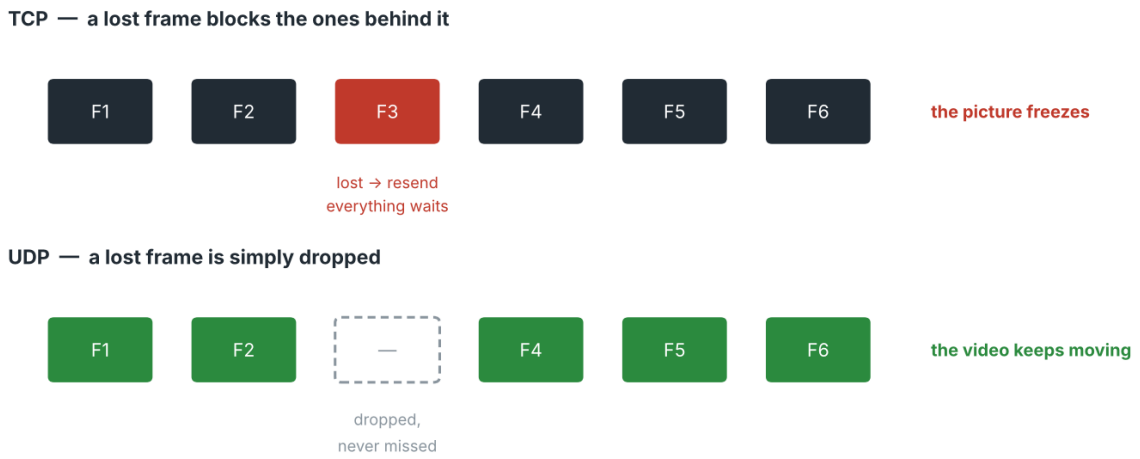


Figure 24.1 — The same lost frame, under each protocol.

In live video a frame that arrives late is of no use — the moment has passed. It is better to throw it away and receive the next one, fresh. TCP insists on delivering the late one and makes you wait for it; UDP lets it drop and carries on. That is exactly why video streaming uses UDP and file downloads use TCP: with a file you want everything, in the right order, and with video you prefer fresh frames to complete ones.

> **IN ONE LINE** TCP is reliable but it waits, which is good for files. UDP is fast with no guarantees, which is good for live video, where one lost frame does not matter because another is already on its way.

24.3 WHAT CHANGES IN THE CODE

In the firmware, sending a frame becomes the equivalent of dropping a postcard in the box:

```
udpVideo.beginPacket(destination, VIDEO_PORT);  // write the address
udpVideo.write(fb->buf, fb->len);               // the contents (the JPEG)
udpVideo.endPacket();                           // into the postbox
```

Listing 24.1 — No waiting for a reply, no delivery confirmation.

ON THE PC SIDE	WHAT CHANGES
The socket type	`SOCK_STREAM` (TCP) becomes `SOCK_DGRAM` (UDP)
The calls	`accept()` and `recv()`, which presuppose a connection, become `recvfrom()`
Why “from”	Because with no connection, the call must also tell you who the packet came from. Under TCP that was unnecessary, since the connection already knew who it was talking to.

Table 24.2 — The matching change at the other end.

PART V

Autonomy and image processing

Up to here the car has been driven by a person. This part is where it starts driving itself, using the frames Part II learned to capture. It is deliberately not machine learning: the rule for following a black line is simple enough to write by hand. What follows is that rule, the image processing that feeds it, the arithmetic that converts a measurement in pixels into a steering command, and an honest record of the corner it still cannot take.

> **WHY THIS PART** Manual driving proved the car worked. The next goal was for it to drive itself, and the simplest version of that is following a line of tape on the floor using the camera it already had.

25. What line following is, and what it is not

Driving by hand proved the car worked. The next step was to let it drive itself along a track of tape on the floor, using the camera it already had. Before writing any code, it is worth deciding what kind of problem that is.

25.1 WHAT "AI" MEANS HERE

The problem AI came to solve is this: there are tasks we know how to do but cannot write the rules for. You can recognise your sister's face, but you cannot write the instruction list "if pixel X is this colour and pixel Y is that colour…". The rule is too complex to write by hand.

The core idea of modern AI is that instead of writing the rules, you give the machine examples and it extracts the rules on its own. Show it a thousand photos with a face and a thousand without, and it finds the pattern. That is learning, or machine learning.

> **LINE FOLLOWING NEEDS NONE OF IT** Following a black line needs no learning at all. The rule is simple and writable by hand: find the black pixels, check whether they are left or right of centre, correct. This is classic computer vision, not AI in the learning sense — you write the rule yourself. Following a person would need learning, because "where is the person in the image" is too complex to write; that is what Part VIII is for.

25.2 THE CONTROL LOOP: SEE, DECIDE, ACT

STAGE	WHAT HAPPENS
See	Each camera frame is a grid of pixels. The line is black, the floor is light. Go through the pixels and mark the dark ones — that separates line from not-line. It is just comparing each pixel against a limit.
Decide	Compute the centre of those pixels — the average horizontal position of the line — and compare it with the centre of the frame. Line at centre means go straight; left of centre means turn left; right of centre means turn right.
Act	Convert that into the same MOV command the gamepad produces, and send it to the car. Next frame, repeat.

Table 25.1 — The whole autonomous mode, in three stages.

25.3 TWO CASES THE SIMPLE MODEL GETS WRONG

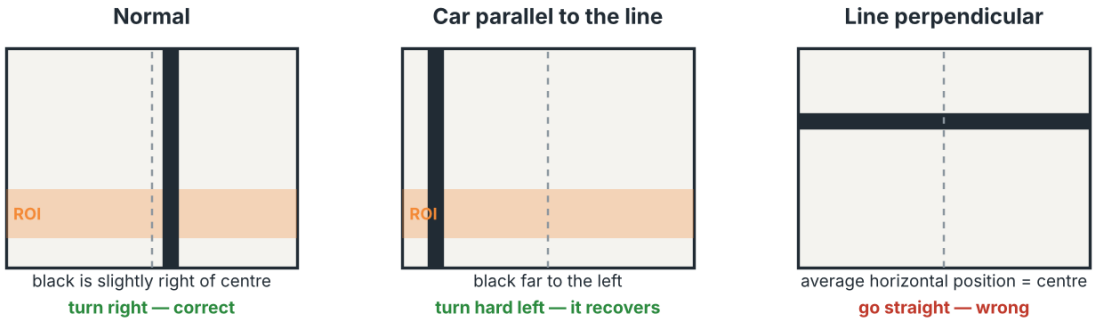


Figure 25.1 — The dashed line is the centre of the frame. The third case is the real blind spot.

- Car parallel to the line. If the car is beside the line, pointing the same way, the camera sees the line pushed against one edge of the frame. The system reads “line far left” and turns hard until the line comes back to the centre of the image, then settles on top of it. It fixes itself — as long as the line stays inside the field of view.
- Line perpendicular to the car. If the line crosses the path left to right, the horizontal centre falls in the middle of the frame, so the model reads “go straight” and drives across it.

THE REAL CAUSE, AND THE FIX Reducing the line to an average horizontal position throws away its orientation — that is the blind spot. The fix is to look only at a thin strip in front of the car rather than the whole frame. A perpendicular line then shows up as black crossing that strip, and it can be treated as “corner ahead” instead of “go straight”.

26. The vision pipeline

Where this fits: the frame reaches the PC, the processing thread analyses it, and the result becomes the same MOV command the controller would produce. Button A on the controller switches between manual and line mode. The ESP32 never knows whether a command came from a person or from the camera.

26.1 STEP BY STEP

Step	Call	What it does
Look at a strip	`frame[384:456, :]`	Keep only a band from 80% to 95% of the image height: the floor just ahead. Walls and furniture never reach the analysis.
Greyscale	`cv2.cvtColor`	Colour does not matter, only dark against light.
Adaptive threshold	`cv2.adaptiveThreshold`	Each pixel is compared with the mean of its 151×151 neighbourhood minus 15, so a shadow or a bright patch on one side does not break the line. The result is inverted: the dark tape becomes white.
Opening	`cv2.morphologyEx`	An 11×11 elliptical opening removes specks (Chapter 27).
Contours	`cv2.findContours`	Every white shape becomes a contour, largest first.
Filter	area, width, height, distance	Reject shapes smaller than MIN_AREA = 600 px, wider than 65% of the image (usually reflections or floor patches), shorter than 30 px, or more than 150 px away from where the line was last seen.
Centre	`cv2.moments`	Of the shapes left, the one closest to the last position is the line. Its moments give its centre cx = m10 / m00.
Error to steering	GAIN, DEADBAND	error = cx − width/2. Inside ±8 px the car goes straight; outside, steering = error × GAIN / (width/2), limited to ±1 (Chapter 28).

Table 26.1 — The pipeline as it runs in the PC server, with the call that implements each step.

Remembering the last position matters more than it looks. A broken line, a reflection or a second piece of tape can all produce valid-looking shapes; preferring the one nearest the previous frame keeps the car on the line it was already following instead of jumping to another.

The command sent is the same shape as always: `MOV:speed,DIR:steering`. The speed is chosen by the main thread (Chapter 29), not by the vision.

26.2 WHY A LIBRARY AND NOT PLAIN PYTHON

- You could write it in plain Python, but you would be rewriting things solved decades ago. A VGA image has 300 000 pixels, and walking them one by one in pure Python, every frame, dozens of times a second, is far too slow — Python is interpreted, and slow at loops over large amounts of data.
- OpenCV is written in C++ underneath, and Python only gives it the orders. The C++ processes those 300 000 pixels in milliseconds. You write in Python while the heavy work runs compiled.
- It is also a tested collection of operations that already exist — threshold, colour conversion, contour detection — written once, optimised, used by millions.

> **WHY TRUST THIS ONE** OpenCV is the de-facto standard in computer vision: over twenty years old, open source so the implementation can be read, and used in both industry and academia. This is not blind faith — it is the most scrutinised option, so serious bugs would have been caught by that many eyes.

27. Morphology: erosion, dilation and opening

After the threshold the image is white (the line) and black (the background), but dirty: stray white dots where there should be none. Morphology cleans that up — it erases what is small and keeps what is large and solid.

The central idea is a stamp that travels across the image. Morphology uses a small shape, called a kernel or structuring element — a 5×5 pixel stamp, say. That stamp passes over every pixel, and at each position asks one question about that pixel's neighbourhood. The answer decides whether the pixel stays white or turns black.

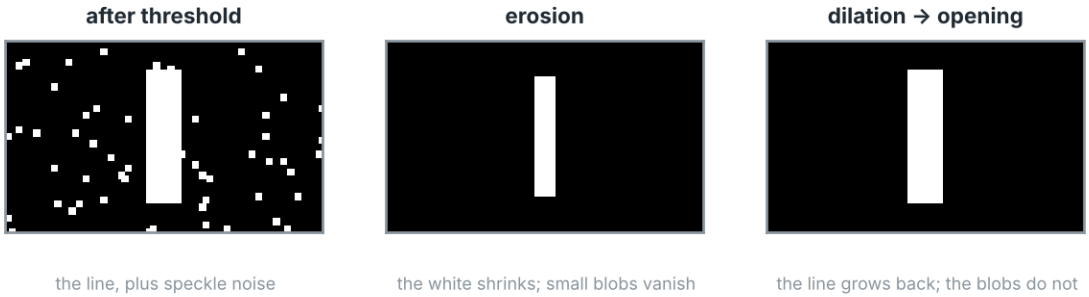


Figure 27.1 — Computed with a 5×5 kernel on a line plus artificial speckle noise.

OPERATION	THE QUESTION IT ASKS	THE EFFECT
Erosion	Are all my neighbours inside the stamp also white?	If yes the pixel stays white; if even one neighbour is black it turns black. The edges of the white are eaten away and the white shrinks. A pixel on the edge of a shape always has a black neighbour beside it, so it dies — the shape loses a shell of pixels. A small blob, which is almost all edge and has no core, is eaten completely.
Dilation	Is any of my neighbours white?	If yes the pixel turns white. The white spreads outwards and grows, filling small holes inside it and giving back the shell that an erosion ate.
Opening	Erosion, then dilation — in that order	The small blobs disappear for ever and the line is left intact. It works like an eraser that only rubs out small things.

Table 27.1 — The trick is entirely in the order.

- Erosion first shrinks everything. The small blobs are too small and vanish completely. The line shrinks at its edges but survives, because it has a core.
- Dilation then grows everything back. The line, having survived, returns to its original size. But the blobs the erosion erased no longer exist — there is no white left there to grow back from.

> **THE SYMMETRIC OPERATION** The inverse, closing (dilation then erosion), fills black holes inside the white instead of erasing blobs. It is the symmetric partner of opening.

27.1 THE KERNEL: SIZE AND EFFECT

The kernel is the neighbourhood the stamp looks at around each pixel. A 5×5 kernel is a square of 5 by 5 pixels — 25 in all — centred on the pixel being evaluated. When erosion asks "are all my neighbours white?", those are the neighbours. 5×5 looks 2 pixels in each direction, 3×3 looks 1, 7×7 looks 3.

KERNEL	SHELL EATEN	WHAT CHANGES IN THE RESULT
3×3	about 1 pixel	Gentle cleaning. Removes specks, but larger blobs survive. Little risk of damaging the line.
5×5	about 2 pixels	Erases blobs up to roughly 4–5 pixels. A good default.
7×7	about 3 pixels	Strong cleaning. Removes larger noise, but if the line has thin parts — a tip that narrows — it can eat them.

Table 27.2 — The rule: the larger the kernel, the more aggressive the erosion. The right choice is the smallest kernel that removes the noise that actually bothers you.

WHY THE CORE SURVIVES AND A THIN TIP DOES NOT A pixel survives erosion if all its neighbours inside the kernel are white. In the middle of the line, even with 7×7, they all are. At a thin tip, a 7×7 kernel already reaches the black background at the sides, so the pixel has black neighbours and dies. It is the same reason small blobs die: they are not wide enough to fill the kernel with white.

THE SAME KERNEL IN BOTH PHASES Opening deliberately uses the same kernel for the erosion and the dilation. The dilation exists to give the line back the shell the erosion ate: if the erosion took 2 pixels, the dilation must grow 2 back. Different kernels would deform the shape — larger on the dilation would thicken it, smaller would leave it thinner. The small blobs are gone for ever either way, because there is nothing left to give back.

```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (11, 11))
thresh = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)
```

Listing 27.1 — getStructuringElement builds the stamp: MORPH_ELLIPSE is an oval shape and (11, 11) its size in pixels. The server uses 11×11 rather than the 5×5 of the examples above, because at 640×480 the specks on this floor are larger. morphologyEx with MORPH_OPEN applies the erosion and the dilation in one call.

28. Proportional control, and how to compute Kp

Chapter 26 ends with one number: how many pixels the line is from the centre of the image. The car does not understand pixels; it understands a steering value from −1 to 1. This chapter is the conversion between the two.

28.1 THE PROBLEM

The error is in pixels — −30 means the tape is 30 pixels left of centre, +80 means it is to the right. But the car does not accept pixels: DIR runs from −1 to 1. The pixels have to be converted.

28.2 THE PROPORTIONAL IDEA

The direction is proportional to the error: the further the tape is from the centre, the harder you turn. Small error, gentle correction; large error, strong correction. It is a multiplication:

$$\text{DIR} = K_p \times \text{error}$$

Kp is the constant that converts pixels of error into how much to turn

The raw error cannot be used directly: an error of 80 would give DIR = 80, far above the maximum of 1. Kp is small precisely in order to bring pixels into the −1 to 1 scale.

28.3 COMPUTING Kp FROM THE WORST CASE

What is the largest error possible? The tape cannot be further away than the edge of the image. With a width of 640 the centre is at 320, and at worst the tape is at one edge:

$$e_{max} = \frac{\text{width}}{2} = \frac{640}{2} = 320 \text{ pixels}$$

That maximum error should correspond to DIR = 1, a full turn. So the mapping is error 0 → DIR 0, and error 320 → DIR 1, which gives:

$$K_p = \frac{1}{e_{max}} = \frac{1}{320} \approx 0.003125$$

written in code as Kp = 1.0 / (width / 2)

- Error 320, the tape at the edge → DIR 1.0, turn as hard as possible.
- Error 160, halfway → DIR 0.5.
- Error 0 → DIR 0. It fits the −1 to 1 scale exactly.

Kp IS NOT GUESSED It is computed from the width of the image. That is what makes it a sensible starting point rather than a number out of nowhere.

28.4 THE CATCH: IT STILL NEEDS TUNING

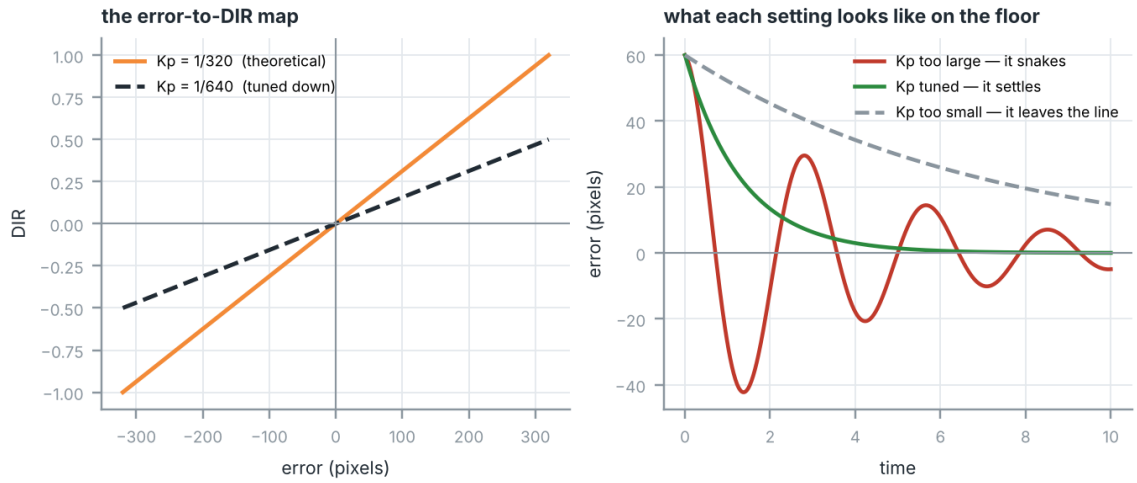


Figure 28.1 — The clean mapping on the left; what each setting produces on the floor, on the right.

Kp = 1/320 is the mathematically clean map: maximum error, maximum turn. In practice it is usually too aggressive. The car reacts on every frame, dozens of times per second. If it turns at full proportional strength on every small deviation, it overshoots the centre, corrects the other way, overshoots again — it snakes. That oscillation is the classic problem of proportional control.

STEP	WHAT TO DO
1	Compute the theoretical Kp: 1 / (width/2). The clean map, as a starting point.
2	Test it on the car. If it snakes, reduce Kp until it runs smoothly. The final value is often half or a third of the theoretical one — 1/640 or 1/960 rather than 1/320.
3	If it reacts too slowly and leaves the line on corners, raise it a little.

Table 28.1 — The calculation gives the right starting point; the fine tuning happens on the car, because it depends on speed, friction and inertia — things the mapping arithmetic does not know.

IF P ALONE IS NOT ENOUGH If it snakes and cannot be tuned out, the next step is adding the derivative term (D) of a PID controller, which anticipates the overshoot and brakes before it happens.

28.5 THE VALUES USED IN THE CODE

The server writes the same idea with one extra knob. The theoretical mapping 1/(width/2) is multiplied by `GAIN`, so the tuning of Table 28.1 becomes a single number:

```
Kp = (1.0 / (width / 2)) * GAIN      # GAIN = 2.5
if abs(error) <= DEADBAND:            # DEADBAND = 8 px
    steering = 0.0
else:
    steering = max(-1.0, min(1.0, Kp * error))
```

Listing 28.1 — Proportional steering as implemented in the PC server.

GAIN = 2.5 means the car reaches full lock when the line is 128 px from the centre, not at the edge of the image. On this car the theoretical value turned too gently for the corners; the dead band stops the small twitches that a higher gain would otherwise cause on straights.

29. Sharp corners and line recovery

STATUS This chapter records how the sharp-corner problem was approached. The recovery approach (B) and the extra curve speed are implemented in the current server, see 29.9. Very tight corners and corners seen too late (29.8) are still the main limitation of the line follower.

29.1 PROBLEM 1 — NOT ENOUGH STEERING ANGLE

On sharp corners — a straight followed by a very tight turn — the car loses the line while trying to turn. The cause is mechanical, not software: the car is differential, steering by the speed difference between the sides, and it does not have enough steering angle to describe the curve. Even with the direction command at maximum, the turning radius is too large and the line leaves the camera's field of view.

When the line leaves the camera's view in a corner, the car does the opposite of what it was doing: it backs up, steering the other way, until the line is back in view, and then follows it normally (Figure 29.1).

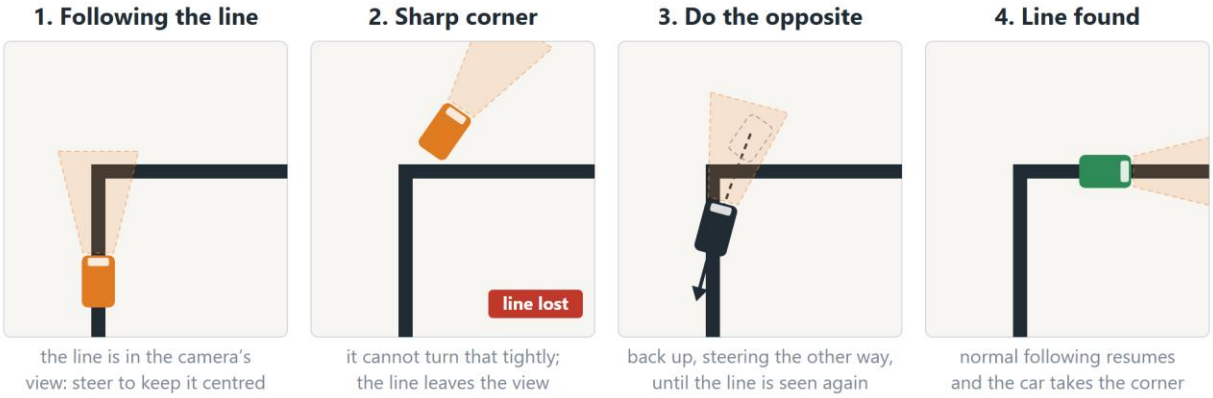


Figure 29.1 — Losing the line in a sharp corner and finding it again: the car does the opposite of what it was doing — backwards, steering the other way — until the line is back in view.

29.2 TWO APPROACHES UNDER STUDY

APPROACH	HOW IT WORKS
A — Preventive: reverse before losing the line	Detect that the corner is too tight while the line is still visible, and reverse before it is lost. The car reverses while holding the steering, which brings the line back to the centre of the field of view, then goes forward again, repositioned. The advantage is that the line is never actually lost, so there is no need to guess where it went.
B — Recovery: reverse after losing the line	A safety net for when the preventive approach is not enough and the line really does leave the field of view. This requires storing information beforehand: a flag saying the car was at full lock, and which way it was turning, so it knows which way to reverse. When the line is lost and there was a corner, the car stops and reverses opposite to the turn until the line reappears, then resumes normal following.

Table 29.1 — Hypothesis A is the main line of work; B is the fallback.

29.3 HOW TO TELL WHETHER THE CORNER IS DESCRIBABLE

METHOD	WHAT IT MEASURES
By the line's current position	Look at where the centre of the line is in the image: very close to the edge means it is about to leave. This can be measured two equivalent ways — by the centre coordinate relative to the edge, or by the error. They are the same information written differently: a large error means the line is near the edge. Simple, one reading per frame.
By the change over time	Compare the current error with the previous one. With the car at full lock: if the error is shrinking, the line is coming back to the centre and the corner is describable; if it stays at the edge or grows, it is too tight and the car must reverse. This sees where the line is going, not only where it is — the idea behind the derivative term of a PID. Finer, but harder to tune.

Table 29.2 — The simplest approach to start with is the first; the second is a refinement if the first is not enough.

29.4 WHY TWO CONDITIONS AND NOT ONE

The temptation is to trigger the reverse on one condition: line at the edge, reverse. That is simpler, but it would make the car reverse whenever the line reaches the edge — even on corners it could have taken by steering alone. On medium corners, where turning hard would have been enough, it would reverse unnecessarily and move in lurches.

- The line is very close to the edge, about to leave the field of view, **and**
- the car is already at full lock, meaning the computed direction is saturated at 1 or −1, so there is nothing more to be done going forward.

> **PLAN A AND PLAN B** Turning at full lock is plan A; reversing is plan B. The two conditions together guarantee that the car only reverses when there is no alternative — never on corners it could still have described by steering.

29.5 DESIGN DECISION — WHY REVERSE

ALTERNATIVE	WHY IT WAS NOT CHOSEN
Pivot, rotating in place	One side forward, the other backward, so the car rotates about its own centre with a radius near zero — the tightest possible turn. But the two L293D drivers share the same direction pins, so both sides are forced to the same direction. Pivoting would mean resoldering connections and using more ESP32 pins.
Servo steering	Wheels that physically turn, as in a real car. It would give a wide, smooth steering angle, but it would mean building a new steering mechanism: servo, tie rods, pivots in the chassis.
Reverse — chosen	It solves the problem with the hardware already present, with no resoldering and no new mechanics. It also prepares the car for situations no forward steering angle would resolve in one go.

Table 29.3 — Note that reverse and pivot are not the same: a pivot rotates the car in place, while reversing repositions it backwards to gain space and point at the line again.

29.6 CALIBRATION DEPENDS ON THE BATTERY

An important point for tuning: the motor power values are not absolute, they depend on the battery charge. The minimum percentage at which the motors start to turn is higher with a discharged battery than with a charged one, because with less voltage the motors have less torque and need more duty to overcome friction and move the car.

> **THE PRACTICAL CONSEQUENCE** A value that works well with a full battery can be insufficient at half charge — the car stops turning, or stops moving altogether. The minimum-speed parameters have to be tuned with this in mind, or revisited according to the state of the battery.

29.7 RECORD — THE TAPE AND THE REFLECTIONS

One of the biggest obstacles early in the project was choosing the right kind of tape and dealing with light reflections. The floor and the tape reflected the ambient light, and in the reflective areas the floor came out almost white — a brightness value near 255 — confusing itself with what was meant to be detected.

- First solution: switch to black tape, because the floor with reflected light was light (near 255) and black tape stood out by being dark. But the reflection problem partly remained.
- Final solution: switch to matte black tape, which does not reflect, and use a floor that also does not reflect — eliminating the false whites at source instead of trying to filter them afterwards.

> **THE LESSON** The contrast between tape and floor, and the control of reflections, matter more than any software tuning. Many detection problems are better solved in the scene — matte materials, even lighting — than in the code.

29.8 PROBLEM 2 — THE CAR SEES THE CORNER TOO LATE

The camera analyses only a narrow strip of floor immediately in front of the car — the region of interest. So the car only sees the corner when it is already right in front of it, and by then there is neither time nor space to react. On sudden corners the result is that the car simply continues straight, because it notices the corner too late.

> **THE TENSION TO RESOLVE** The narrow strip is what fixes the perpendicular-line blind spot of Chapter 25 and what makes the 90° corners work at all. Widening it, or adding a second strip further ahead, is the obvious direction — but it reintroduces the orientation problem the strip was chosen to avoid.

29.9 WHAT WAS IMPLEMENTED

Mechanism	How it works in the server
Recovery by reversing (approach B)	When no contour passes the filters, the state changes from FOLLOW to REVERSE. The car backs up at REVERSE_SPEED = 0.55 and steers to the side opposite to the last error, which swings the front back towards the line. As soon as a valid contour appears, the state returns to FOLLOW.
Extra speed in tight curves	When the steering exceeds CURVE_THRESHOLD = 0.70, the speed rises linearly from BASE_SPEED = 0.53 to MAX_CURVE_SPEED = 0.60 at full lock. A differential car turns by slowing one side; the extra speed gives the outer wheels enough force to complete the turn instead of stalling.
Memory of the last position	The contour closest to the last centre is followed, which keeps the car on its line through gaps and reflections.

Table 29.4 — The parts of this chapter that are now in the code.

```
if abs_dir > CURVE_THRESHOLD:
    strength = (abs_dir - CURVE_THRESHOLD) / (1.0 - CURVE_THRESHOLD)
    move = BASE_SPEED + strength * (MAX_CURVE_SPEED - BASE_SPEED)
else:
    move = BASE_SPEED
```

Listing 29.1 — The curve speed, computed in the main thread from the steering.

The preventive approach (A) was not needed once the curve speed and the recovery worked together. The corner-seen-too-late problem (29.8) remains open.

PART VI

Phase I reference and handover

Parts I to V explained how the system works. This part is what a new person on the team needs in order to run it, test it and change it: the hardware and protocol at a glance, the safety mechanisms, a reference of every function, the seven test scenarios that exercise every path through the firmware, and an honest account of where Phase I ended.

> **WHY THIS PART** A reference for anyone who needs to run, test or change the Phase I car without reading the whole report first.

30. System reference

The car is a remote-control robot with two motors driven by PWM through an H-bridge, controlled from a PC over Wi-Fi. The firmware is written in C++ on the Arduino framework, and three principles guided the design decisions:

- Robustness — the robot has to keep working when things fail: the Wi-Fi drops, the PC shuts down, the router restarts.
- Real-time response — when the operator sends stop, the motors stop immediately, with no latency.
- Configuration without tools — anyone can set up a new Wi-Fi network without connecting the ESP32 to a computer.

30.1 HARDWARE

COMPONENT	ESP32 PINS	FUNCTION
H-bridge (direction)	IN1=12, IN2=13	Motor direction control
H-bridge (speed)	EN1=14, EN2=15	PWM signal for speed
Wi-Fi (internal)	ESP32 chip	TCP/IP communication
Flash (internal NVS)	ESP32 chip	Persistent memory for credentials

Table 30.1 — Phase I hardware. The camera pins of Part II are additional to these.

30.2 THE TWO MODES, AT A GLANCE

MODE	WHEN	WHAT IT DOES
A — Configuration (Access Point)	No Wi-Fi network saved in flash	Creates its own Wi-Fi network and listens on port 1883 for someone to send credentials. On RESET:NOW it restarts and tries Mode B.
B — Operation (Station + TCP client)	At least one network saved in flash	Scans the air, joins the best available network, then connects to the PC and begins receiving commands. If no network works, it falls back to Mode A.

Table 30.2 — Chapters 9 and 10 cover the decision in detail.

30.3 THE MESSAGE PROTOCOL

Every message exchanged between the ESP32, the configuring phone or PC, and the control PC ends with a newline. There are three commands:

```
WIFI:NetworkName,NetworkPassword\n     -> save credentials to flash (AP mode)
RESET:NOW\n                            -> restart the ESP32 (AP mode)
MOV:speed,direction\n                  -> movement command (operation mode)
```

Listing 30.1 — The complete protocol. The newline is the delimiter of Chapter 13.

30.4 THE 2.4 GHz LIMITATION

CRITICAL The ESP32 can only join Wi-Fi networks in the 2.4 GHz band. It cannot see, and cannot connect to, 5 GHz networks.

The reason is purely physical, and it is about the chip's radio hardware:

REASON	EXPLANATION
Antenna and RF circuits	The ESP32's antenna is designed to operate in the 2.400–2.4835 GHz range. The physical dimensions of the antenna and the radio-frequency components are optimised for that band.
Cost	Adding 5 GHz support would require extra components — amplifiers, filters, dual-band antennas — making the chip more expensive. The ESP32 was designed to be affordable.
Range	2.4 GHz has greater range and better penetration through walls than 5 GHz. For IoT and robotics that is an advantage: the robot can be further from the router.
Power consumption	Transmitting at 5 GHz uses more energy. For a battery-powered device, 2.4 GHz is more efficient.

Table 30.3 — Four reasons that all point the same way.

The most common problem follows from this. Many modern routers broadcast the same SSID on both bands:

SSID	BAND	DOES THE ESP32 SEE IT?
Vodafone-05718A	2.4 GHz	Yes
Vodafone-05718A	5 GHz	No

Table 30.4 — The user sees one name on the phone, but the router may be connecting them to the 5 GHz version.

- Separate the SSIDs in the router (recommended): give the two bands different names, for example Home-2G and Home-5G. That gives full control.
- Check that 2.4 GHz is enabled: some routers allow it to be switched off.
- Force the phone onto 2.4 GHz while configuring, before sending credentials to the ESP32.

HOW TO CONFIRM THE BAND Android: Wi-Fi → network name → details → check Frequency, which should read 2.4 GHz. Windows: Settings → Network → connection properties → check Network band. Router: open the admin page and look at the connected-device list, which normally shows the band for each one.

31. The safety mechanisms

A car controlled over Wi-Fi must never depend on the network in order to stop. Wi-Fi drops, the PC freezes, the router restarts — and at that moment the last command received may still be "full speed forward". These are the mechanisms that make the car stop on its own.

31.1 HEARTBEAT

Imagine the PC's network cable is unplugged while the robot is driving forward. With no protection, the robot would carry on for ever. The heartbeat solves this: whenever the ESP32 receives a message from the PC it updates a timestamp, and the loop checks how long ago that was.

```
if (millis() - lastHeartbeat > 250) {
    stop_motors();   // the PC has been silent for more than 250 ms
}
```

Listing 31.1 — The Phase I threshold. Chapter 11 describes the layered version this grew into.

The PC therefore has to send commands at least four times a second for the robot to keep moving. If it goes quiet, the motors stop within 250 ms — almost instantaneous to a human, but long enough to tell a real failure apart from a small network delay.

31.2 ANTI-LAG — FLUSHING THE BUFFER

The other real-time problem: the operator sends "forward, forward, forward, stop" in quick succession. If the ESP32 is busy for an instant, those four messages pile up in the buffer. Processing them one at a time would leave the robot still driving forward when the operator has already said stop.

```
while (client.available() > 0) {
    memset(buffer, 0, sizeof(buffer));
    client.readBytesUntil('\n', buffer, 127);   // overwrite
    lastHeartbeat = millis();
}

// on leaving the while, buffer holds the LAST command
if (strncmp(buffer, "MOV:", 4) == 0) {
    processar_comando(buffer);
}
```

Listing 31.2 — Read everything in one pass, discard the old, act on the most recent. Chapter 23 is the reasoning; Chapter 15 is when the opposite rule applies.

31.3 WHAT return MEANS INSIDE loop()

A detail about the Arduino framework that matters here: using `return` inside `loop()` does not exit the program. It exits that one iteration. Arduino calls loop() again immediately, for ever.

> **THE PRACTICAL IMPLICATION** In the main loop, if the PC is not connected, the code returns to exit. But Arduino calls loop() again at once, so it tries to connect again. As long as the ESP32 has Wi-Fi it keeps trying to reach the PC indefinitely, waiting 5 seconds between attempts. For a robust system, that is the correct behaviour.

32. Function reference

Every function in the Phase I firmware, with what it does and where it is called from.

32.1 INITIALISATION

FUNCTION	CALLED FROM	WHAT IT DOES
`setup()`	automatic	Runs once at power-on. Initialises the hardware and decides the mode, AP or operation.
`pinos_setup()`	`setup()`	Configures the four H-bridge pins as OUTPUT and drives them LOW.
`pwm_channel_setup()`	`setup()`	Binds EN1 and EN2 to PWM channels 0 and 1, at 1000 Hz, 8-bit resolution.
`setup_flash_memory()`	`setup()`	Initialises the NVS. If the partition is corrupted, it erases and restarts.

Table 32.1 — Initialisation functions.

32.2 WI-FI AND CONFIGURATION

FUNCTION	CALLED FROM	WHAT IT DOES
`enable_access_point()`	setup / fallback	Turns on AP mode, creates the provisioning network and opens the server on port 1883.
`loop_config_mode()`	setup / fallback	A blocking infinite loop accepting TCP in AP mode. Reads WIFI: and RESET:NOW.

Table 32.2 — Provisioning functions. loop_config_mode() never returns, which is why Chapter 10 matters.

32.3 FLASH (NVS)

FUNCTION	CALLED FROM	WHAT IT DOES
`Write_to_flash(ssid, pass)`	`loop_config_mode`	Writes Wi-Fi credentials as a blob in flash, under the key wifi_N.
`dump_nvs_to_serial()`	Write_to_flash / debug	Prints every saved network to the Serial Monitor.

Table 32.3 — Persistence functions. Chapter 12 covers the NVS API they rest on.

32.4 MAIN LOOP

FUNCTION	CALLED FROM	WHAT IT DOES
`loop()`	automatic	The main iteration. Checks Wi-Fi, TCP and the heartbeat; reads and processes MOV:.
`stop_motors()`	loop(), on failures	Stops the motors immediately.
`processar_comando(data)`	`loop()`	Receives a MOV: command and calls motor_logic().
`motor_logic(speed, steering)`	`processar_comando`	Computes the individual PWM for each motor. Chapter 2 has the implementation.

Table 32.4 — Operation functions.

33. Test scenarios

Before changing the code it is worth testing every path, to see how the system reacts. Open the Serial Monitor at 115200 baud and follow the scenarios. Running all seven before each commit is a project rule.

33.1 SCENARIO 1 — EMPTY FLASH, FIRST TIME

#	ACTION	EXPECTED SERIAL OUTPUT	RESULT
1	Power on the ESP32	--- ACCESS POINT MODE --- IP: 192.168.4.1	AP active
2	Join the provisioning network from a phone	(no output)	Network visible
3	Send over TCP to 192.168.4.1:1883: WIFI:Name,Password	OK: network saved.	Saved
4	Send: RESET:NOW	OK: restarting...	Restarts

Table 33.1 — The provisioning path of Chapter 9.

33.2 SCENARIO 2 — A SAVED NETWORK IS AVAILABLE

#	ACTION	EXPECTED SERIAL OUTPUT	RESULT
1	Power on the ESP32	Trying: Name / Connected to: Name	Joined
2	Wait — automatic	Trying to contact the PC...	Retries every 5 s
3	On the PC, run a listener on port 1883	Connected successfully!	TCP established
4	Send from the PC: MOV:100,0.5	Command received: MOV:100,0.5	Processed

Table 33.2 — The normal operating path.

33.3 SCENARIOS 3 AND 4 — NETWORKS THAT FAIL

#	CONDITION	EXPECTED BEHAVIOUR	RESULT
3	Two or more networks saved; the strongest is unavailable	Trying: StrongNet ... Failed: StrongNet, then Trying: AltNet / Connected to: AltNet	Joins the second
4	Every saved network unavailable	Trying: Net1 / Failed, Trying: Net2 / Failed, then All networks failed. Rescue mode.	Enters AP

Table 33.3 — These two scenarios are what Improvement 2 in Chapter 16 exists to pass.

33.4 SCENARIOS 5 TO 7 — FAILURES DURING OPERATION

#	ACTION	EXPECTED BEHAVIOUR	RESULT
5	Switch off the router while the ESP32 is running	The motors stop; after 10 s the ESP32 restarts	stop_motors, then restart
6	Close the listener on the PC	The motors stop; after 5 s, Trying to contact the PC...; reopening it resumes	Reconnect loop
7	The PC stops sending for 250 ms	The motors stop silently; when MOV: resumes, the command is received	Heartbeat

Table 33.4 — Each of these exercises one layer of the loop described in Chapter 11.

34. Results, limitations and where Phase I ended

34.1 WHAT WORKS

The car follows a line autonomously on a suitable floor, using the full pipeline of Part V, and switches cleanly between manual and autonomous modes from the gamepad. Getting there meant working through real engineering problems rather than tutorial ones:

- Reflections that read as false white, until matte tape and a non-reflective floor fixed the contrast at the source.
- A video path that had to move from TCP to UDP to stay real-time.
- A threading architecture — capture, processing, control — with a most-recent-frame policy, so the car always acts on the present.

34.2 KNOWN LIMITATIONS

LIMITATION	DETAIL
Turning radius	Shared direction pins prevent an in-place pivot. Very tight curves can exceed the car's turning ability, and the reverse recovery helps but does not fully solve them.
Detection depends on the environment	Contrast between line and floor, and controlled lighting, matter more than any software tuning.
Wi-Fi range	The on-board antenna is weak. Far from the router the link degrades, and a long enough dropout restarts the chip.

Table 34.1 — The three limitations that shaped the Phase II planning in Part VII.

34.3 SUGGESTED DIRECTIONS

DIRECTION	WHAT IT WOULD GIVE
A Raspberry Pi as a central server	Always on at around 5 W, so it can run 24/7. It could manage several robots at once, offer a web dashboard reachable from any device, keep centralised logs of commands and errors, and provide the processing power for computer vision. Each ESP32 would identify itself on connecting, and the Pi would keep a table of active robots.
FreeRTOS tasks	The ESP32 already ships with FreeRTOS. Separate tasks for TCP reception, motor control at 100 Hz, video streaming and sensors would let the camera, movement and telemetry all run without blocking one another. Part II already took the first step, by giving the camera its own task.
TLS encryption	Commands currently travel in plain text, so anyone on the same network can read or alter them. The ESP32 supports TLS natively: replacing WiFiClient with WiFiClientSecure encrypts the whole TCP conversation.
Other ideas	A mobile surveillance mode combining camera, movement and motion detection; a dedicated phone app with a virtual joystick instead of a terminal listener; a REMOVE:SSID command to delete one network from flash without a full erase; and obstacle avoidance using distance sensors.

Table 34.2 — Directions considered at the end of Phase I. The Raspberry Pi line became Part IX.

> **ON SECURITY WORK** Never implement features that attack the Wi-Fi networks around the robot. Besides being illegal in most countries, it is unethical and can cause serious harm. Protect your own system; do not compromise anyone else's.

34.4 OPEN TASKS AND PROJECT RULES

- Add ESP32-CAM support: video streaming in parallel with control. (Done in Part II.)

- Increase the global buffer from 32 to 128 bytes, to support long SSIDs.
- Add a `REMOVE:SSID` command, to delete networks from flash without a full flash erase.

PROJECT RULES Never use the String class — only fixed-size char arrays (Chapter 13). Never call nvs_open without a matching nvs_close (Chapter 12). Run all seven test scenarios before every commit. Document each new function with a comment at the top explaining its parameters and return value.

PART VII

Planning the second phase

The point where the project stops being a line follower. This part is the plan as it stood before Phase II began: the questions that had to be answered, the trade-offs behind each one, and the argument for which order to answer them in.

WHY THIS PART Phase I ended with a car that could drive itself along a line. The next question was what it should be useful for, and the answer was putting out a fire. This part records the questions asked before building anything, as they stood at the time.

STATUS OF THIS PART This is a historical plan. It records the questions that guided the second phase. The implementation that followed uses a Raspberry Pi 5, a pan/tilt mechanism and MOSFET switching for the pump — not every alternative below was adopted. Parts VIII and IX are what was actually built.

35. The open questions

The idea: the car stops merely following a line and becomes a small autonomous robot that patrols a route, detects a fire with a dedicated camera, stops, puts it out with water, and carries on. Six areas had to be resolved.

35.1 THE WATER SYSTEM

COMPONENT	WHAT IT IS FOR
Water pump	A small pump, of the 5–12 V kind used in Arduino projects, pushing water from the reservoir out through a hose.
Reservoir	An on-board tank. The size is a compromise: more water puts out more fire, but it weighs more and takes up chassis space.
Relay	The pump draws more current than a processor pin can supply, so it cannot be connected directly. The relay is a switch controlled by the processor: the pin sends a weak signal to the relay, and the relay switches the pump's heavy current. It is the bridge between the control signal and the power.

Table 35.1 — Open questions: what flow and pressure does the pump need to put out a small flame at short range? What reservoir capacity balances endurance against weight? A plain relay, or a relay module with protection already included?

35.2 POWER — IS THE BATTERY ENOUGH?

The pump is a new and heavy consumer, joining the motors, the processor and the camera. Whether the current battery can carry all of it has to be checked, especially at the peak — when the pump switches on while the motors are already running.

- What is the peak draw with pump, motors, processor and camera all at once?
- Does the current LiPo supply that current without the voltage sagging?
- Alternatives: a larger battery, a separate battery for the pump, or a capacitor to cover the peak.

ALREADY KNOWN FROM PHASE I Battery voltage affects the minimum motor power (Chapter 29). With one more large consumer, voltage sags may affect the car's behaviour — to be verified.

35.3 PROTECTING THE ELECTRONICS

With water on board, the electronics are exposed to splashes. A cover or enclosure is needed to protect the processor, the H-bridges and the connections from water and spills. Open: a closed 3D-printed box, or a lid over the electronics area? And how do the wires for motors and pump pass through while keeping the seal?

35.4 A NEW PROCESSOR AND CAMERA FOR FIRE DETECTION

The idea: a dedicated camera, possibly rotating 360°, looking only for fire, while the main camera keeps following the route. On detecting fire the car leaves the line, approaches, extinguishes, and resumes.

- A rotating 360° camera driven by a motor or servo, or a fixed wide-angle one?
- Fire detection by colour and brightness — simple, like the line — or by a neural network, more robust but heavier?
- Is the current ESP32 enough for two cameras plus the detection, or is something more capable needed?

35.5 WHERE THE PROCESSING RUNS

The central architectural decision. Today the processing is on the PC and the ESP32 only captures and sends. For the robot to be genuinely autonomous the processing should move on board — but the ESP32 has no muscle for heavy vision.

OPTION	THE TRADE-OFF
Keep the PC as the brain	Simple, but the robot stays tied to the PC and to the Wi-Fi.
Move to a capable on-board computer	A Raspberry Pi or a Jetson, for real autonomy.
Hybrid	Simple detection on board, heavy processing on the PC.

Table 35.2 — Tied to this: if there are two processors — an ESP32 for the motors and a Pi for the vision — what channel do they talk over, and which one is in charge?

35.6 EMERGENCY STOP

A robot that moves on its own and carries water needs a reliable way to stop it immediately — an interruption that cuts the movement regardless of what the software is doing. A physical emergency button that cuts power to the motors, or a software interrupt that forces a stop? Ideally both: physical for real safety, software for remote control.

35.7 THE SUGGESTED ORDER

One possible order, running from what unblocks the rest to what depends on the others: first power (35.2), because it decides what is possible at all; then the water system (35.1) and the protection (35.3), which are the core of the new function; then fire detection (35.4) and the architecture decision (35.5), which is the largest; with the emergency stop (35.6) accompanying all of it from early on, for safety.

36. Why the model has to be tested first

Training and measuring a first model early gives useful numbers: resolution, memory, inference time and processor load. But software and hardware influence each other. Camera quality, lighting, mechanical stability, water range and available power also limit what the network can do.

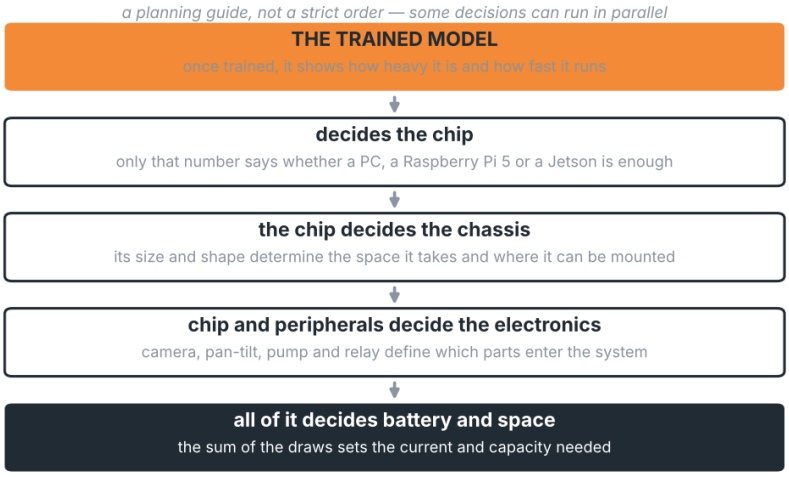


Figure 36.1 — The dependency cascade that argues for training first.

With a first measurement of the model, the hardware study gains a concrete basis. Processors can be compared and the required resources estimated, while the camera, the mechanics and the power circuit are tested in parallel.

- The model helps choose the processor.
- The processor, the mechanical load and the peripherals constrain the chassis.
- The average and peak draws help size the power supply.

> **THE PRACTICAL ADVICE** A good first step is to try the detection on the PC and, in parallel, measure what the camera and the water system can actually do. The aim is to reduce unknowns before consolidating the design — not to wait for the software to solve physical limitations. Every decision should be checked against real tests.

PART VIII

Neural networks and fire detection

Part V made the point that following a line needs no learning at all: the rule is simple enough to write by hand. Finding a flame is not. This part is the machinery that solves the problems you cannot write rules for — what a convolution computes, why layers are stacked, how a network is trained, how a detection becomes a position in the image, and then the actual implementation: the datasets, the code, and how to measure whether it learned anything.

> **WHY THIS PART** To put out a flame, the robot first has to find it in an image. A line could be found with a hand-written rule; a flame cannot. So the project trained a neural network, FireNet. This part builds one from the ground up — a single neuron, a filter, an activation, training — and ends with the network that runs on the robot.

37. From pixels to a decision

37.1 THE PROBLEM — AN IMAGE IS ONLY NUMBERS

To a computer an image is not a scene. It is a grid of numbers: each pixel is a value from 0 to 255, or three values for red, green and blue, exactly as Part II described them leaving the sensor. The sentence “there is a flame in the top-right corner” is written nowhere in that grid. It has to be extracted from the raw numbers.

> **THE WHOLE PROBLEM, IN ONE SENTENCE** How do you turn a grid of numbers into a meaningful answer such as “flame, here”? Everything in this part is the machinery that answers that one question.

37.2 THE NEURON: ONE SMALL CALCULATION

Everything in a neural network is built from one object, repeated many times. A neuron receives several numbers, multiplies each by its own **weight** — how much that input counts — adds the products, adds one more number called the **bias**, and passes the result through an **activation function**.

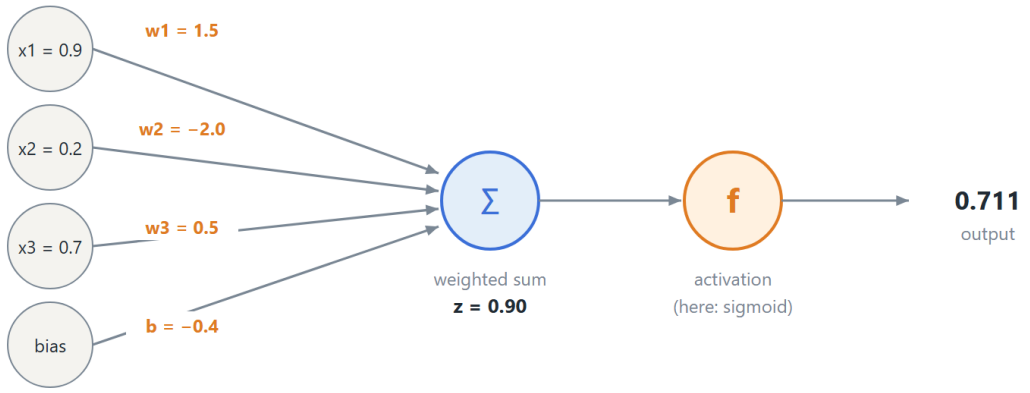


Figure 37.1 — A neuron with three inputs and a bias. Σ computes the weighted sum z; f turns it into the output.

```
z = w1·x1 + w2·x2 + w3·x3 + b
  = 0.9 × 1.5  +  0.2 × (−2.0)  +  0.7 × 0.5  −  0.4
  = 1.35 − 0.40 + 0.35 − 0.40  =  0.90
output = sigmoid(0.90) = 0.711
```

Listing 37.1 — The same neuron, computed by hand.

Two of those numbers are negative on purpose. A negative weight means that input lowers the sum: it is the neuron's way of saying "this measurement is evidence against a flame". The bias moves the point at which the sum crosses zero. Without it, z would always be 0 when all inputs are 0, and the neuron could only separate the data around that point, which rarely has anything to do with the problem. With a bias, `z = w·x + b` crosses zero wherever the data actually splits.

Training (Chapter 38) is nothing more than finding the weights and biases that make the outputs right.

37.3 THE BUILDING BLOCK — CONVOLUTION

The most fundamental mechanism in vision is the convolution, and it is already familiar from this project: it is the same idea as the morphology kernel of Chapter 27 — a small stamp that slides across the whole image.

A convolution is a small filter, a 3×3 matrix say, that passes over every position of the image. At each position it multiplies the pixels underneath by its own values, sums the result, and produces one number. That number says how strongly this patch matches the pattern the filter is looking for.

$$\text{out} = \sum_i \sum_j \text{patch}(i,j) \times \text{filter}(i,j)$$

one output value, from one position of the filter

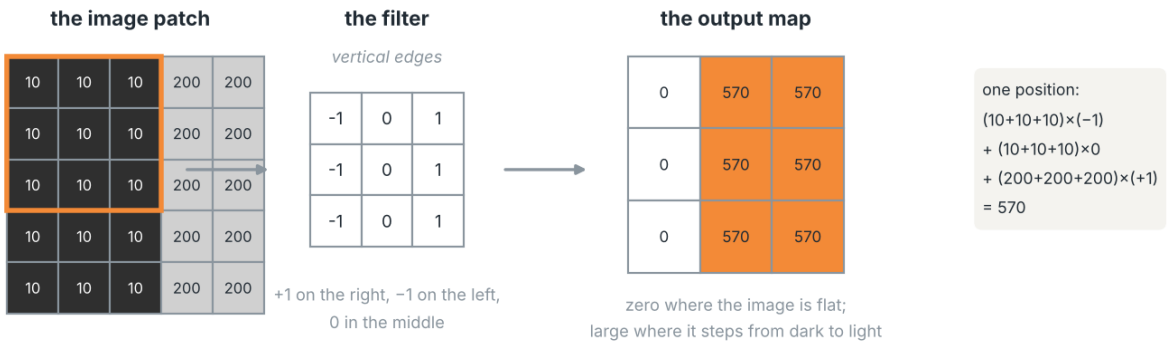


Figure 37.2 — A vertical-edge filter on a patch that steps from dark to light, computed in full.

- Where the image has a vertical transition — dark on the left, light on the right — the sum comes out large.
- Where the image is uniform, it comes out zero.
- Passing it over the whole image gives a new map where vertical edges are highlighted and everything else is zero.

> **THE JUMP FROM HERE** That map is not a decision yet — it is a measurement of one pattern at every position. The rest of this chapter is how such measurements are combined into an answer.

37.4 A FILTER IS A NEURON REUSED EVERYWHERE

A neuron that looked at the whole image at once would need one weight per pixel and would have to learn separately what a flame looks like in every corner. A convolution filter avoids that: it is a neuron with 9 inputs (a 3×3 patch) whose **same 9 weights** are reused at every position. That is why a filter finds the same pattern wherever it appears — the weights do not change from place to place — and why a whole layer of FireNet has only a few thousand weights instead of millions.

> **WHERE THE FILTER STARTS** The clean −1 0 1 filter of Figure 37.2 was chosen to make the arithmetic readable. In a real network the 9 weights start as random numbers and detect nothing. A filter like −1 0 1 is where training can end, not where it begins.

37.5 FROM EDGES TO A FLAME — STACKING LAYERS

A single edge is not a flame. The trick is to stack convolutions in layers, each working on the output of the previous one, seeing more complex patterns each time.

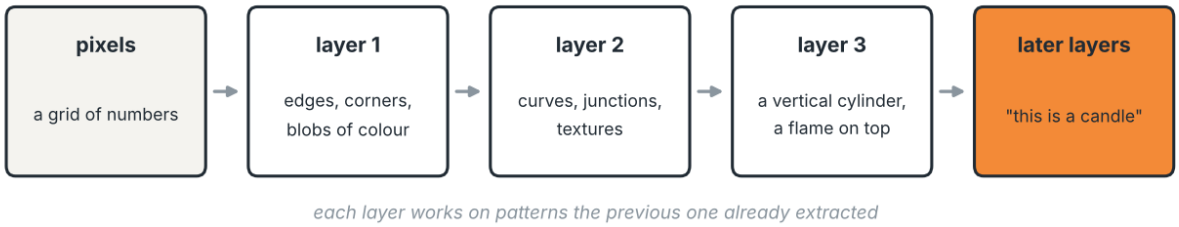


Figure 37.3 — Each layer works on patterns the previous one already extracted.

A stack like this is a Convolutional Neural Network, a CNN — the foundation of all vision AI. Because it has many stacked layers it is called deep learning: deep means many layers, and more layers means more complex patterns, but heavier to train and to run.

37.6 ReLU — WHAT MAKES STACKING WORTH ANYTHING

After each convolutional layer comes a tiny operation called an activation, and the most common one is ReLU. The rule could not be simpler: any negative number becomes zero, positives stay as they are.

$$\mathrm{ReLU}(x) = \max(0, x)$$

applied to every number in the feature map

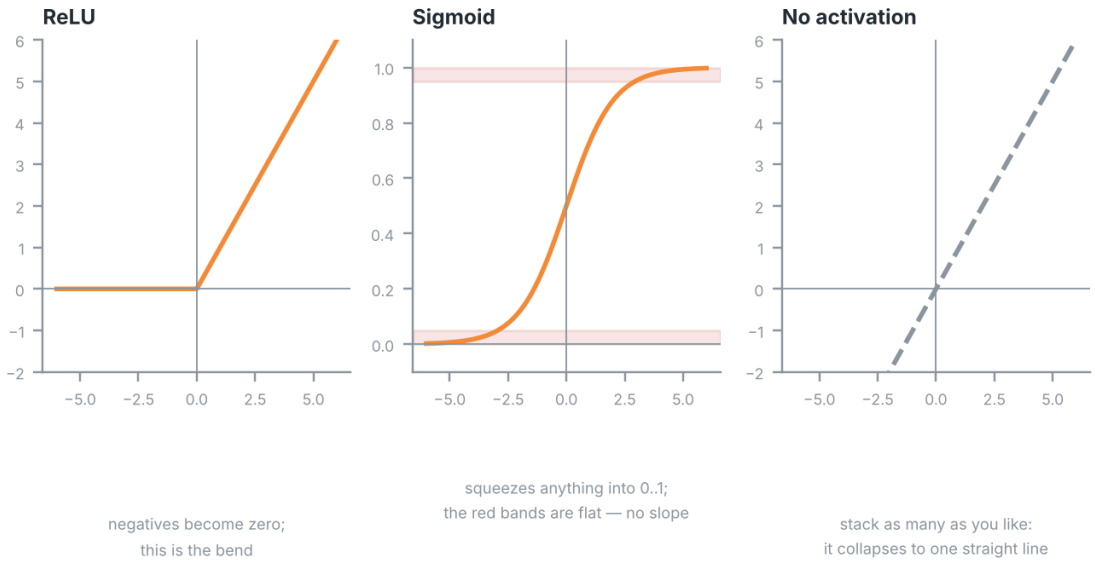


Figure 37.4 — ReLU, sigmoid, and what happens with no activation at all.

WHY IT IS NEEDED Without something like ReLU, stacking layers would be pointless: mathematically, many stacked layers of pure multiply-and-add collapse into a single layer, and a single layer can only learn very simple things. ReLU breaks that collapse. By zeroing the negatives it introduces a bend, which lets the network combine its layers into genuinely complex patterns — the difference between only being able to draw straight lines and being able to draw any shape. Sections 37.7 to 37.11 show this step by step, with numbers.

37.7 WHAT A FILTER OUTPUT OF 570, 0 AND −570 MEANS

Take a row of 5 pixels, 10, 10, 200, 10, 10: the simplest picture of a flame, a bright column on a dark background. It has **two transitions**: dark to light on the left of the flame, and light to dark on the right. Slide filter A = (−1, 0, 1) along it. At each position the filter computes "right pixel minus left pixel", so it answers one question: how much brighter is the right side than the left?

```
pixels:             10    10   200    10    10
position 1:  −10 + 0 + 200 = 190    (dark -> light: left edge of the flame)
position 2:  −10 + 0 +  10 =   0    (both sides equal: no edge)
position 3: −200 + 0 +  10 = −190   (light -> dark: right edge of the flame)

over 3 identical rows (a 3×3 filter):   570    0   −570
```

Listing 37.2 — The output 570, 0, −570: one number per position of the filter.

So **570** says "there is a dark-to-light edge here", **0** says "nothing changes here", and **−570** says "there is a light-to-dark edge here". The size tells how strong the edge is; the sign tells which way it goes.

> **THE OBVIOUS DOUBT** ReLU turns −570 into 0. Does that not throw away the right edge of the flame? No — because a layer never has only one filter. Training also produces the mirror filter B = (1, 0, −1), which answers the opposite question and gives −570, 0, 570. After ReLU, filter A keeps 570, 0, 0 (the left edge) and filter B keeps 0, 0, 570 (the right edge). Nothing is lost: each edge now lives in its own map, as a positive number.

37.8 WHY RELU IS NEEDED: TWO FILTERS THAT CANCEL

The next layer reads all the maps and adds them, each with a weight. Take the simplest case, both weights equal to 1:

	Position 1 (left edge)	Position 2 (no edge)	Position 3 (right edge)
Filter A, raw	570	0	−570
Filter B, raw	−570	0	570
Next layer adds them, no ReLU	0	0	0
Filter A after ReLU	570	0	0
Filter B after ReLU	0	0	570
Next layer adds them, with ReLU	570	0	570

Table 37.1 — Without ReLU both edges vanish in the sum; with ReLU both survive.

Both filters found both edges, and without ReLU the next layer still receives nothing: every positive number meets a negative one and they cancel. Filter B was only filter A multiplied by −1, so it added no information at all. With ReLU each filter has places where it is exactly zero, a zero cannot cancel anything, and both edges reach the next layer.

37.9 WHY RELU IS NEEDED: THE COLLAPSE, STEP BY STEP

The cancellation above is not bad luck with the numbers. Without an activation, any stack of layers is one multiplication. Take one input x, two hidden neurons h1 and h2, and one output y, with no activation:

```
h1 = w1·x                       h2 = w3·x
y  = w2·h1 + w4·h2
   = w2·(w1·x) + w4·(w3·x)      replace h1 and h2
   = (w1·w2)·x + (w3·w4)·x      regroup
   = (w1·w2 + w3·w4) · x
   =        W        · x        one neuron with weight W
```

Listing 37.3 — The algebra of the collapse.

With numbers: w1 = 2, w2 = 3, w3 = 1, w4 = 4.

```
x = 5:   h1 = 10,  h2 = 5,   y = 3·10 + 4·5 = 50
one neuron with W = 2·3 + 1·4 = 10:   y = 10 · 5 = 50      same answer, for every x
```

Listing 37.4 — Three neurons give exactly what one neuron gives.

Adding a third layer only multiplies W by more numbers; adding biases only adds a constant. However deep the stack, without an activation it is a single `y = W·x + b`. Every extra neuron is wasted.

37.10 ONE NEURON CAN ONLY DRAW A STRAIGHT LINE

Reduce each image to two measurements between 0 and 1: x1, the amount of orange, and x2, the brightness of the brightest spot. A candle flame is orange but not completely, bright but less than a lamp, so flames sit in the middle of the plot and everything else surrounds them.

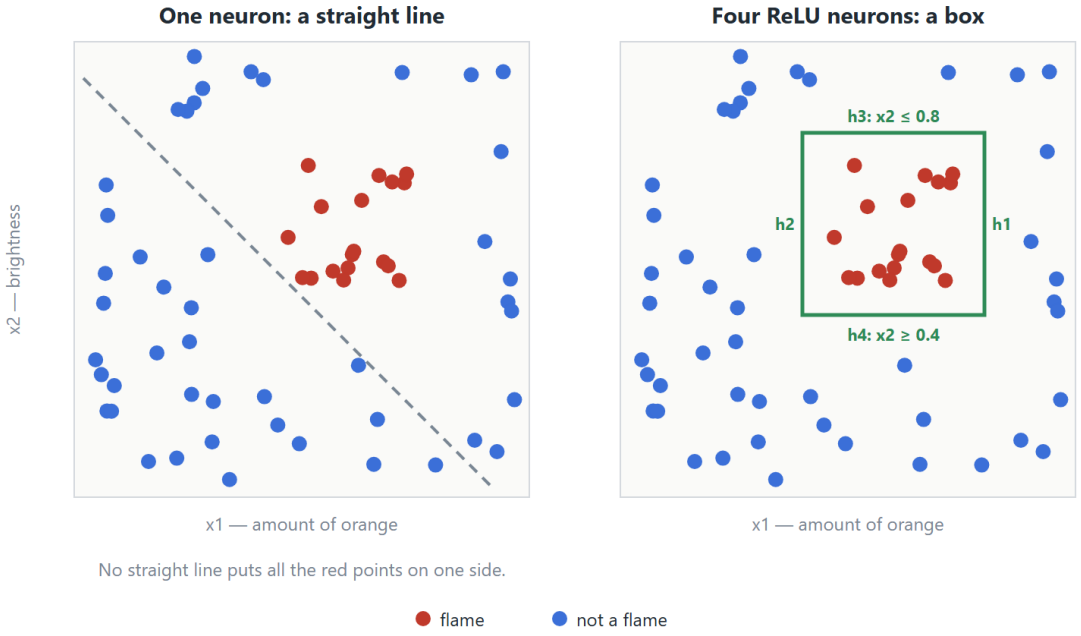


Figure 37.5 — Left: the boundary of one neuron is always a straight line. Right: four ReLU neurons enclose the flames.

A neuron decides by the sign of `z = w1·x1 + w2·x2 + b`. The points where z = 0 form a straight line, and by the collapse above, adding more neurons without an activation still gives one straight line. A region surrounded on all sides cannot be cut out with one line.

37.11 HOW RELU FENCES IN THE ANSWER

ReLU turns each neuron into a **wall**: zero on the safe side, growing on the other. Put one wall on each side of the flame region, written so that it stays at zero while its limit is respected:

```
h1 = max(0, x1 − 0.8)     # wall on the right:  too much orange
h2 = max(0, 0.4 − x1)     # wall on the left:   too little orange
h3 = max(0, x2 − 0.8)     # wall on top:        too bright
h4 = max(0, 0.4 − x2)     # wall at the bottom: too dark
S  = h1 + h2 + h3 + h4    # how far outside the fence
score = 1 − 5·S           # "flame" when score > 0.5
```

Listing 37.5 — The fence of Figure 37.5, as four neurons and an output.

Point (x1, x2)	h1	h2	h3	h4	S	Score	Answer
(0.60, 0.60) inside	0	0	0	0	0	1.00	flame
(0.85, 0.60) a bit too orange	0.05	0	0	0	0.05	0.75	flame (near the edge)
(0.95, 0.60) far too orange	0.15	0	0	0	0.15	0.25	not a flame
(0.20, 0.90) dark-orange, very bright	0	0.20	0.10	0	0.30	−0.50	not a flame

Table 37.2 — Testing the fence: only points inside get a high score.

Every wall is zero or positive, never negative, so one wall can never cancel another. S is zero only when all four limits are respected at the same time, and crossing any single wall is enough to lower the score.

> **WHY THE RELU IS ESSENTIAL HERE** Remove the max() and the four walls become plain linear neurons. Add them: S = (x1 − 0.8) + (0.4 − x1) + (x2 − 0.8) + (0.4 − x2) = −0.8. The x1 and x2 terms cancel and S is the same number everywhere — the fence disappears completely. That is the collapse of 37.9 happening in front of you.

In a trained network nobody writes these numbers: training finds the walls by itself, and with more neurons it can build fences of any shape.

37.12 HOW MANY NEURONS

The number of neurons in a layer is chosen before training, and it decides how many walls the fence can have. Too few, and the fence cannot follow the shape of the data; too many, and it can wrap around every single training image, noise included.

> **THE STUDENT AND THE EXAM** A student who scores 99% on the exercises they studied but 72% on a new test has memorised the exercises instead of learning the subject. A network does the same: a high score on the training images and a low score on validation images means it memorised them. A student with 80% on both has learnt what they know properly but is still missing something — the network needs better data or more capacity, not less.

Training score	Validation score	Diagnosis
Low	Low	Too few neurons or layers, or bad labels: the fence is too simple.
High	Low	Memorised the training images: more varied data, augmentation (38.9), or a smaller model.
High	High	About right.

Table 37.3 — Reading the model from the two scores.

37.13 POOLING — SHRINKING, AND STAYING ROBUST

Pooling reduces the size of a feature map, keeping only what matters. The most common kind, max pooling, divides the map into small squares — 2×2, say — and keeps only the largest value from each. The map comes out four times smaller.

- Less data to process, so the network is faster and lighter.
- More subtly, it makes detection robust to position. If the flame shifts three pixels, the exact pixels change but the strongest response in each square survives the shrink, so the pattern is still recognised. Pooling teaches the network to care that a feature is there, not exactly which pixel it sits on.

37.14 THE SIGMOID: FROM A RAW SUM TO A PROBABILITY

The raw sum of the last neuron has no limits: it could be −20 or 500. To read it as a confidence it is squeezed between 0 and 1 by the sigmoid:

```
sigmoid(z) = 1 / (1 + e^(-z))
sigmoid(4.3) = 1 / (1 + 0.0136) = 0.987      # 98.7% confident
```

Listing 37.6 — The sigmoid.

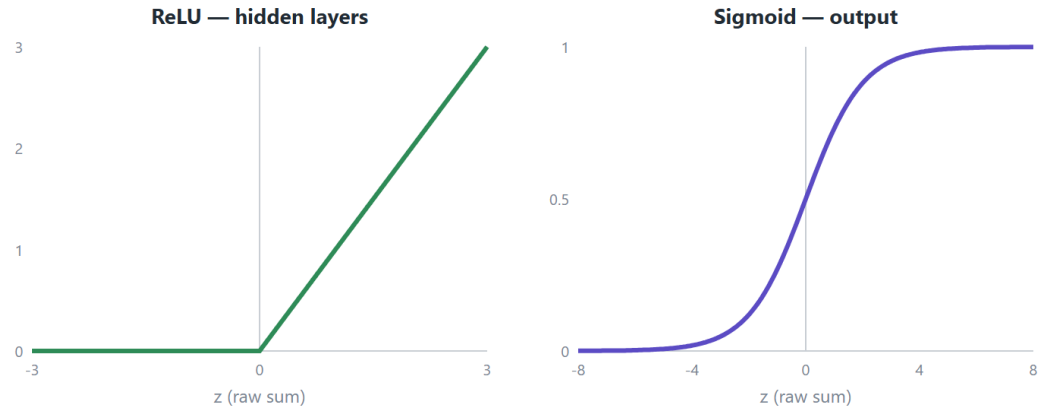


Figure 37.6 — ReLU, used inside the network, and the sigmoid, used on the output.

A step function (1 if z > 0, else 0) would also give an answer between 0 and 1, but it cannot be trained: its slope is zero everywhere, and training corrects each weight in proportion to the slopes along the chain (Chapter 38). A zero slope means no correction ever reaches the weights. The sigmoid has a slope everywhere: `sigmoid'(z) = sigmoid(z) · (1 − sigmoid(z))`.

37.15 WHICH ACTIVATION GOES WHERE

Position	Activation	Why
Hidden layers	ReLU	Prevents the linear collapse; it is only a comparison with zero, so it is cheap.
Output: yes/no	Sigmoid	The answer must read as a probability between 0 and 1.
Output: one of several classes	Softmax	Spreads the probability over the classes so they add up to 1.
Output: a measurement	None	The value must not be squeezed into a range.

Table 37.4 — The activation depends on where the neuron is, not on preference.

FireNet uses all three rules at once. Its hidden convolutions use ReLU. Its output has five values per cell: presence uses a sigmoid (a probability), width and height use a sigmoid (fractions of the image, between 0 and 1), and the two position offsets use no activation at all, because they are measurements that can be negative.

38. How a network learns

So far the network has a structure but random filters — it detects nothing useful. Training is what turns those random numbers into real detectors.

The idea is best pictured as walking down a mountain in fog. The height of the mountain is the model's error. Your position is the current value of all the weights, the filter numbers included. You cannot see the valley, but you can feel the slope under your feet, so you step downhill, feel again, step again — until you reach the bottom.



Figure 38.1 — Training starts at a random point with high error and walks down to the valley.

38.1 ONE TRAINING CYCLE, IN FIVE STEPS

#	STEP	WHAT HAPPENS
1	Forward pass	An image flows through the network with the current weights and produces a prediction. At the start it is nonsense.
2	Loss	The prediction is compared with the truth you annotated, and the gap is measured as a single number, the error. A large error means high on the mountain.
3	Backpropagation	The key step. With millions of weights, how do you know which way to nudge each one? Using derivatives and the chain rule, the network works backwards from the error, layer by layer, computing each weight's exact contribution to it — the slope, for every weight.
4	Gradient descent	Every weight is nudged a little in the direction that reduces the error. How big the nudge is is the learning rate. This is the step downhill.
5	Repeat	The cycle repeats over thousands of images. One full pass over all of them is an epoch; after many epochs the weights settle at low error.

Table 38.1 — The five steps, which turn out to be almost exactly five lines of code.

$$w \leftarrow w - \eta \frac{\partial E}{\partial w}$$

the weight update: slide downhill by the slope, scaled by the learning rate η

38.2 WHAT THE FORWARD PASS ACTUALLY IS

There is nothing magical under the forward pass — it is multiplying matrices. At the lowest level, one layer is:

$$\text{output} = (\text{input} \times \text{weights}) + \text{bias}$$

then ReLU is applied to the output

Where input is the numbers coming in, weights are the filter numbers that training adjusts, and bias is an extra learned number. Each layer multiplies its input by its weights, adds the bias, applies ReLU, and passes the result on.

```
import torch.nn as nn

layer  = nn.Conv2d(3, 32, kernel_size=3)  # 3 channels in (RGB), 32 filters of 3x3
output = layer(image)                     # <- the forward pass of one layer
```

Listing 38.1 — Nobody writes the matrix multiplication by hand; the library has it, optimised.

```
class MyNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3)
        self.conv2 = nn.Conv2d(32, 64, 3)
        self.relu  = nn.ReLU()
        self.pool  = nn.MaxPool2d(2)

    def forward(self, x):                 # <- the forward pass
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        return x
```

Listing 38.2 — A whole network stacks these and passes the image through all of them. The function named forward is literally that.

```
prediction = model(image)                    # 1. forward pass
error      = loss_function(prediction, truth)  # 2. loss
error.backward()                             # 3. backpropagation
optimizer.step()                             # 4. gradient descent
optimizer.zero_grad()                        # reset for the next image
```

Listing 38.3 — error.backward() is the entire backpropagation: the library computes every weight's slope automatically. optimizer.step() is the gradient descent. Wrap it in a loop over all images, repeat for several epochs, and that is training.

38.3 MEASURING THE ERROR

$$E = \frac{1}{n}\sum_{k=1}^{n}(\hat{y}_k - y_k)^2$$

the mean squared error over n values

WHY THE SQUARE AND NOT THE ABSOLUTE VALUE The square makes large errors count disproportionately more than small ones, so the training is pushed hardest where it is most wrong. It is also smooth everywhere, which matters because step 3 needs a derivative at every point — and the absolute value has a corner at zero where that derivative does not exist.

38.4 WHY THE SQUARE: THE SLOPE NEAR THE TARGET

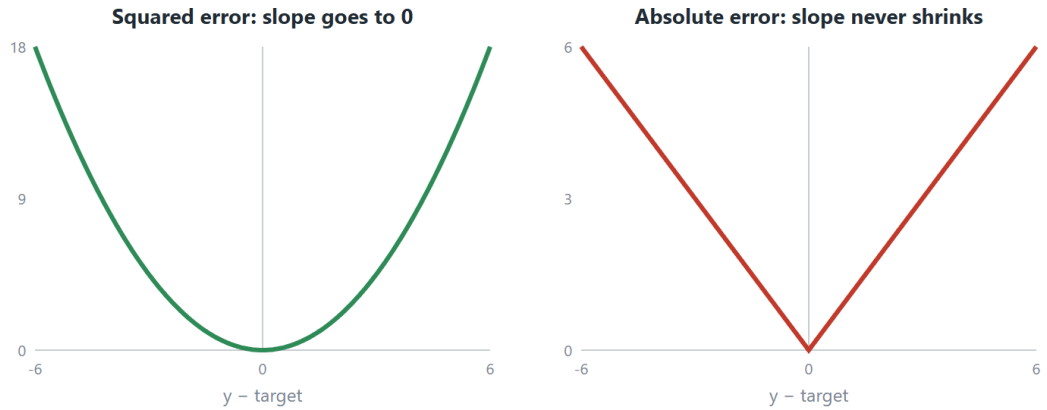


Figure 38.2 — Near the target the squared error flattens out; the absolute error keeps the same slope.

The correction applied to a weight is proportional to the slope of the error. With the square, the slope shrinks as the prediction approaches the target, so the steps shrink and training settles on the target by itself. With the absolute value the slope never changes, so the steps stay the same size and the weights jump back and forth across the target. The square also makes a miss of 10 cost four times a miss of 5, so training attacks the large errors first.

38.5 BACKPROPAGATION WITH NUMBERS

A network of two weights, small enough to follow: input x = 2, hidden h = x·w1, output y = h·w2, with w1 = 3, w2 = 1, and a labelled target of 12.

```
forward:   h = 2 × 3 = 6        y = 6 × 1 = 6
error:     E = 0.5 × (y − target)² = 0.5 × (6 − 12)² = 18

backward (chain rule, one link at a time):
  dE/dy  = y − target          = −6
  dE/dw2 = dE/dy × h           = −6 × 6 = −36
  dE/dh  = dE/dy × w2          = −6 × 1 = −6
  dE/dw1 = dE/dh × x           = −6 × 2 = −12
```

Listing 38.4 — Each weight's share of the error is the product of the slopes along its path.

Both results are negative: raising either weight lowers the error. In FireNet the same chain runs through about 24 000 weights; `error.backward()` computes all of them this way.

38.6 UPDATING THE WEIGHTS, IN SMALL STEPS

```
w_new = w − η × dE/dw                # η = learning rate = 0.01
w1 = 3 − 0.01 × (−12) = 3.12
w2 = 1 − 0.01 × (−36) = 1.36
forward again:  h = 6.24   y = 8.49   E = 0.5 × (8.49 − 12)² = 6.17
```

Listing 38.5 — One step takes the error from 18 to 6.17.

The steps must be small because every weight's correction is computed as if the others stayed still, but all of them move at once. Small steps keep that interference small, and the next step, computed with the new weights, corrects what the previous one disturbed. This is why the learning rate matters and why FireNet halves it when progress stalls.

38.7 WHY THE PIXELS ARE DIVIDED BY 255

A pixel arrives from the file as a whole number from 0 to 255. Each neuron multiplies and adds hundreds of them, so its sum easily reaches tens of thousands. A sigmoid fed such a sum sits flat against 0 or 1, where its slope is practically zero — and a zero slope means no correction, so the network stops learning. Large inputs also make the corrections themselves huge, since each one is proportional to its input. Dividing by 255 puts every value between 0 and 1 without losing anything: the proportions between pixels are exactly the same. FireNet does this in Python and in the C++ on the Pi.

38.8 WHAT A CHANNEL IS

Each pixel has three values: red, green and blue. The same image can be stored as a grid of triples or as three separate grids. OpenCV uses the first, with shape (180, 180, 3); PyTorch and ONNX Runtime want the second, (3, 180, 180). The numbers are identical, only their order changes. A filter scans a pattern in space, so it is faster to read a whole grid of reds in a row, and it is why FireNet's first convolution has a depth of 3: it reads the three grids together.

> **A SILENT BUG** OpenCV also stores colours as BGR, not RGB. Feeding BGR to a network trained on RGB swaps red and blue — exactly the colours that tell a flame from the background. The Pi pipeline converts to RGB before inference for this reason.

38.9 EPOCHS, AND WHY THE IMAGES ARE CHANGED

Each image only nudges the weights a little, so the whole dataset is shown many times; each full pass is an **epoch**. With 50 epochs every image passes through the network 50 times, which gives the network the chance to memorise the exact images instead of learning what a flame looks like. **Data augmentation** removes what there is to memorise: every time an image is loaded it is slightly different

— mirrored, brighter or darker, slightly rotated or cropped — so there is no fixed picture to learn by heart. Chapter 40 lists the exact changes FireNet uses. In the terms of the student in 37.12: augmentation keeps changing the exercises, so the only way to score well is to learn the subject.

> **THE LABELS MUST MOVE TOO** When an image is mirrored, the flame moves to the other side, so its x becomes 1 − x. Forgetting this teaches the network to point to the wrong side of every flipped flame. The same applies to rotations and crops, and a change that would cut a flame out of the image is skipped.

39. From a feature map to a decision

The CNN turns the image into rich, digested feature maps. What reads a decision out of them — fast enough for real time — is the detection head.

Before this approach, detectors were slow: they slid over thousands of image patches, asking “object here? here? here?” one at a time. The one-pass family, of which YOLO (You Only Look Once) is the best known, is named after its idea: look at the whole image a single time and produce all detections at once.

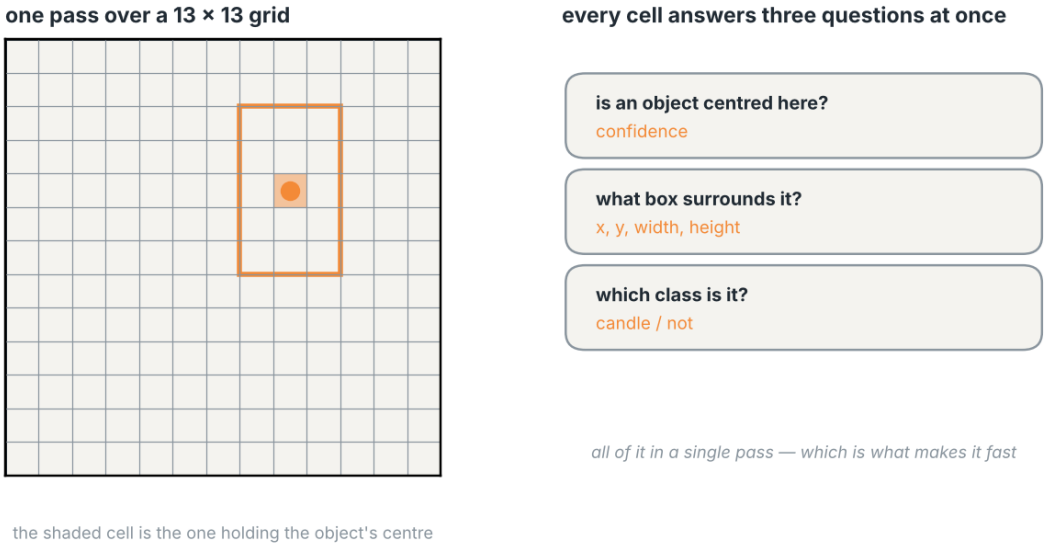


Figure 39.1 — The image is divided into a grid, and every cell answers simultaneously.

The image is divided into a grid — 13×13 cells, say — and passed once through the CNN. For every cell it outputs at the same time: whether an object is centred there, the box that surrounds it, which class it is, and with what confidence. All of it in one pass, which is why it is fast. It does not repeat the analysis a thousand times; it makes one pass and reads every answer off the grid.

39.1 THE PIPELINE, END TO END

- The convolution, ReLU and pooling stack turns pixels into meaning.
- The detection head turns meaning into a decision with a location.
- Every filter in that stack was found by training — walking down the error mountain over thousands of annotated images.

39.2 HOW IT IS ARRANGED IN THIS PROJECT

The setup is deliberately simple, which removes the hardest problems. The car follows a line on the floor — already solved with classic computer vision in Part V, no learning needed. The flames sit at the sides of the line, and there are no obstacles on the line itself.

ELEMENT	ROLE
The OV2640 on the ESP32	Supplies video to the PC for line following, as in Parts II and V.
The camera on the Raspberry Pi	Supplies images to FireNet for flame detection.
On accepting a flame	The Raspberry Pi withdraws the ESP32's permission to move. The car stops on the line; the camera and the hose move on the pan/tilt to aim. The car does not need to leave the line to approach the flame.
Afterwards	The permission to move returns when the aiming logic and the absence of targets allow it.

Table 39.1 — Two cameras, two processing paths. Part IX covers the pan/tilt and the water.

AN HONEST NOTE ON NAMING The grid-based detection explained here shares its ideas with single-pass detectors, but the project's FireNet is not a complete implementation of YOLO.

40. The dataset

Public fire datasets are full of forest fires, burning buildings and stock photos. The robot sees something else: a candle on the floor, 20 to 150 cm away, through a small camera under room lighting, often next to reflections. A network trained on the public images learns the wrong thing, so the dataset for FireNet was recorded with the robot's own camera and annotated with a tool written for the project.

EARLIER VERSIONS The first experiments used the public Flame and D-Fire datasets, a 160×160 input and a dense layer that could only report one flame. They showed that the pipeline worked, and why its data did not match the robot. Everything below describes the version that runs on the Pi.

40.1 FROM A RECORDING TO LABELLED IMAGES

Step	Tool	What happens
Record	`rpicam-vid` on the Pi	One-minute clips at 640×480 and 30 fps, rotated 180° to match the mounted camera. Real scenes, with and without flames, and with the objects that cause false detections.
Extract	`extract_frames.py`	Saves one image every N frames. Every 60 frames (two seconds) avoids filling the dataset with near-identical images.
Annotate	`annotate_images.py`	A box is drawn around each flame. **No flame** saves an empty label, which is information too: it teaches the network what is not a flame.
Finalize	annotator	Copies the session into Datasets/fire_detection, split 80% training and 20% validation, then verifies the copy before deleting the session originals.

Table 40.1 — The four steps from a video to the dataset.

40.2 THE LABEL FORMAT

Each image has a .txt file with the same name. Each line is one flame: a class number and four numbers between 0 and 1 — the centre and the size of the box as fractions of the image. An empty file means the image was checked and has no flame.

```
0 0.512 0.274 0.041 0.083
0 0.731 0.402 0.035 0.070
```

Listing 40.1 — Two flames in one image: class 0, centre x and y, width and height.

40.3 WHY THE SPLIT USES TIME, NOT CHANCE

Two frames taken a second apart are almost the same picture. If they were shuffled at random, one would land in training and its twin in validation, and the validation score would measure memory, not learning. The annotator therefore puts the earlier frames of each session in training and the later ones in validation. Even so, both halves come from the same rooms, so an independent recording is still the only fully honest test.

40.4 FROM THE LABEL TO THE GRID

The network does not output a list of boxes; it outputs a 45×45 grid (Chapter 41). Each label is therefore converted into the same grid before training. The cell containing the flame's centre is marked with presence 1, and stores where inside that cell the centre lies and how large the box is.

```
column = int(x * 45)                  # x = 0.512  ->  column 23
row    = int(y * 45)                  # y = 0.274  ->  row 12
grid[0, row, column] = 1              # a flame is centred here
grid[1, row, column] = x*45 - column - 0.5   # -0.46: left of the cell centre
grid[2, row, column] = y*45 - row - 0.5
grid[3, row, column] = width          # fractions of the image
grid[4, row, column] = height
```

Listing 40.2 — encode_boxes() in data.py: one flame becomes one marked cell.

40.5 AUGMENTATION

During training each image is changed slightly every time it is loaded: brightness and contrast within ±10%, a horizontal flip half of the time, a rotation of up to 5°, and a crop keeping 90–100% of the image. The boxes move with the image. If a rotation or crop would cut an annotated flame, that change is skipped, so a flame image never silently becomes a no-flame image. Validation images are never altered, so scores stay comparable between runs.

40.6 THE CACHE

Decoding hundreds of JPEGs and resizing them on every epoch is slow. `build_cache.py` stores every image already resized to 180×180 together with its labels; training rebuilds it automatically when the dataset or the image size changes.

41. FireNet, training and export

41.1 THE NETWORK

FireNet is deliberately small, because it has to run many times per second on the Pi's CPU. It has no dense layer, so every part of the image is treated the same way, and the output is a grid instead of a single answer.

Layer	Output size	Parameters	What it does
Input	3 × 180 × 180	—	An RGB image, values scaled to 0–1.
Conv 3×3, 16 filters + ReLU	16 × 180 × 180	448	Edges and colour contrasts.
Max-pool 2×2	16 × 90 × 90	—	Halves the size.
Conv 3×3, 32 filters + ReLU	32 × 90 × 90	4 640	Combinations: bright blobs, warm edges.
Max-pool 2×2	32 × 45 × 45	—	Halves the size again: now one value per grid cell.
Conv 3×3, 64 filters + ReLU	64 × 45 × 45	18 496	Flame-like patterns.
Conv 1×1, 5 filters	5 × 45 × 45	325	For each cell: presence, x offset, y offset, width, height.

Table 41.1 — FireNet in full: 23 909 parameters, about 94 kB.

Each output cell looks at an 18×18-pixel patch of the 180×180 input, which is enough to recognise a candle flame at the distances the robot works at. Because each cell answers for its own small region, several flames give several marked cells, and there is no limit of one flame per image.

> **WHY 180×180** A larger input sees smaller flames but costs more time on the Pi; a smaller one loses distant candles. 180 divides cleanly into a 45×45 grid, and the Pi still processes the camera in real time. Changing it means retraining, exporting and changing the matching constant in inference.h.

41.2 THE GRID SHRINKS, EACH CELL SEES MORE

Two opposite things happen as the image moves through FireNet: the grid gets smaller (180 → 90 → 45 cells across), and the part of the original image each cell summarises gets larger. The filter is always 3×3; what grows is how much image each cell underneath it already represents.

After	Grid	Each cell sees
First convolution	180 × 180	3 × 3 pixels
First pooling	90 × 90	4 × 4 pixels
Second convolution	90 × 90	8 × 8 pixels
Second pooling	45 × 45	10 × 10 pixels
Third convolution	45 × 45	18 × 18 pixels

Table 41.2 — The receptive field grows layer by layer.

18 × 18 pixels of a 180 × 180 image is enough to see a candle flame and a little of what surrounds it, which is what lets FireNet tell a flame from a small bright spot.

41.3 THE LOSS: THREE QUESTIONS AT ONCE

Training compares the 5×45×45 prediction with the 5×45×45 label grid and asks three questions, each with its own weight in settings.h:

Part	Weight	What it measures
Presence	FIRE_WEIGHT = 1, NO_FIRE_WEIGHT = 2	Binary cross-entropy on the presence map. Flame cells and empty cells are averaged separately: with 2025 cells and one or two flames, a plain average would let the network score well by saying “no flame” everywhere (the trap described in 41.5). The empty cells weigh double because false alarms stop the car.
Position	POSITION_WEIGHT = 1	Mean squared error of the offsets and the box size, counted only in cells that really contain a flame.
Count	COUNT_WEIGHT = 3	The presence probabilities of a whole image are summed into an estimated number of flames and compared with the real number. It pushes against both duplicate detections and missed flames.

Table 41.3 — The three parts of the loss. Total = presence + 1 × position + 3 × count.

41.4 FROM THE GRID BACK TO BOXES

After training, the same grid is read the other way. A cell whose presence, after a sigmoid, is above `CONFIDENCE_THRESHOLD` = 0.65 becomes a box: the centre is the cell position plus the predicted offset, and the size comes from the last two values. Neighbouring cells often fire for the same flame, so non-maximum suppression (NMS) keeps the most confident box and removes any other box that overlaps it by more than 30% (IoU 0.3). The Python evaluation and the C++ program on the Pi apply exactly the same rules.

41.5 MEASURING: WHY A SINGLE PERCENTAGE LIES

If 85% of the validation images had no flame, a network that never detected anything would be 85% “accurate”. The score used here counts flames instead of images. A predicted box is correct when its centre is within `MAX_DISTANCE` = 10 px (at 180×180) of a real flame that has not been matched yet.

- **Precision** — of the boxes predicted, how many were real flames.
- **Recall** — of the real flames, how many were found.
- **F1** — 2 × precision × recall / (precision + recall), the single score that saves the best checkpoint. It cannot be high unless both are.

The training script also reports the average position error in pixels and how many no-flame images stayed silent, because a model can find every flame and still raise false alarms on empty floors.

41.6 THE TRAINING RUN

Setting	Value	Why
Optimiser	Adam	Adapts the step for each weight; robust without much tuning.
Learning rate	0.001, halved after 5 epochs without a new best, minimum 0.001/32	Large steps early, finer steps once progress stalls.
Epochs	50	Only a better validation F1 replaces the saved model, so extra epochs cannot make it worse.
Batch	32 images	Averages the correction over several images.
Start	saved weights, or new ones with --fresh	New recordings refine the existing model instead of starting from zero.

Table 41.4 — Training settings, all read from settings.h.

In one recorded experiment the network reached a validation F1 of 81.98% after a single epoch, with this threshold and NMS. That figure is from one run on the current dataset; it is not a guarantee for new scenes.

41.7 EXPORT TO THE PI

The Pi does not run PyTorch. `model_converter.py` loads the best checkpoint and writes it to ONNX, an open format that ONNX Runtime executes in C++. The input is named `image` and the output `predictions`, and the C++ code must use the same names and the same 1×3×180×180 shape.

```
torch.onnx.export(model, torch.zeros(1, 3, 180, 180), "firenet.onnx",
                  input_names=["image"], output_names=["predictions"],
                  opset_version=17)
```

Listing 41.1 — The export. The ONNX file holds the network only; threshold and NMS live in settings.h.

ONE SETTINGS FILE The confidence threshold, NMS and training values are written once, in firmware/raspberry/settings.h. The C++ program includes it; the Python tools read the same file. The PC evaluation and the robot can therefore never disagree about what counts as a flame.

PART IX

The Raspberry Pi and putting the fire out

Part VIII ends with a model that can say “flame, here”. This part is everything that turns that answer into water on the flame: the signals that drive the servos, how far the jet can reach, the program that runs on the Pi, the wire that stops the car, the aiming, the spraying, and what the finished system can and cannot do.

> **WHY THIS PART** FireNet can say “flame, here”. Turning that into water on the flame needs hardware that can aim and spray, and a program on the Raspberry Pi that stops the car, aims and decides when to fire.

42. PWM on the Raspberry Pi 5

The pan/tilt mechanism needs two servo signals, and the pump needs switching. Chapter 2 generated PWM on the ESP32 with LEDC, in a few lines. On the Pi it is a different mechanism entirely, because Linux sits between the program and the hardware.

The Raspberry Pi 5 has a chip called RP1, which includes the PWM0 peripheral. That peripheral generates the pulses automatically once the program has configured it — the same hardware-does-the-work principle as the LEDC in Chapter 2.

42.1 HOW LINUX EXPOSES IT

Linux presents the controller as a folder named `pwmchipN` inside `/sys/class/pwm/`. The N identifies the controller to Linux, not a channel — and the PWM0 peripheral is not necessarily named pwmchip0, so the matching folder has to be identified on the system.

ELEMENT	ROLE IN THIS PROJECT
`pwmchipN`	The controller exposed by Linux. N must be identified on the system.
Channel 2	PWM for GPIO18 (pan)
Channel 3	PWM for GPIO19 (tilt)

Table 42.1 — The pin-to-channel correspondence is fixed by the hardware: GPIO18 is physical pin 12 and channel 2; GPIO19 is physical pin 35 and channel 3.

> **TWO DIFFERENT NUMBERS, EASILY CONFUSED** The channel associated with a GPIO is a hardware correspondence — GPIO18 to channel 2, GPIO19 to channel 3. The number in pwmchipN identifies where Linux made the controller available, and it has to be looked up.

42.2 THE FOUR CONFIGURATION STEPS

#	STEP	WHAT IT DOES
1	Configure the overlay and reboot	The overlay in /boot/firmware/config.txt enables the peripheral and configures the pins to carry the PWM outputs.
2	Identify pwmchipN	Find where Linux made the PWM0 peripheral available.
3	Export channels 2 and 3	Write the channel numbers into the special export file. That makes the pwm2 and pwm3 folders appear.
4	Configure each channel	Write the wanted values into the special files period, duty_cycle and enable.

Table 42.2 — The structure after exporting a channel.

```
/sys/class/pwm/pwmchipN/
    export
    unexport
    npwm
    pwm2/
        period
        duty_cycle
        polarity
        enable
```

Listing 42.1 — These are special Linux files, not documents stored on the card. When the program writes to them, the driver configures the peripheral.

- The overlay is stored in config.txt, so the Pi reapplies it on every reboot.
- The /sys folders are recreated by Linux: pwmchipN appears when the controller is made available, and pwm2 and pwm3 appear after the channels are exported. They normally have to be exported again after a reboot, which the program does.

42.3 FINDING AND CONFIRMING THE PINS

```
ls -l /sys/class/pwm/        # lists the available controllers and where they point

pinctrl get 18-19            # on the Pi 5, shows the current function of the pins
```

Listing 42.2 — If the overlay configured those pins for PWM, the expected output is GPIO18 = PWM0_CHAN2 and GPIO19 = PWM0_CHAN3 — meaning the PWM0 peripheral, channels 2 and 3.

```
grep -A 20 "Name:   pwm-2chan" /boot/firmware/overlays/README
```

Listing 42.3 — Reading the official overlay manual that ships with the Pi.

PIECE	WHAT IT IS
`grep`	Searches for a text inside a file and shows the line where it finds it.
`-A 20`	Also shows the 20 lines after that one — A for After.
`"Name: pwm-2chan"`	The text to search for. It is the line where this overlay's section begins in the manual.
`/boot/.../README`	The official manual of every overlay, installed on the Pi.

Table 42.3 — Why it is used this way: the manual holds hundreds of overlays, and this shows only the pwm-2chan part.

The lines that matter in what it returns are the legal pin-and-function combinations. `18,2(Alt5)` means that for pin 18 to act as PWM, `func` has to be 2; `19,2(Alt5)` means the same for pin 19 with `func2`.

43. The water system: how far the jet reaches

Aiming is only half the problem. The other half is knowing whether the water will arrive at all — and that is a question with a closed-form answer, derived here from the pump's flow rate.

43.1 WHAT THE RANGE DEPENDS ON

- v — the speed at which the water leaves the nozzle.
- θ — the angle of the nozzle above the horizontal.
- Δh — the height difference between the nozzle and the target: the nozzle sits 20 cm above the floor, where the candle is.

> **AND THE REAL LOSSES** Beyond those three there are real losses — air drag and the break-up of the jet — which reduce the ideal range. Section 43.7 is about them.

43.2 GETTING v FROM THE FLOW RATE Q

The water leaving the nozzle can be seen as a cylinder of water advancing: at each instant a disc of water of area A, the nozzle's cross-section, moves at speed v. In one second that disc travels a distance v, forming a cylinder of length v and area A. The volume of that cylinder is:

$$Q = A\,v$$

and since this happens in one second, that volume is exactly the flow rate Q

This follows from the conservation of mass: water is incompressible, so every volume that enters the nozzle per second has to leave per second — it can neither accumulate nor vanish inside. Isolating v:

$$v = \frac{Q}{A} \qquad\qquad A = \frac{\pi d^2}{4}$$

A from the circle-area formula with r = d/2, where d is the nozzle's internal diameter

43.3 WHY v = Q/A IS NOT THE ANSWER

That expression gives only the exit speed — one number, how fast the water comes out. It says nothing about how far the water gets. To know that, the speed has to be launched at an angle θ and then follow a trajectory through the air, and it is gravity that curves that trajectory. Without gravity the water would travel in a straight line indefinitely; gravity is what fixes the instant and the point at which it reaches the target's level, and therefore the horizontal distance covered.

STEP	WHAT IT PRODUCES
1	v decomposed into vx and vy, using θ — the speed in each direction.
2	vy under gravity — the vertical velocity as a function of time.
3	from vy(t) to the vertical position y(t).
4	the condition y = −0.20 m — the time of flight.
5	vx × time of flight — the horizontal distance, which is the range R.

Table 43.1 — The full chain. Each step is derived below.

43.4 STEPS 1 AND 2 — DECOMPOSITION, AND GRAVITY

v is a vector launched at an angle θ above the horizontal, so it decomposes by trigonometry: θ is the angle between v and the horizontal, and v is the hypotenuse of a right triangle whose legs are vx and vy.

$$v_x = v\cos\theta \qquad\qquad v_y = v\sin\theta$$

directly from the definitions of cosine and sine in that triangle

For the vertical component, the starting point is Newton's second law: the sum of the forces on a body equals its mass times its acceleration. With drag assumed negligible, the only vertical force is the weight, so the mass cancels:

$$\sum F = m\,a \qquad \Longrightarrow \qquad a_y = -g$$

with the y axis positive upwards and its origin at the nozzle, so y(0) = 0

$$v_y(t) = v_y(0) - g\,t$$

the vertical velocity at any instant of the flight

43.5 STEP 3 — FROM VELOCITY TO POSITION

WHY THIS STEP IS NEEDED AT ALL The flight does not end on any condition about velocity. The water does not stop when vy reaches zero — at that instant it is merely at its highest point, and it carries on falling afterwards. The flight ends on a condition about position: the instant the water reaches the candle's level.

Average velocity is displacement divided by elapsed time. Applied between instant 0 and a general instant t, with the origin at the nozzle so that y(0) = 0, the displacement is simply y(t), which gives y(t) = v_y,med × t. But v_y,med is still unknown — and that is where step 2 comes in.

$$v_{y,\,med} = \frac{v_y(0) + v_y(t)}{2} = v_y(0) - \frac{g\,t}{2}$$

in a linear variation, the average is the arithmetic mean of the start and end values

That mean is valid because the velocity is a straight line in time: as much time is spent above the average value as below it, and the deviations cancel exactly. This would be false for any non-linear variation. Substituting it back:

$$y(t) = v\sin\theta\,t - \tfrac{1}{2}\,g\,t^2$$

the height of the water above the nozzle at any instant of the flight

43.6 STEPS 4 AND 5 — TIME OF FLIGHT, AND RANGE

The time of flight comes from no new formula. It comes from imposing on y(t) the physical condition that marks the end of the trajectory: the nozzle is 20 cm above the floor and the origin is at the nozzle, so the floor is at y = −0.20 m.

$$v\sin\theta\,t_v - \tfrac{1}{2}\,g\,t_v^2 = -0.20$$

v, θ and g are known; the only unknown is the time of flight

The horizontal step is short, because no force acts horizontally. The weight is vertical and the drag was assumed negligible, so the sum of horizontal forces is zero, so the horizontal acceleration is zero, so vx does not change during the whole flight. When velocity is constant, distance is simply velocity times time — no average velocity and no quadratic needed:

$$R = v_x\,t_v$$

this is where the two axes, treated separately until now, rejoin: what links them is the time

With the example values — v ≈ 3.93 m/s at θ = 20°, giving vx ≈ 3.69 m/s and a time of flight of about 0.380 s:

$$R \approx 3.69 \times 0.380 \approx 1.40\ \text{m}$$

at 20°, the ideal model predicts the water reaches the candle's level about 1.40 m from the nozzle

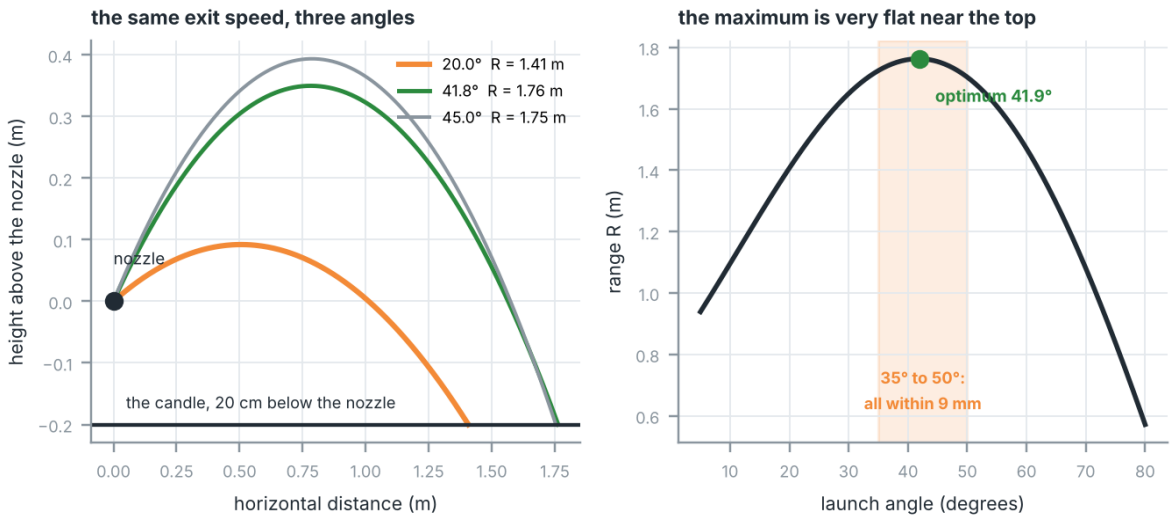


Figure 43.1 — Computed from the equations above, for an exit speed of 3.93 m/s and a 20 cm drop.

43.7 THE OPTIMUM ANGLE IS NOT 45°

45° is only optimal when the launch and the arrival are at the same height, which is not the case here: the nozzle is 20 cm above the candle.

The angle controls a trade-off. Raising it increases vy(0), which increases the time of flight; but at the same time it decreases vx, and the range is the product of the two. The optimum is where those two effects balance. The height difference breaks that balance in favour of lower angles: the extra 20 cm of fall already provides flight time at no cost in vx, so it pays less to buy time with angle and more to preserve horizontal speed.

$$\theta_{ot} = \arctan\left(\frac{v}{\sqrt{v^2 + 2gh}}\right)$$

obtained by writing R as a function of θ and setting its derivative to zero; h is the nozzle height above the target

ANGLE	RANGE
41.8° (the optimum)	1.763 m
45°	1.754 m
The difference	9 millimetres

Table 43.2 — The maximum is very flat near the top.

> **THE PRACTICAL CONSEQUENCE** Any angle between 35° and 50° gives practically the same range, so there is no point tuning the nozzle mount to the degree. What changes the range significantly is v — that is, the pump's flow rate and the nozzle diameter.

43.8 HOW TO INCREASE THE RANGE

Going back to the start of the chain and combining v = Q/A with A = πd²/4:

$$v = \frac{4Q}{\pi d^2}$$

If the flow rate Q were held constant, the speed would grow as 1/d². For the ideal model with a fixed angle and equal launch and arrival heights — or as an approximation when the height difference is small — that gives:

$$R \propto \frac{Q^2}{d^4}$$

under those assumptions, doubling Q would multiply the range by four, and halving d by sixteen

THESE FACTORS ARE NOT A PREDICTION Narrowing the nozzle changes the flow rate and the losses, and the height difference also alters the relationship. They indicate what to adjust, not what will happen.

BRAKE	WHY IT LIMITS THE GAIN
The pump curve	v = Q/A assumes Q stays the same when the nozzle is narrowed. That is false. A narrower nozzle offers more resistance to the flow, which raises the pressure at the pump outlet, and no real pump maintains its flow rate against rising pressure — the flow falls. Every pump has a characteristic curve relating the flow it can deliver to the pressure it works against. So there is an optimum diameter, where the gain in $1/d^2$ stops outweighing the loss in Q. That point depends on the specific pump and is found from the manufacturer's curve or experimentally — it does not come out of this derivation.
Jet coherence	Higher speed with a narrower nozzle encourages the jet to break into droplets. Small drops have a lot of area for very little mass, and air drag slows them quickly. Past a certain point, increasing v stops increasing the real range, because the water stops arriving as a jet and starts arriving as spray.

Table 43.3 — The two brakes on the simple proportionality.

43.9 THE LEVERS, IN ORDER OF EFFECT

LEVER	EFFECT
Nozzle diameter	The strongest effect, limited by the pump curve and by atomisation.
Flow rate	Test a suitable pump, or adjust the supply only within the limits specified for the pump and the circuit.
Angle	Almost no effect near the optimum — about 9 mm between 41.8° and 45°.

Table 43.4 — The model says what to adjust; the final value has to be measured.

THE LIMITS OF THE WHOLE MODEL Every value above is an ideal range, computed without air drag. With a 3 mm nozzle at a low flow rate, the jet tends to break into droplets, and the real range will be shorter than calculated. How much shorter depends on whether the jet leaves coherent or atomised, and that has no simple closed formula — it is measured, not calculated. The practical step is to build the system and compare the real range with two or three nozzles of different diameters.

44. The program on the Raspberry Pi

Everything on the Pi is one C++ program, `firenet`. It runs from an SSH terminal without a monitor, and each pass of its main loop handles one camera frame from capture to action.

Stage	File	What happens
Capture	camera.cpp	libcamera delivers 640×480 NV12 frames at 30 fps through GStreamer. They are converted for OpenCV and rotated 180°, because the camera is mounted upside down.
Prepare	main.cpp	A copy is resized to 180×180 RGB for the model; the full image is kept for the video.
Infer	inference.cpp	ONNX Runtime runs FireNet on a 1×3×180×180 tensor and returns the 45×45 grid.
Decode	main.cpp, detection_filter.cpp	Sigmoid, threshold 0.65 and NMS turn the grid into flame boxes, exactly as in Python (41.4).
Decide	car_control.cpp, aim_controller.cpp	Should the car stop? Which flame? Where should the servos go? Is it time to spray?
Act	servos.cpp, pump.cpp	Hardware PWM moves pan and tilt; a timed GPIO pulse runs the pump.
Show	video_server.cpp	Boxes and status are drawn on the frame, which is served as MJPEG at http://<pi>:8080/.

Table 44.1 — One frame through the program.

Three start-up modes keep testing safe:

```
./firenet                                  # video only, car never released
./firenet --aim 1450 1400 -1 -1            # stop the car and aim, no water
./firenet --aim 1450 1400 -1 -1 --pump     # aim and spray
```

Listing 44.1 — The two numbers after --aim are the servo home pulses; the next two are the directions of pan and tilt.

> **WHY THE BROWSER VIDEO** The Pi has no screen on the robot. The MJPEG server keeps only the newest frame and serves up to four viewers, so a slow phone never delays the detection loop. Closing the browser does not stop anything.

45. Stopping the car: the GO/STOP wire

The Pi and the ESP32 talk through a single wire: Pi GPIO23 to ESP32 GPIO2, with a shared ground. HIGH means the car may obey the PC; LOW means the motors stay stopped, whatever the PC sends.

45.1 WHY A WIRE AND NOT A NETWORK MESSAGE

- It cannot be lost, delayed or reordered like a UDP packet.
- The ESP32 input has a pull-down resistor, so a disconnected wire or a Pi that has not started reads as STOP. The safe state needs no software at all.
- It needs no protocol: the ESP32 checks one pin before every motor update.

45.2 WHEN THE PI PULLS IT LOW, AND WHEN IT LETS GO

Event	Rule
Stop	A flame in 2 consecutive frames (CAR_FLAME_CONFIRM_FRAMES). One isolated false detection does not stop the car.
Settle	Wait 0.5 s (CAR_SETTLE_SECONDS) so the chassis stops rocking; only frames captured after that may confirm a target.
Hold	Stay stopped while aiming, spraying, returning the camera home and settling at home.
Release	At least 5 frames without a flame, spanning at least 1 s, with the camera home and no pump pulse running.
Safety	A processing gap longer than 1 s, a normal exit or any reported error sets the wire LOW.

Table 45.1 — The stop and release rules (normal settings).

The ESP32 remembers the last command the PC sent while the wire is LOW. When the wire goes HIGH again and the PC connection is still fresh, the car resumes that command without the driver touching the controller.

> **LIMITATION** The wire is a level, not a heartbeat. If the Pi program froze while the wire was HIGH, the ESP32 would not notice. The ESP32's own PC watchdog still stops the car if the PC goes silent.

46. Aiming the nozzle

The aiming is a small state machine in aim_controller.cpp. Each frame it is in exactly one state:

State	Shown in the video	What it does
Search	Confirming flame	Camera at home. Picks the flame nearest the image centre and confirms it in 4 of the last 6 frames.
Track	Aiming	Moves pan and tilt step by step, waiting 0.25 s after each step for the image to settle.
Hold	Target centered	The flame is within ±1% of the aim point; spraying is allowed (Chapter 47).
Return	Returning to search position	The target has been missing for 3 s: the servos walk home in small steps.
Cooldown	Settling	Waits 2 s at home before searching again.

Table 46.1 — The aiming states.

46.1 HOW FAR TO MOVE

The error is the distance, as a fraction of the image, between the flame and the aim point. It becomes a change in servo pulse width: proportional to the error, but capped, so one bad detection cannot swing the nozzle across the room. Errors above 15% of the image use a larger gain and cap.

Speed level	Small errors (gain / max step)	Large errors (gain / max step)
1	60 / 12 µs	60 / 12 µs
2	60 / 12 µs	120 / 24 µs
3 (normal)	90 / 18 µs	120 / 24 µs
4 (fast test mode)	120 / 24 µs	180 / 36 µs

Table 46.2 — AIM_SPEED_LEVEL. On an SG90, 1000 µs covers about 180°, so 12 µs is roughly 2°.

46.2 WHERE TO AIM: THE CALIBRATION TABLE

The nozzle is not where the camera is, and the water falls on its way. Centring the flame in the image would therefore miss. Instead, the flame is steered to a different point of the image, chosen from how high the flame was in the image before the camera moved. A flame high in the image is far away and needs the jet aimed higher.

Initial flame Y	Target X	Target Y
0.0102	0.540	0.440
0.0200	0.560	0.470
0.0298	0.530	0.440
0.0400	0.530	0.440
0.0509	0.535	0.482
0.0555	0.530	0.500
0.0570	0.542	0.500
0.0756	0.546	0.500
0.0807	0.560	0.470
0.1004	0.549	0.466
0.1219	0.559	0.470
0.1244	0.555	0.477
0.1448	0.558	0.486
0.2129	0.566	0.479
0.2339	0.548	0.485
0.2821	0.568	0.484
0.3657	0.590	0.500
0.4346	0.550	0.490
0.4559	0.550	0.490

Table 46.3 — AIM_CALIBRATION, measured with candles on the floor. Between rows the target is interpolated; HORIZONTAL_AIM_ADJUSTMENT (+0.02) and VERTICAL_AIM_ADJUSTMENT (+0.01) are added to every row.

Each row came from a test: run, watch where the jet lands, correct that row, rebuild. If the jet passes to the right of the flame, the target X goes up; if it passes above, the target Y goes down. The table is an empirical fit for one mounting and one candle height, not a distance measurement, and remounting the nozzle invalidates it.

46.3 TARGETS THAT CANNOT BE REACHED

If a correction is still needed but a servo is already at its limit (1000 or 2000 µs), the controller does not spray at the wrong place. It reports “Aim unreachable”, returns home and ignores that target, following its position as the car moves, until it has been absent for 2 s.

47. Spraying, and the reflections it creates

47.1 THE WATER PULSE

- The target must stay centred for 3 frames and at least 0.5 s before water is requested.

- GPIO17 drives the gate of an IRLZ44 logic-level MOSFET, which switches the pump from the battery.
- The pulse length (PUMP_SPRAY_SECONDS) is timed by the lgpio library, independently of the frame loop, so a slow frame cannot lengthen it.
- During the pulse both servos are frozen and the car stays stopped.
- After the pulse ends, the next attempt waits 10 s, and each target gets at most 3 attempts. Returning home starts a new cycle.

> **A MISSING FLAME IS NOT A DEAD FLAME** When the detection disappears after a pulse, the flame may be out, or hidden by steam, or simply missed for a frame. The program does not claim success; it waits, and returns home only after 3 s without the target.

47.2 REFLECTIONS ON WET FLOORS

Water on the floor reflects the flame, and FireNet sees the reflection as a second flame right next to the one it just sprayed. Without a filter, the car would stop again for the reflection the moment it was released. After a pulse, once the camera is home, new detections within 0.15 of the sprayed position (REFLECTION_IGNORE_RADIUS) are ignored unless they match a flame that was already there before spraying. The filter ends 2.5 s after the car is allowed to move again.

> **LIMITATION** This is position matching, not recognition. A real second flame very close to the first can be ignored for those few seconds, and a reflection that appears farther away is not filtered.

48. One settings file, and the fast test mode

Every value that changes behaviour lives in firmware/raspberry/settings.h, with a comment saying what raising or lowering it does. The C++ program includes it and the Python training tools read the same file, so a value is never written twice. Only measured data (the calibration table), pin numbers and the home pulses live elsewhere.

`FAST_TEST_MODE = true` replaces the timing values with a faster preset for testing, while the confidence, calibration, spray length and attempt limit stay the same:

Value	Normal	Fast test mode
Frames to stop the car	2	1
Settle time after stopping	0.5 s	0.15 s
Detections to confirm a target	4 of 6	1
Aiming speed level	3	4
Wait after each servo step	0.25 s	0.08 s
Centred before spraying	3 frames, 0.5 s	1 frame, 0.1 s
Settle at home	2 s	0.3 s
Wait between attempts	10 s	2 s
Clear frames to release the car	5 over 1 s	2 over 0.3 s

Table 48.1 — Faster reactions, at the cost of more blur, overshoot and false stops.

49. Results and limitations of Phase II

49.1 WHAT WORKS

- The car follows the line, stops for a candle flame, aims and sprays it, and resumes the line on its own.
- FireNet runs in real time on the Pi's CPU from a model of about 94 kB trained on the robot's own images.
- The stop wire, the shared settings file and the start-up modes make each stage testable on its own.

49.2 LIMITATIONS

Limitation	Detail
Calibration	The aim table is empirical and valid for one mounting, one home position and one candle height.
No position feedback	The servos report nothing back; a jam or a mechanical stop cannot be detected.
Look-alikes	Bright objects, sunlight and reflections can still be detected as flames; NMS removes duplicates, not mistakes.
Single wire	The stop wire is a level, not a heartbeat; a frozen Pi could leave the car allowed to move.
Fire safety	This is a candle-scale prototype, not a fire-safety device.

Table 49.1 — What the current system does not do.

Appendix A. Technical reference notes

These notes clarify concepts used throughout the report: API contracts, communication protocols and power connections. Where they correct something stated earlier in an older source, that is said explicitly.

A.1 NETWORK ARCHITECTURE — THE CURRENT PORT MAP

PATH	PROTOCOL	PORT	NOTE
ESP32 control	UDP	1883	Movement commands
ESP32 video	UDP	1884	JPEG split into datagrams
Wi-Fi provisioning	TCP	1883	WIFI: and RESET:NOW, in access-point mode only
Raspberry Pi stream	HTTP / TCP	8080	A separate browser stream

Table A.1 — The current firmware. Earlier TCP control and HTTP camera examples in this report are historical development stages, and are labelled as such where they appear.

WHAT THE UDP VIDEO PROTOCOL DOES AND DOES NOT DO It splits the JPEG data into datagrams and the PC reconstructs frames using the JPEG markers of Chapter 8. It does not provide reliable delivery or explicit packet sequencing — which is the deliberate trade-off of Chapter 24.

A.2 NVS — THE API CONTRACT, CORRECTED

NVS holds persistent key–value pairs in an ESP32 flash partition. The namespace identifies a logical grouping of keys, such as `wifi_count` and `wifi_0`.

- `nvs_open()` returns an opaque handle to an open namespace in that partition. It is not a documented physical flash address and should not be described as a pointer directly locating stored entries.
- `nvs_get_u32()` and `nvs_get_blob()` retrieve values by key; `nvs_set_u32()` and `nvs_set_blob()` update the stored record.
- After a successful set, `nvs_commit()` is required to ensure pending changes have reached non-volatile storage, followed by `nvs_close()` when finished.

THE CORRECTION The library can write data before commit. The claim that all set operations remain only in RAM until commit is not guaranteed by the API. The underlying design is log structured, with new versions of values written to available entries and stale entries eventually reclaimed.

Source: Espressif ESP-IDF Non-Volatile Storage Library documentation, docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/storage/nvs_flash.html, and the ESP-IDF nvs.h API declaration.

A.3 POWER WIRING

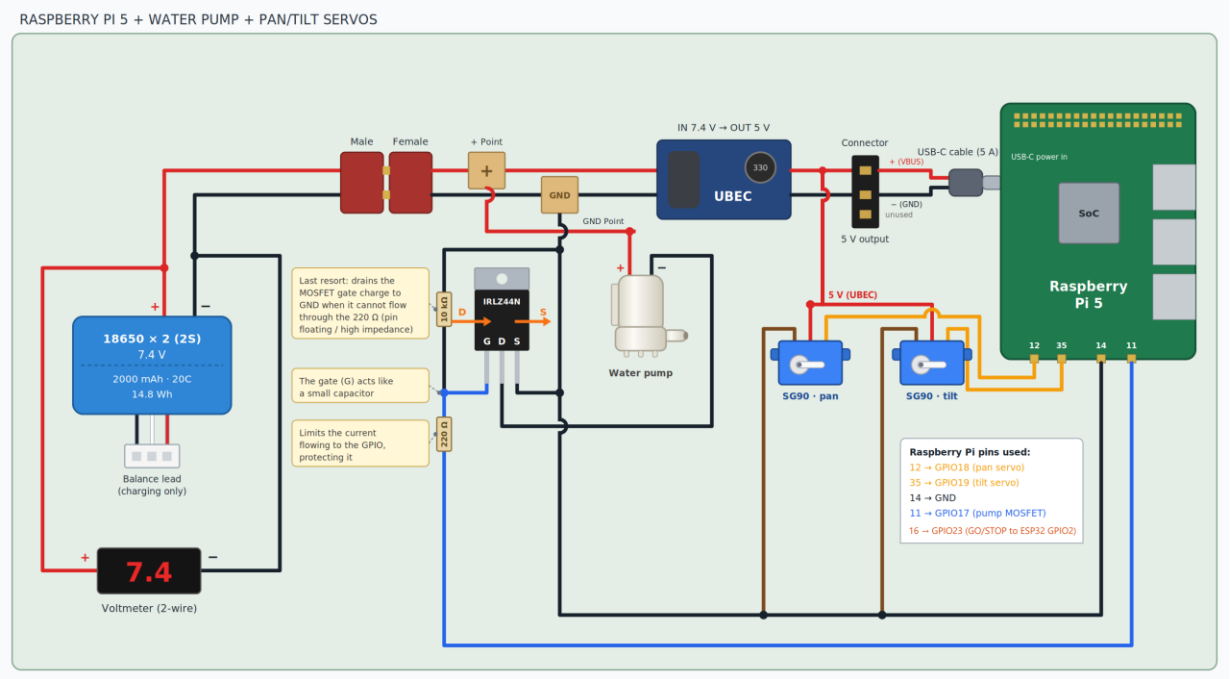


Figure A.1 — Raspberry Pi 5, water pump and pan/tilt servos. The servos take 5 V from the UBEC output; the pump runs from the 7.4 V battery through the IRLZ44.

- Both SG90 servos are powered from the regulated 5 V output of the UBEC, never directly from the 7.4 V battery (8.4 V when fully charged).
- The servo grounds are common with the Raspberry Pi; the PWM pins only carry the signal.
- The UBEC must supply the Raspberry Pi and both servos at the same time; its rated current has to cover that combined peak.
- The pump is an inductive load switched by an IRLZ44 MOSFET. A flyback diode across the pump should be fitted to protect the MOSFET from the voltage spike when it switches off.

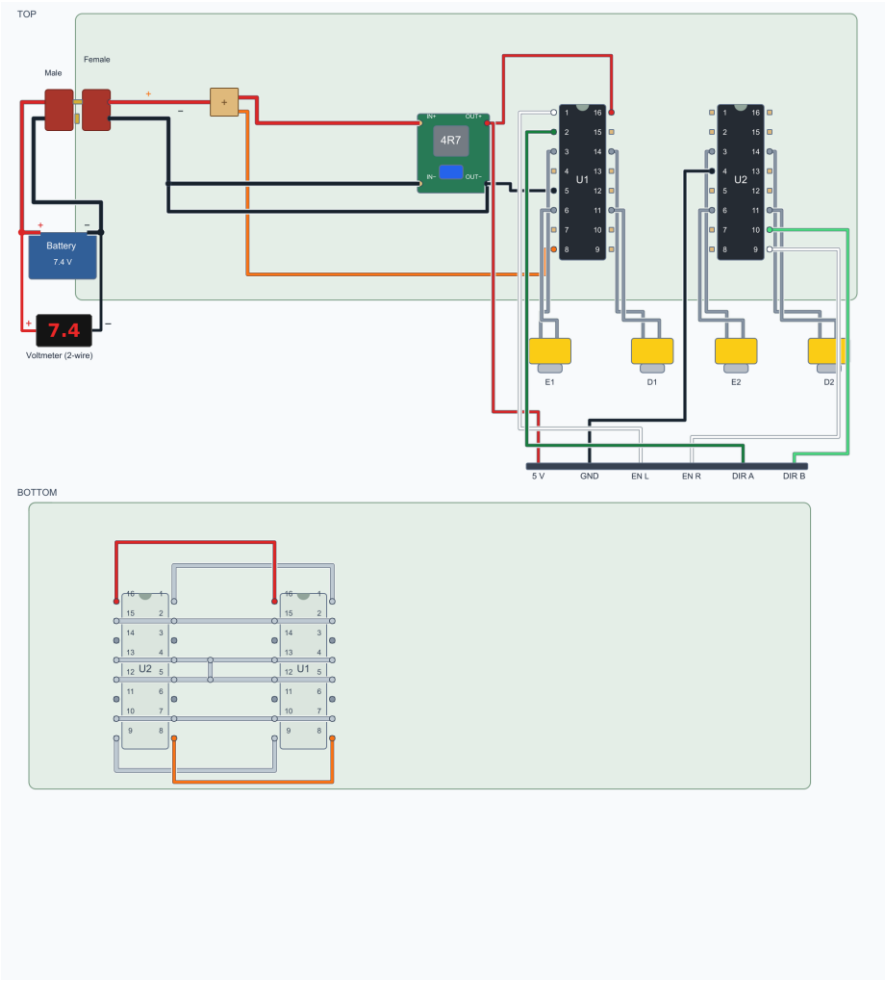


Figure A.2 — The two L293D drivers on both sides of the protoboard, with the 5 V step-down and the voltmeter.

Sources: TowerPro SG90 servo specifications; Vishay IRLZ44 datasheet; AI-Thinker ESP32-CAM schematic.

A.4 CLOSING NOTE

I learned a great deal from this project, and it brought far more challenges than I expected. It was the thing I enjoyed most and, at the same time, the thing I hated most.

The biggest lesson was that problems appear where you would never expect them. Ironically, two of the things that took the most work were not in the code: the type of tape on the floor — if it reflected too much light, the car could not detect the line — and, above all, the angle between the camera and the hose.